



APRENDIZAJE DE MÁQUINA

CLASIFICADOR KNN Y OPTIMIZACIÓN DE HIPERPARÁMETROS

Especialización en Inteligencia Artificial
FIUBA

Dr. Ing. Facundo Adrián Lucianna
Esp. Lic. María Carina Roldán

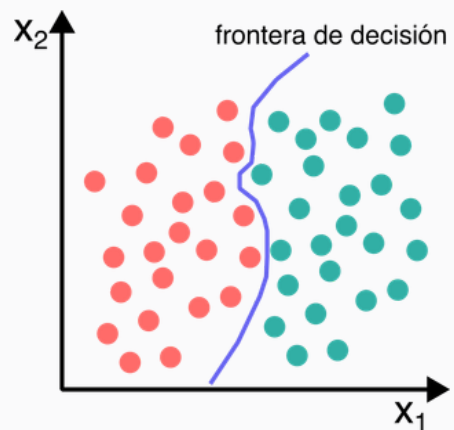
En la clase anterior vimos ...

Introducción a Machine Learning. Definiciones. Ciclo de vida de un proyecto de Aprendizaje Automático. Metodología para construir modelos.

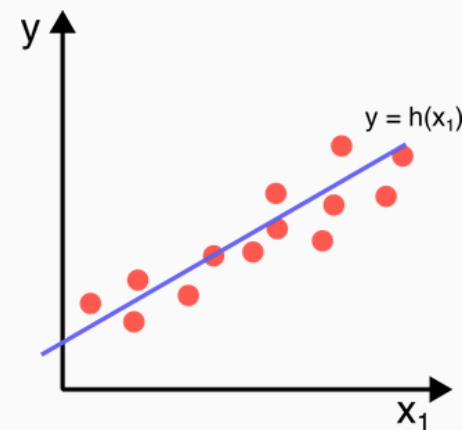
Repaso de la clase anterior

- Definición de Machine Learning
- Tipos de aprendizaje

SUPERVISADO

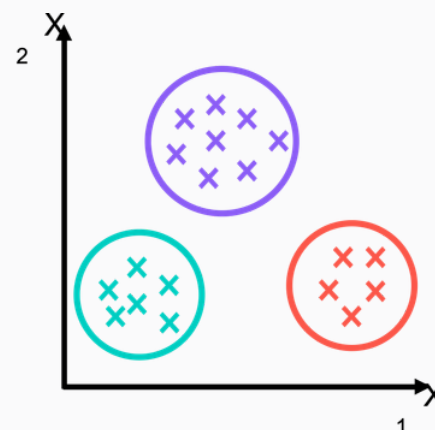


Clasificación

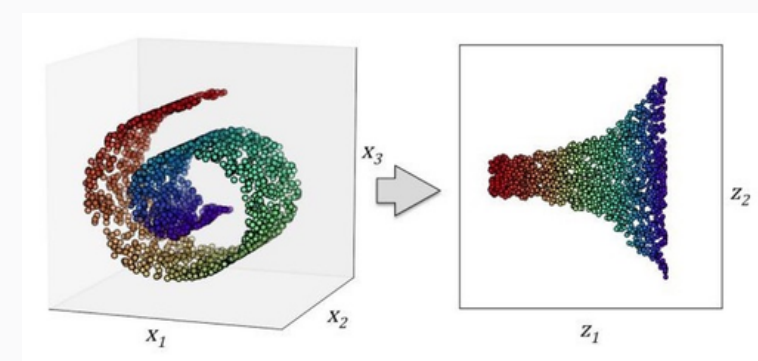


Regresión

NO SUPERVISADO

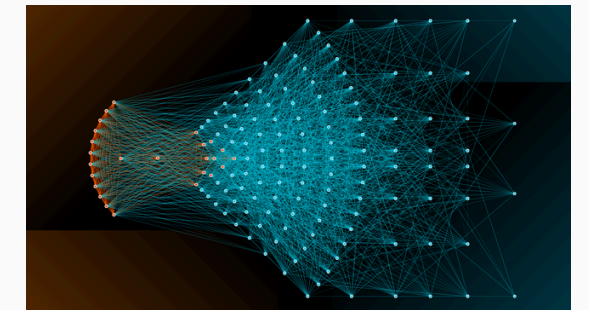


Clustering



Reducción dimensional

APRENDIZAJE PROFUNDO



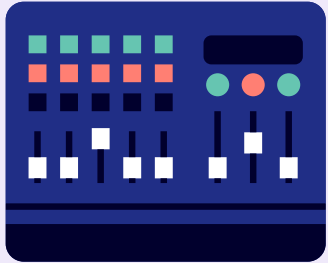
Redes neuronales

Repaso de la clase anterior

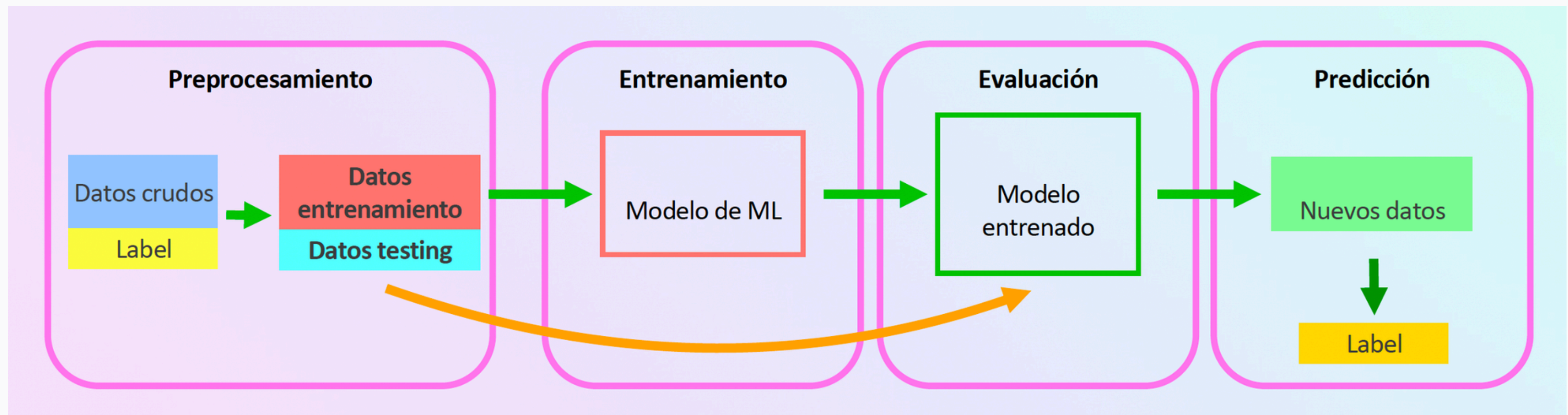
features					label
Position	Experienc	Skill	Country	City	Salary (\$)
Developer	0	1	USA	New York	103100
Developer	1	1	USA	New York	104900
Developer	2	1	USA	New York	106800
Developer	3	1	USA	New York	108700
Developer	4	1	USA	New York	110400
Developer	5	1	USA	New York	112300
Developer	6	1	USA	New York	116100
Developer	7	1	USA	New York	117800

observación

Los algoritmos de Machine Learning tienen parámetros “internos” que no dependen directamente de los datos: son los **hiperparámetros**.



Repaso de la clase anterior



Repaso de métricas de evaluación

MATRIZ DE CONFUSIÓN

		Valores reales	
		1	0
Predicción	1	Verdadero positivo (TP)	Falso positivo (FP)
	0	Falso negativo (FN)	Verdadero negativo (TN)

P: Total de positivos reales (TP + FN)

N: Total de negativos reales (TN + FP)

Repaso de métricas de evaluación

Métrica	Definición	¿Qué mide?
Exactitud (<i>Accuracy</i>)	$ACC = (TP+TN) / (P+N)$	Qué porcentaje del total fue clasificado correctamente.
Precisión (<i>Precision</i>)	$Precision = TP / (TP+FP)$	De todo lo que predijo como positivo, ¿cuánto es realmente positivo?
Sensibilidad (<i>Recall</i>)	$TPR = TP / P = TP / (TP+FN)$	Capacidad de identificar correctamente los positivos.
F1-score o F_β -score	$F_\beta = (1 + \beta^2) \frac{precision \times recall}{(\beta^2 \times precision) + recall}$	Media armónica que pondera más la Precisión o la Sensibilidad (<i>Recall</i>) según el valor de beta.
Especificidad	$TNR = TN / N = TN / (TN+FP)$	Capacidad de identificar correctamente los negativos.
Exactitud Balanceada	$BA = (TPR+TNR) / 2$	Promedio de Sensibilidad (<i>Recall</i>) y Especificidad. Ideal cuando hay desbalance de clases.

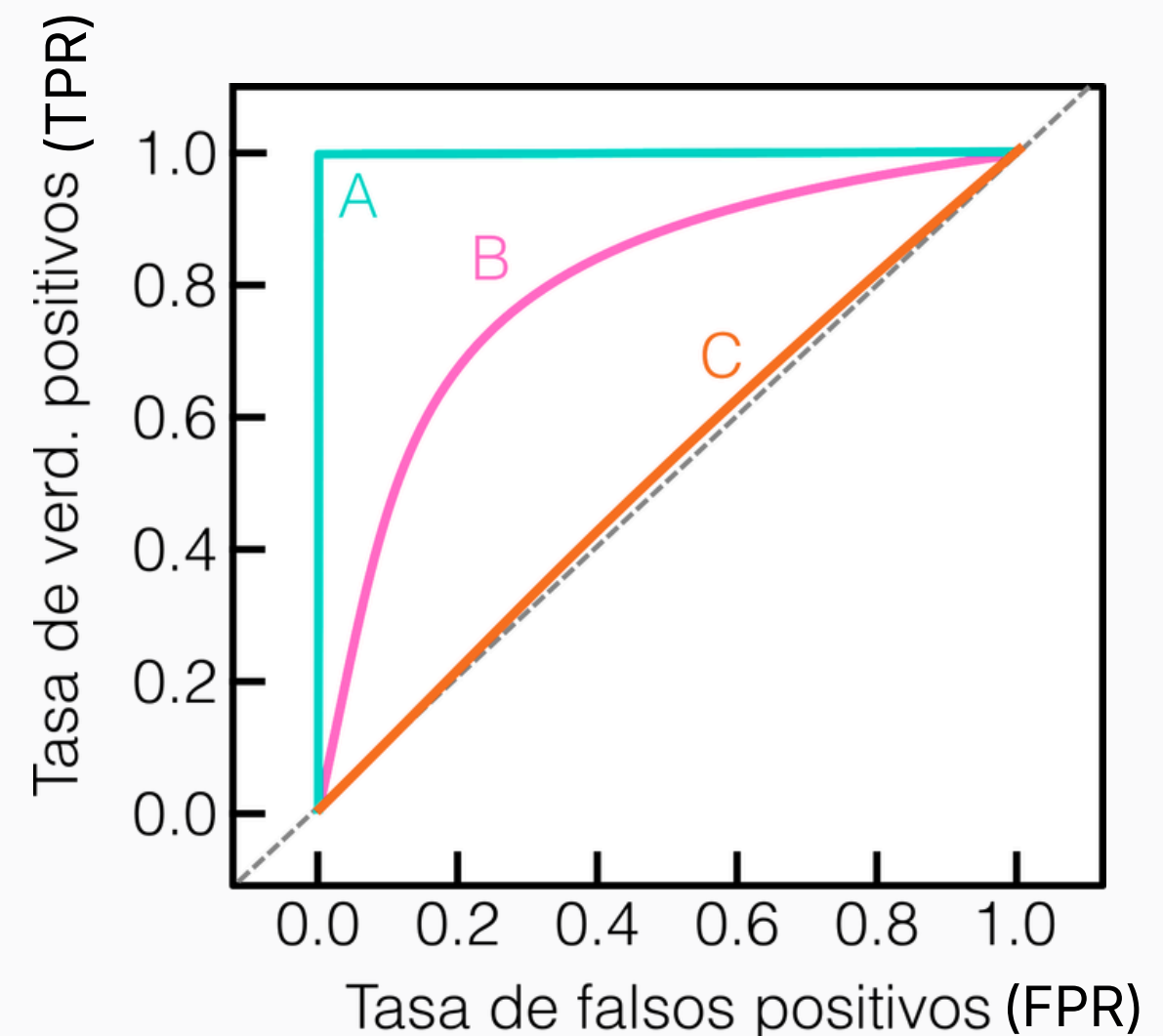
Repaso: CURVA ROC

La **curva ROC** muestra el comportamiento de la sensibilidad (*recall*) frente al falso positivo (1-especificidad) a distintos umbrales de clasificación.

- Empieza en umbral 1, donde TPR es 0 y FPR es 0.
- Termina en umbral 0, donde TPR es 1 y FPR es 1.

En la gráfica:

- A es la curva de un clasificador perfecto.
- B es la curva de un clasificador estándar.
- C es la curva de un clasificador que adivina (el peor caso).



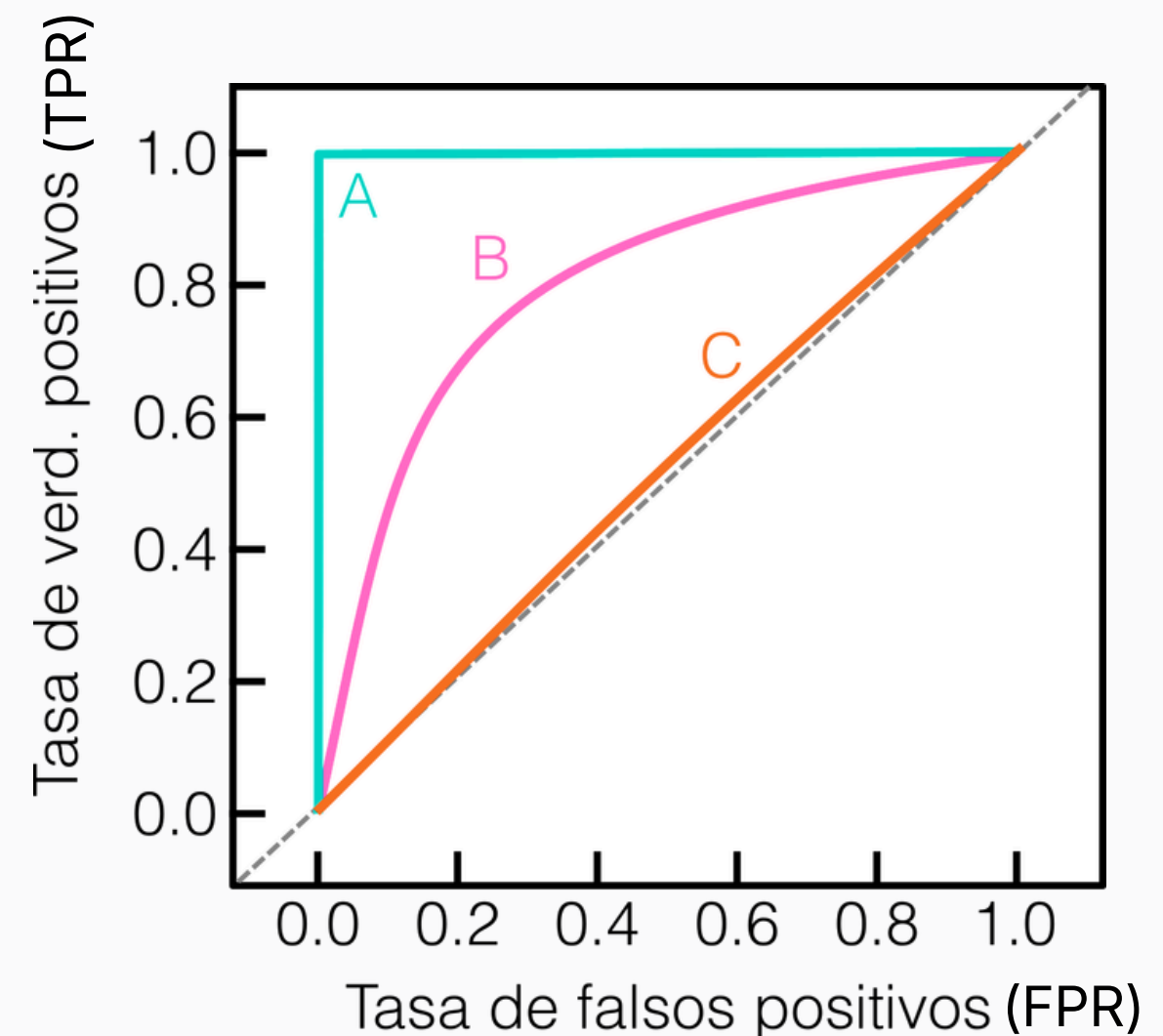
Repaso: CURVA ROC

La **curva ROC** permite encontrar el valor umbral que mejor resultado da.

Además, permite comparar clasificadores sin tener que preocuparse del valor umbral elegido.

El **área bajo la curva (AUC)** nos permite pasar esta curva a una métrica:

- A tendrá un $AUC = 1$
- B tendrá un $0.5 < AUC < 1$
- C tendrá un $AUC = 0.5$



***MEDIDAS DE
DISTANCIA***

Medidas de distancia

Muchos algoritmos de aprendizaje automático toman decisiones a partir de qué tan parecidos o diferentes son los datos entre sí.

Por ejemplo, un algoritmo puede querer saber:

- ¿Cuánto se parecen Alicia y Lara? ¿Carmen es más parecida a Alicia o a Lara?

NOMBRE	EDAD	GÉNERO	GRUPO SANGUÍNEO	Factor RH	ALTURA (cm)	Peso (k)
Alicia	20	M	A	+	160	62
Lara	35	M	B	-	172	68
Carmen	40	M	A	-	165	56

Medidas de distancia

A la medida de similitud entre observaciones la llamamos **distancia**.

Aunque intuitivamente pensamos en la distancia como la separación entre dos puntos en el espacio, en ML existen distintas maneras de definirla y calcularla.

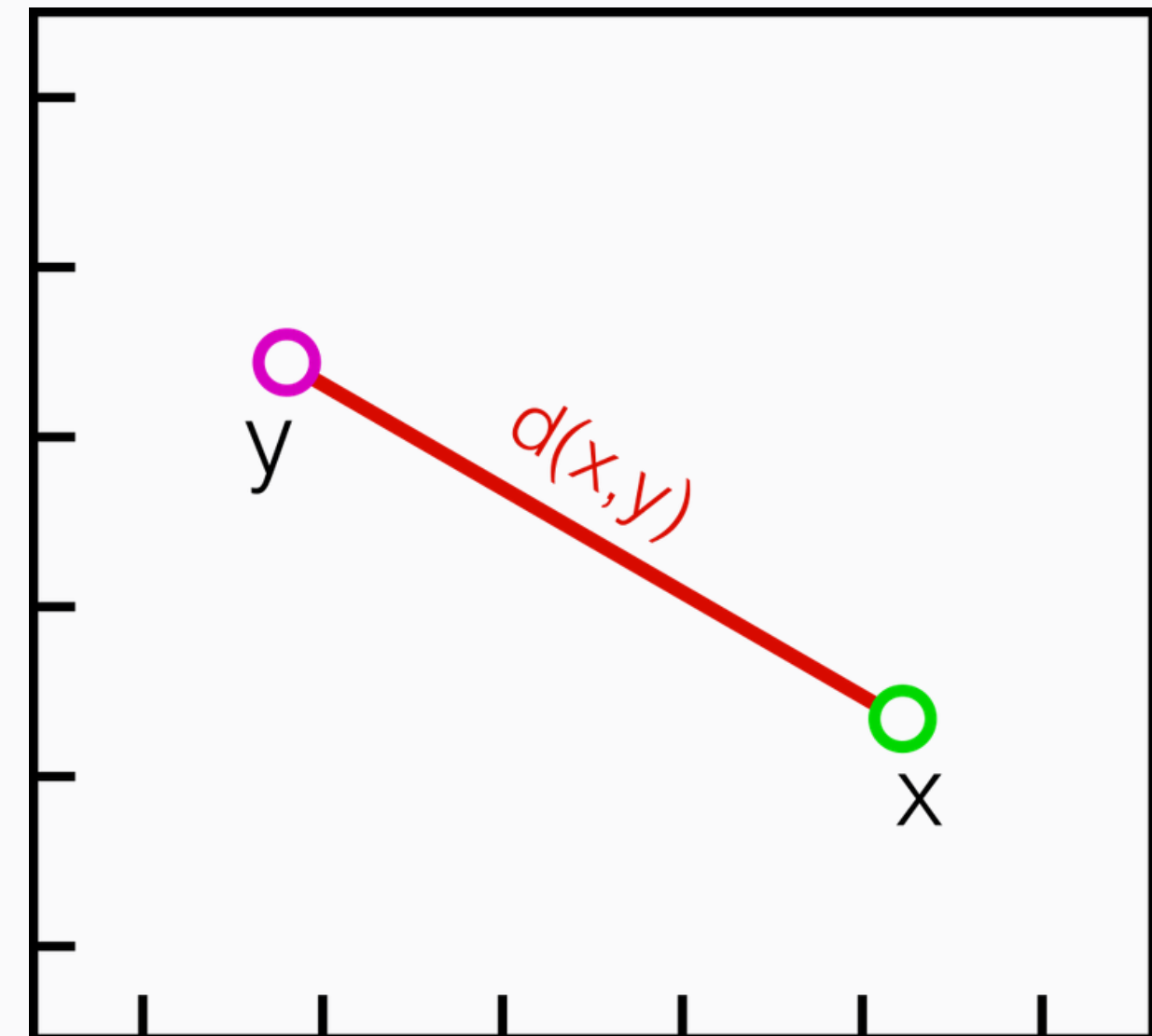
A continuación veremos algunas de las más utilizadas.

Medidas de distancia: euclidiana (módulo 2)

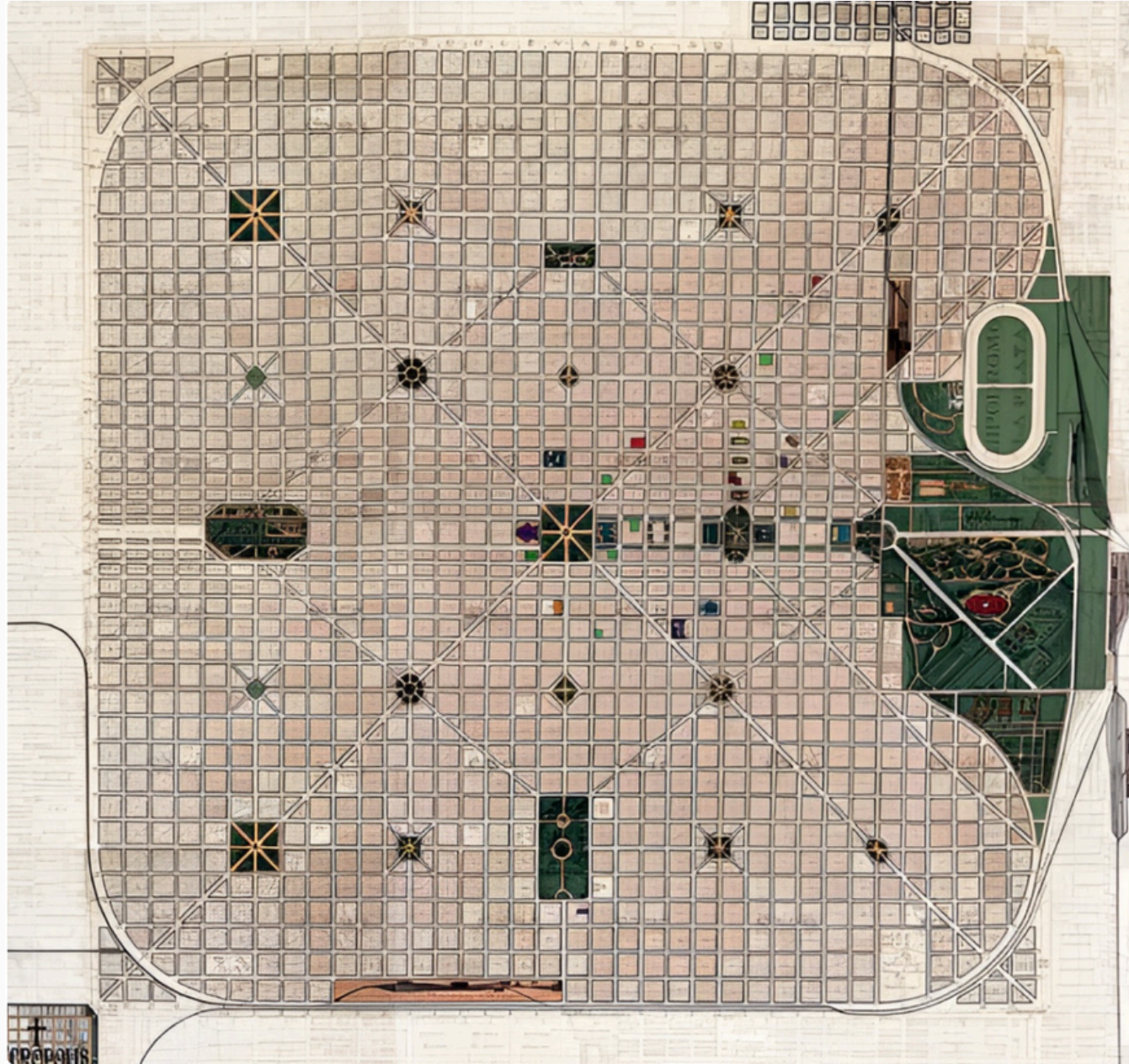
Es la más conocida. Es la mínima distancia (una recta) entre dos puntos en un espacio euclidiano. Es adecuada para datos numéricos continuos.

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

NOMBRE	EDAD	ALTURA (cm)	Peso (k)
Alicia	20	160	62
Lara	35	172	68
Carmen	40	165	56



Medidas de distancia: Manhattan (módulo 1)



Se conoce también como **distancia taxi** o de cuadra de ciudad, ya que mide distancias como en una ciudad.

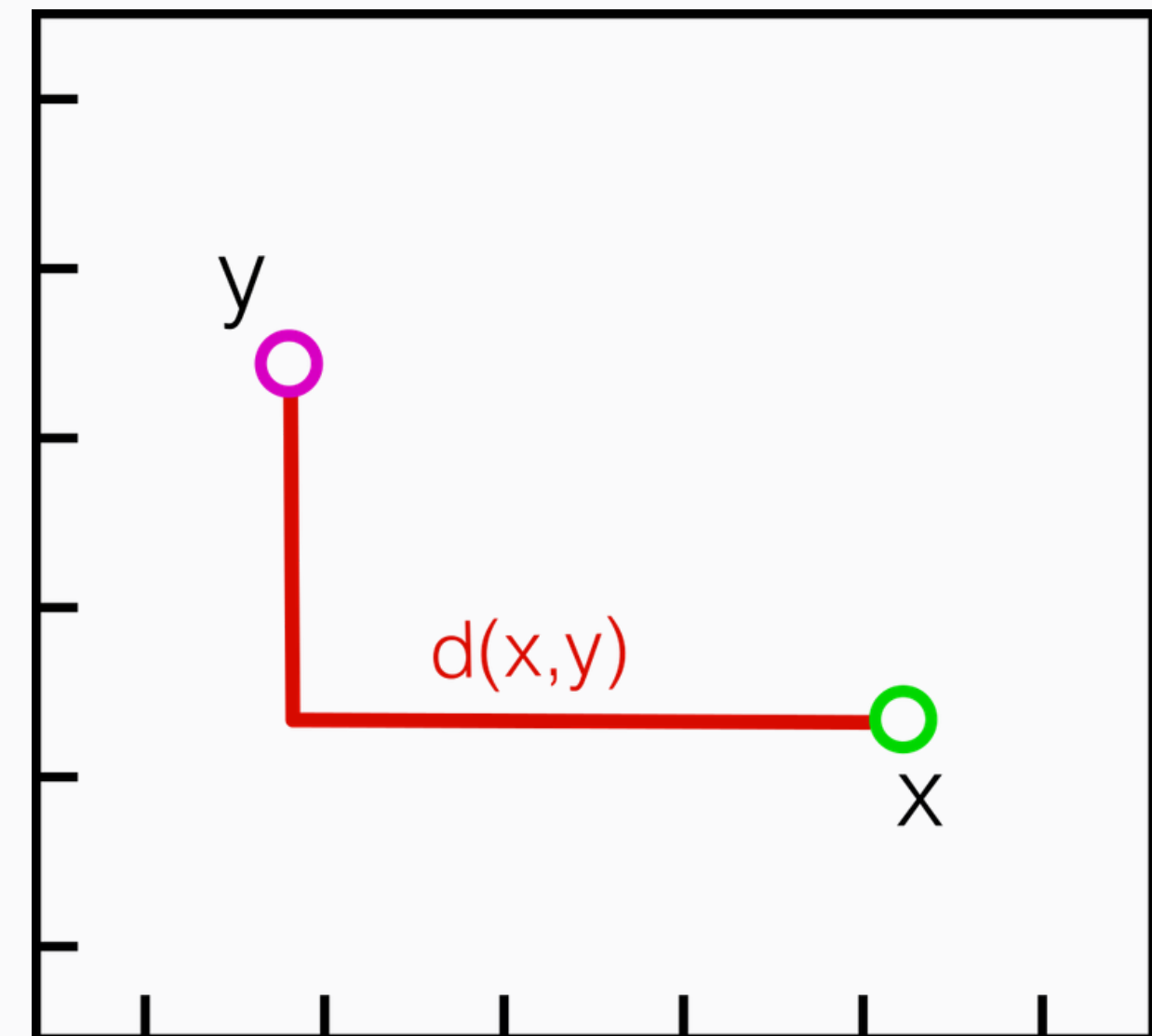
Fuente: <https://www.argentina.gob.ar/capital-humano/cultura/monumentos/la-plata-casco-urbano-fundacional>

Medidas de distancia: Manhattan (módulo 1)

Es la medida del valor absoluto que separa dos puntos. Es adecuada para datos que pueden tener correlaciones no lineales y no sigue la suposición de varianzas iguales en todas las dimensiones.

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

NOMBRE	EDAD	ALTURA (cm)	Peso (k)
Alicia	20	160	62
Lara	35	172	68
Carmen	40	165	56

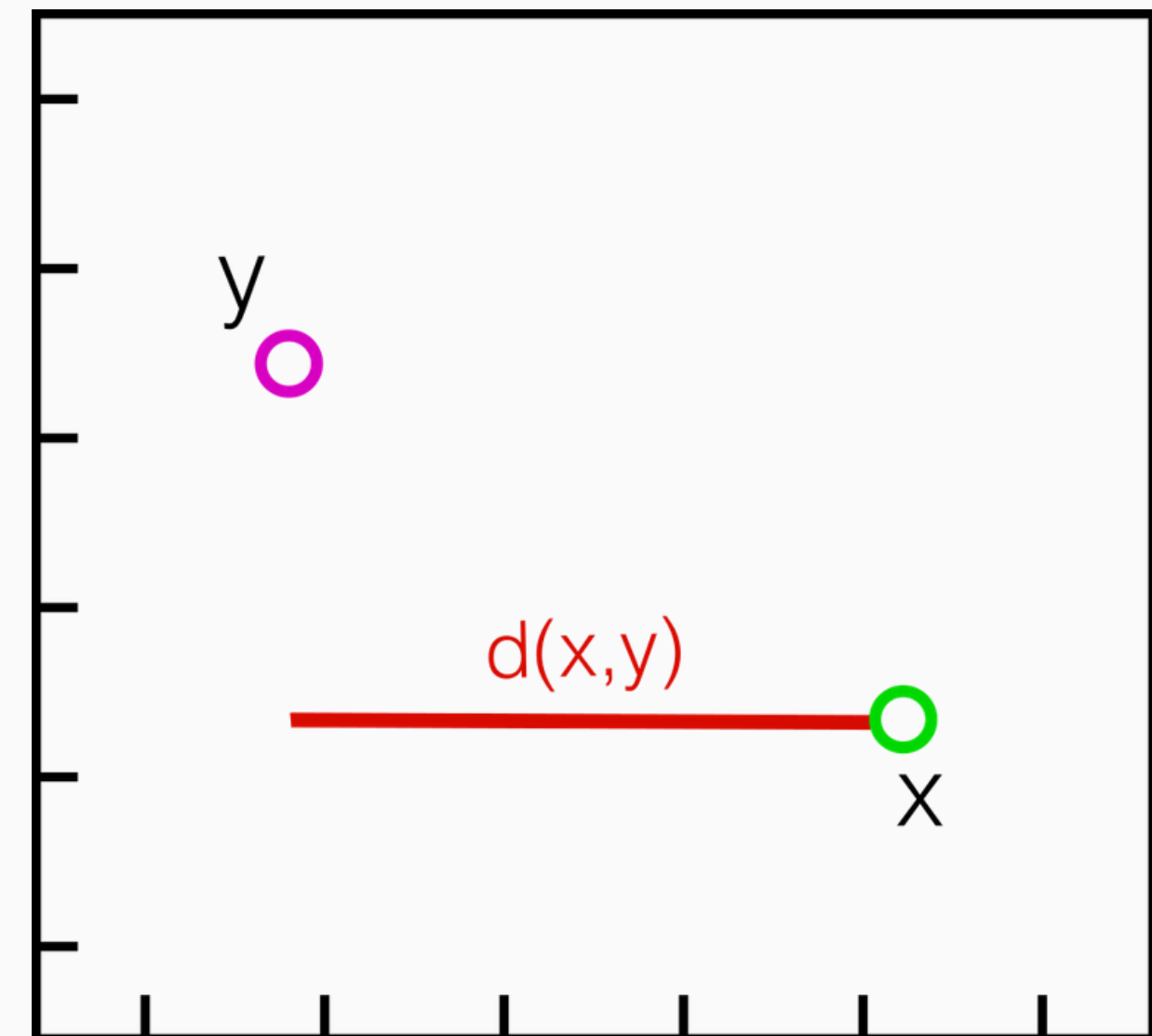


Medidas de distancia: Chebyshev (módulo infinito)

Es la diferencia máxima entre las coordenadas de dos puntos. Es adecuada cuando las dimensiones son independientes y la distancia máxima es relevante.

$$d(x, y) = \max(|x_i - y_i|)$$

NOMBRE	EDAD	ALTURA (cm)	Peso (k)
Alicia	20	160	62
Lara	35	172	68
Carmen	40	165	56

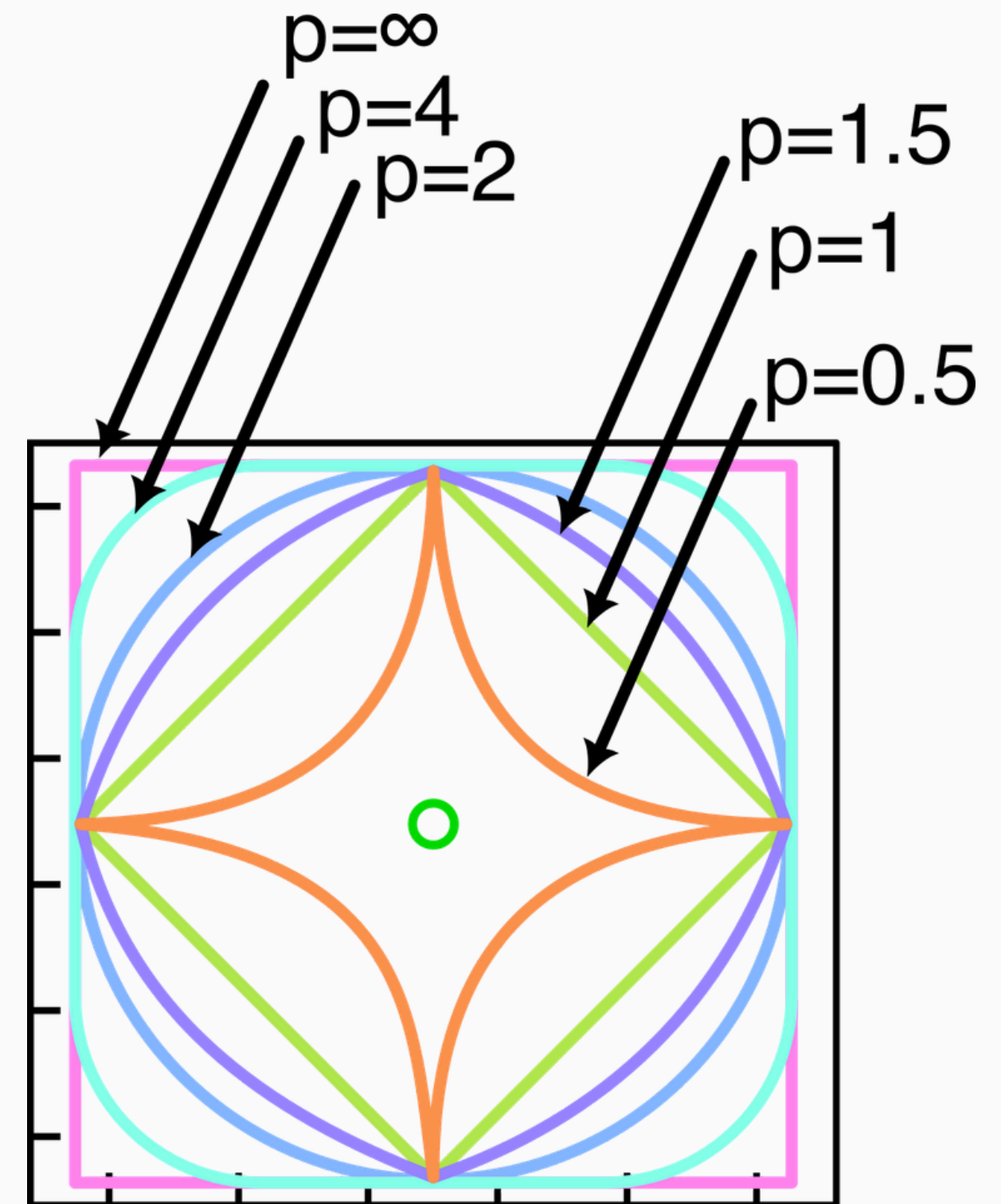


Medidas de distancia: Minkowski

Es una medida generalizada que incluye a las anteriores.

Posee un parámetro p que es el que permite seleccionar el tipo de distancia.

$$d(x, y) = \left(\sum_{i=1}^n (x_i - y_i)^p \right)^{1/p}$$



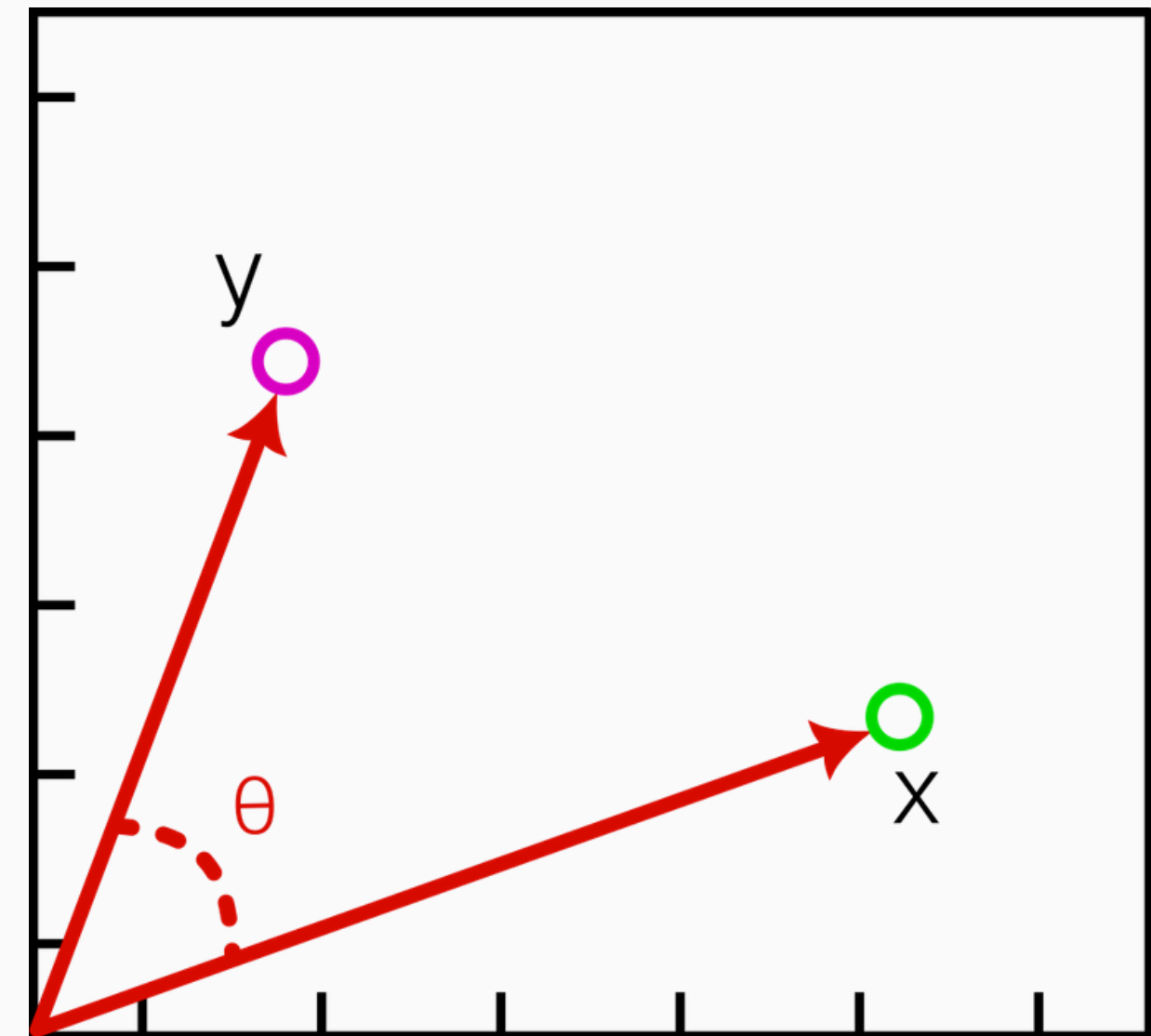
Medidas de distancia: coseno

- La **similitud coseno** mide la similitud entre dos vectores como el coseno del ángulo entre ellos.
- La **distancia coseno** es 1 menos la similitud coseno.

$$S_C(x, y) = \cos(\theta) = \frac{x \cdot y}{\|x\| \|y\|}$$

$$d(x, y) = 1 - S_C(x, y)$$

Es adecuada para datos en los que la magnitud de los vectores es irrelevante. Solo interesa su orientación.

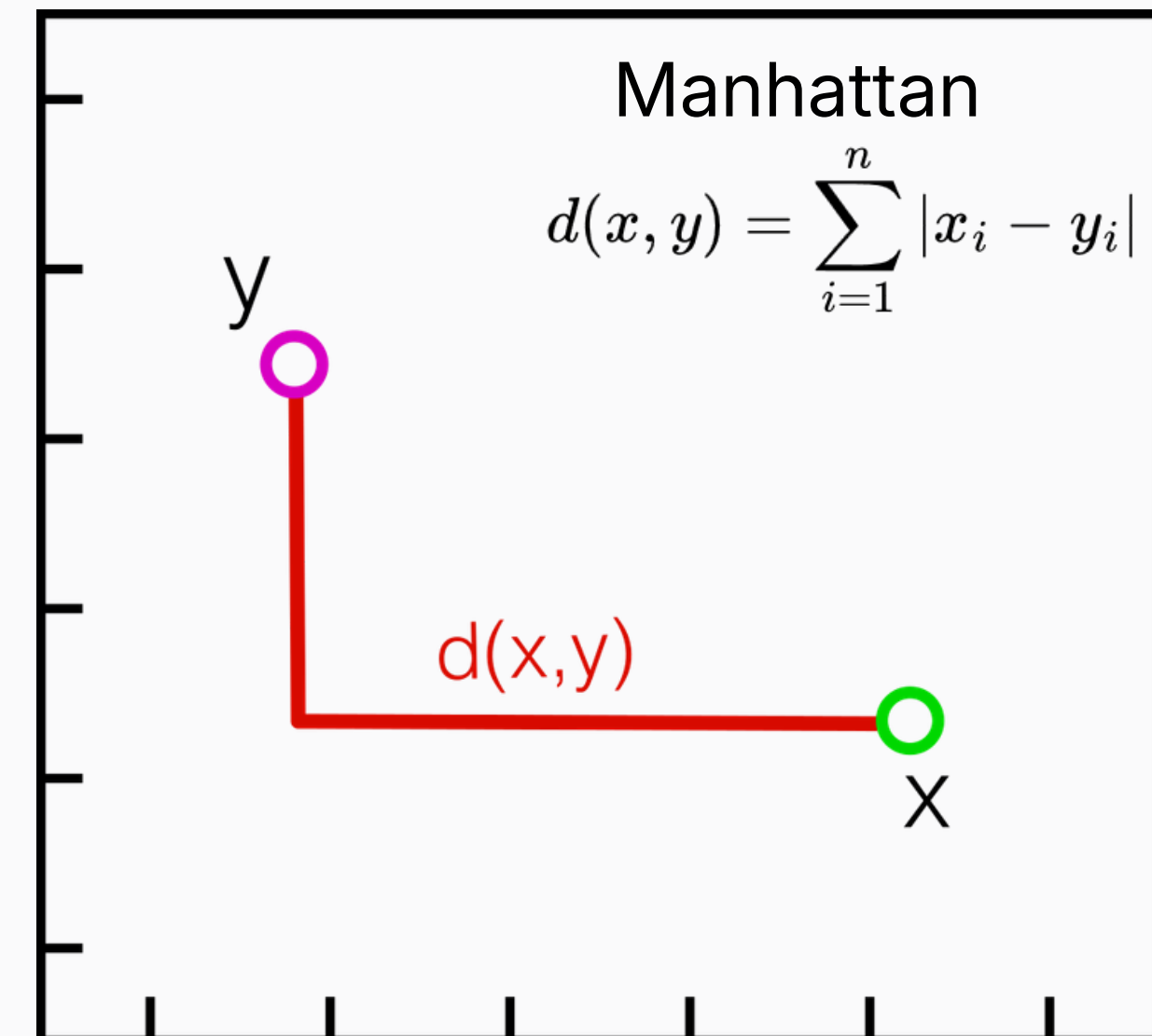


Medidas de distancia: Canberra

- Es la distancia de Manhattan ponderada.
- Se puede pensar como una diferencia porcentual.
- Pondera más las diferencias en las dimensiones donde los valores son pequeños.
- Al estar ponderada, es más robusta a outliers.

$$d(x, y) = \sum_{i=1}^n \frac{|x_i - y_i|}{|x_i| + |y_i|}$$

NOMBRE	EDAD	ALTURA (cm)	Peso (k)
Alicia	20	160	62
Lara	35	172	68
Carmen	40	165	56



Medidas de distancia: Jaccard

- Se utiliza comúnmente en conjuntos o datos binarios.
- El **índice de Jaccard** mide la similitud entre dos conjuntos como el tamaño de su intersección dividido por el tamaño de su unión.
- La **distancia de Jaccard** es 1 menos el índice de Jaccard.

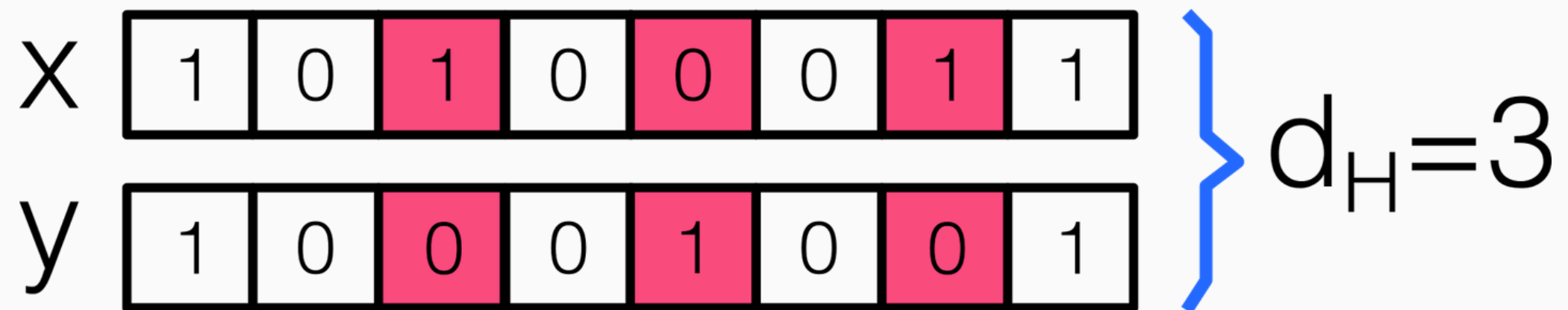
$$J(x, y) = \frac{|x \cap y|}{|x \cup y|}$$

$$d(x, y) = 1 - J(x, y)$$

x	1	0	1	0	0	0	1	1	J=2/5 d _J =3/5
y	1	0	0	0	1	0	0	1	

Medidas de distancia: Hamming

- Se utiliza comúnmente en conjuntos o datos binarios o booleanos.
- Mide la cantidad de elementos diferentes entre sí.



Medidas de distancia: Gower

- La **similitud de Gower** mide qué tan parecidas son dos observaciones considerando simultáneamente variables numéricas y categóricas:
 - Para las variables numéricas se calcula como $s(x_k, y_k) = 1 - \frac{|x_k - y_k|}{R_k}$
 - Para las variables categóricas la similitud es 1 si son iguales, 0 si son diferentes.
- La **distancia de Gower** es 1 menos la similitud de Gower.

$$S(x, y) = \frac{\sum_{k=1}^p w_k \times s(x_k, y_k)}{\sum_{k=1}^p w_k}$$

$$d(x, y) = 1 - S(x, y)$$

NOMBRE	EDAD	GÉNERO	GRUPO SANGUÍNEO	Factor RH	ALTURA (cm)	Peso (k)
Alicia	20	M	A	+	160	62
Lara	35	M	B	-	172	68
Carmen	40	M	A	-	165	56

CLASIFICADOR

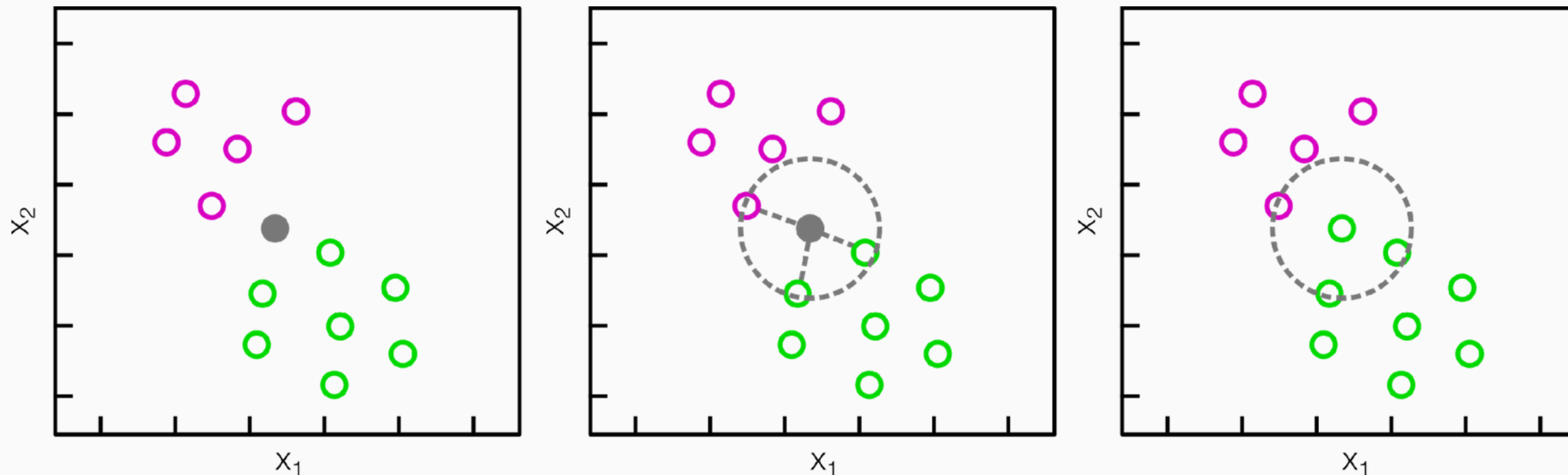
KNN

KNN (*K Nearest Neighbor*)

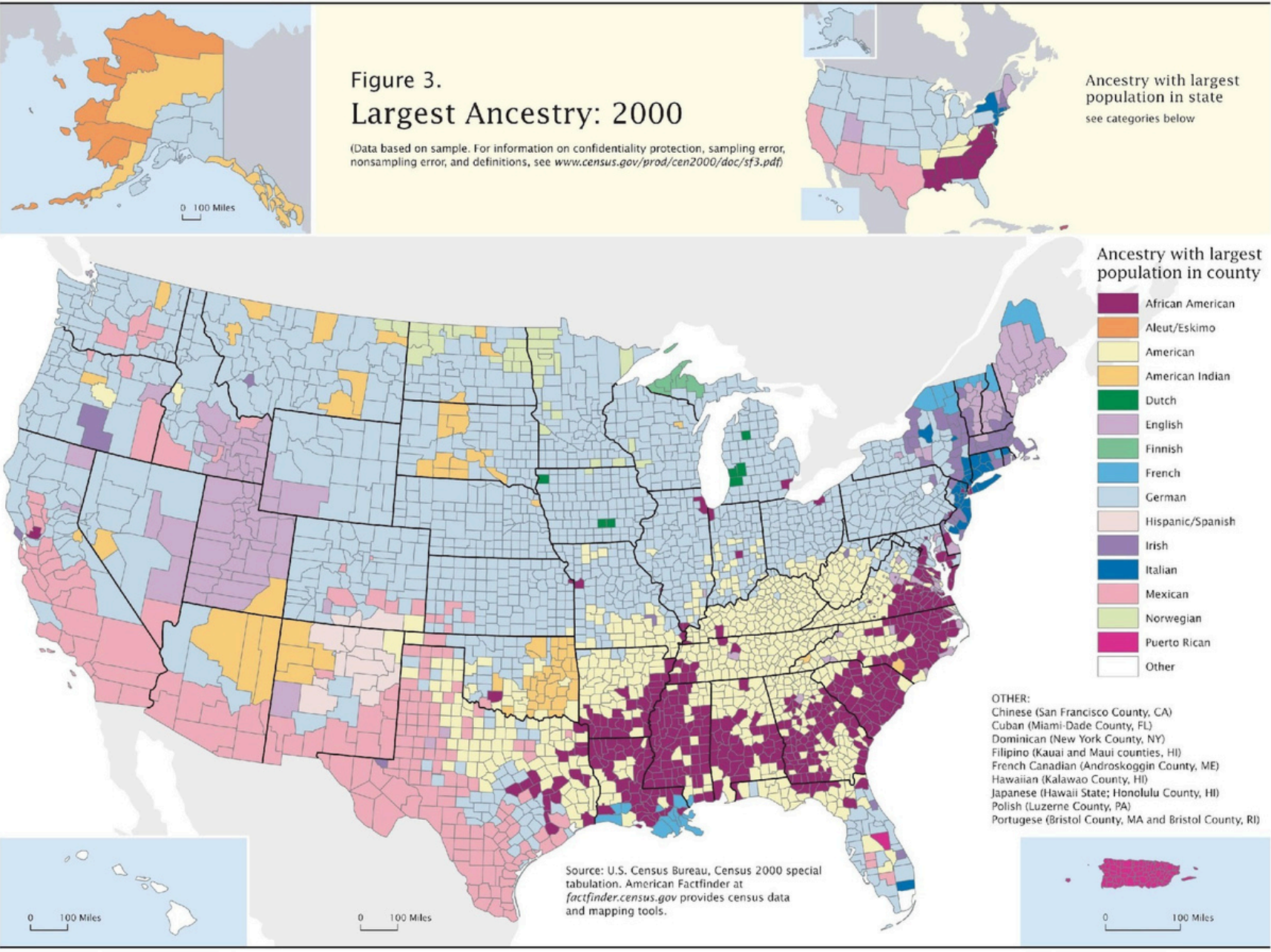
El clasificador de **k vecinos más cercanos** (KNN o k-NN) es un algoritmo de clasificación basado en la proximidad entre observaciones. Su idea central es que las observaciones cercanas tienden a ser similares, por lo que una nueva observación se clasifica según la categoría predominante entre sus k vecinos más próximos.

La clase se elige en función de la moda de la clase de sus vecinos.

Este modelo **no obtiene una salida de probabilidad**, solo nos dice de qué clase es.



KNN



KNN

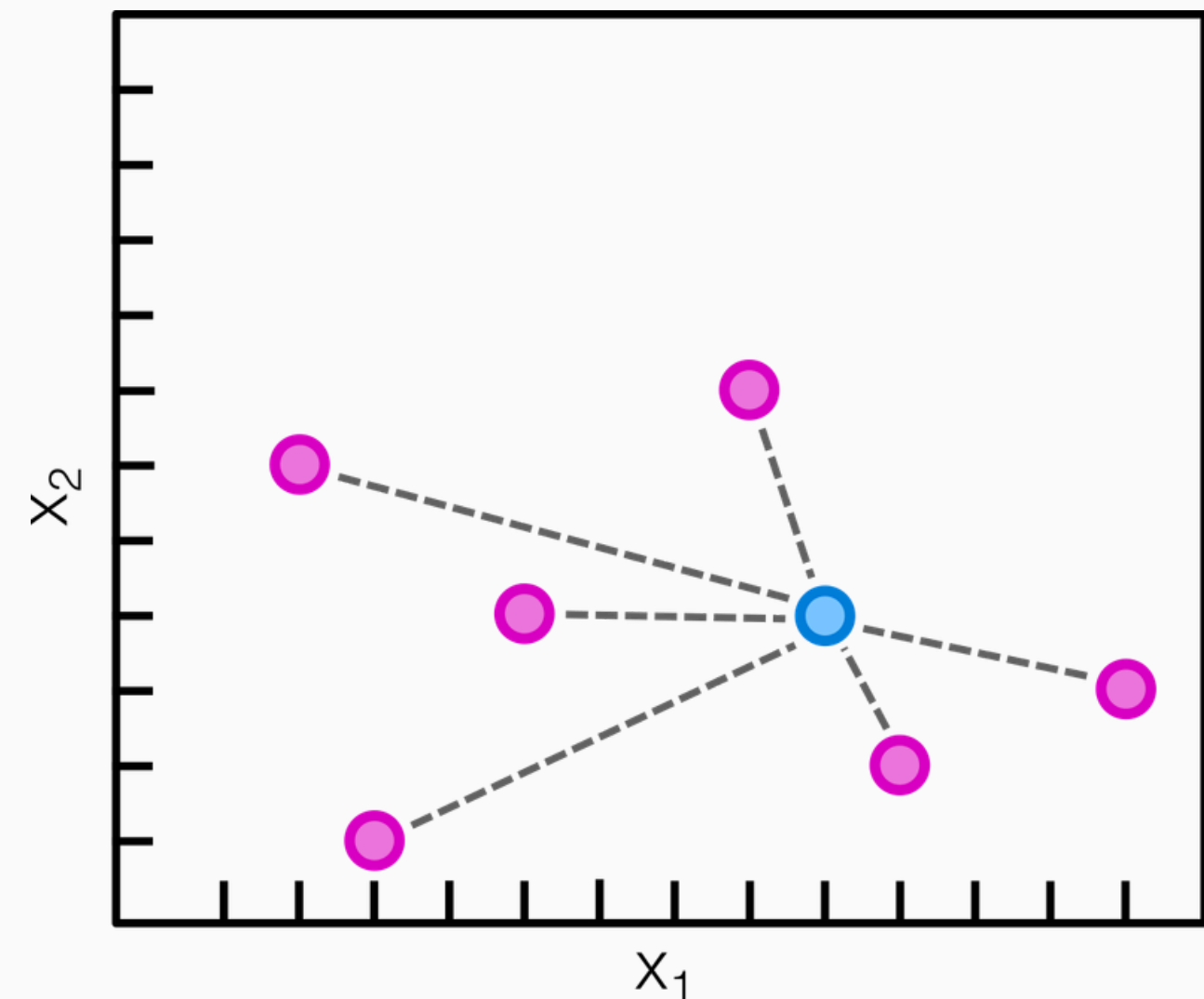
Este algoritmo se fija en la distancia entre observaciones.

¿Ahora la pregunta es cómo medimos la distancia?

Como vimos, hay múltiples maneras de medir distancia entre dos observaciones.

KNN requiere que elijamos cuál medida de distancia vamos a usar:

- euclidiana, manhattan, minkowski, etc.

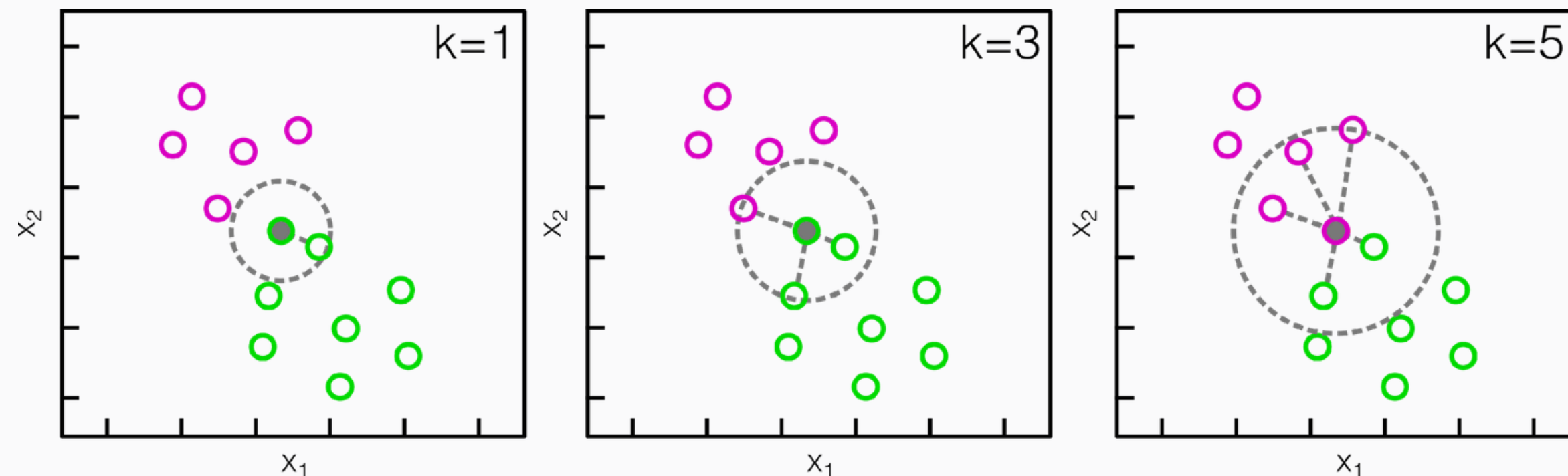


KNN

Además de la métrica de distancia, debemos definir el valor de **k**, es decir, cuántos vecinos se usarán para determinar la clasificación de una observación. En general, se recomienda tener un número impar para **k** para evitar empates en la clasificación.

Definir **k** es un acto de equilibrio:

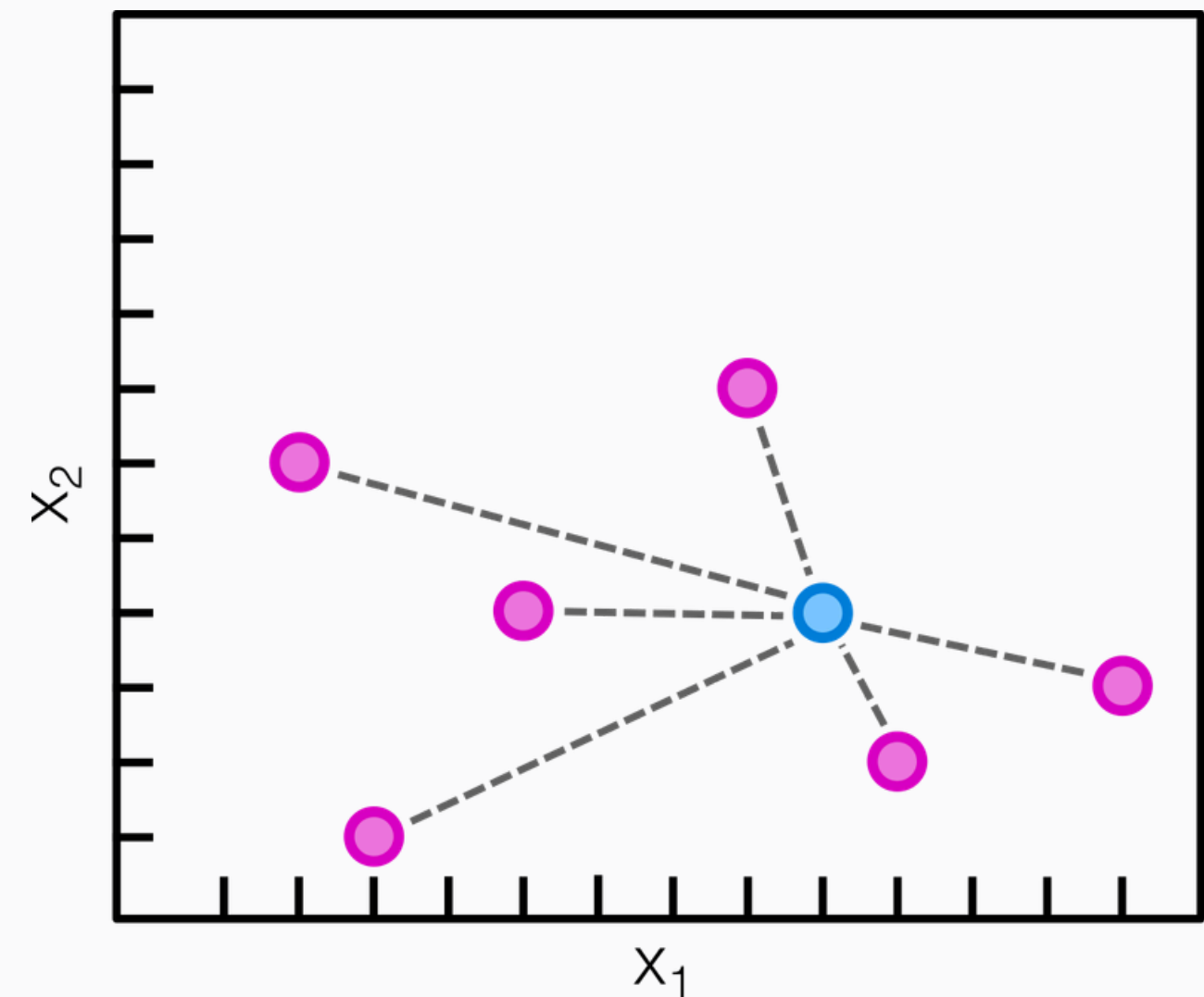
- Valores bajos de **k** pueden tener una varianza alta, pero un sesgo bajo.
- Valores altos de **k** pueden tener un sesgo alto y poca varianza.



KNN

Este algoritmo no tiene “entrenamiento” ya que debe guardar todo el dataset para evaluar a nuevos valores a que clase pertenece.

Si el dataset de entrenamiento es muy grande, puede tener dificultades para almacenarse o ejecutarse.



***ENCONTRANDO A
LOS VECINOS***

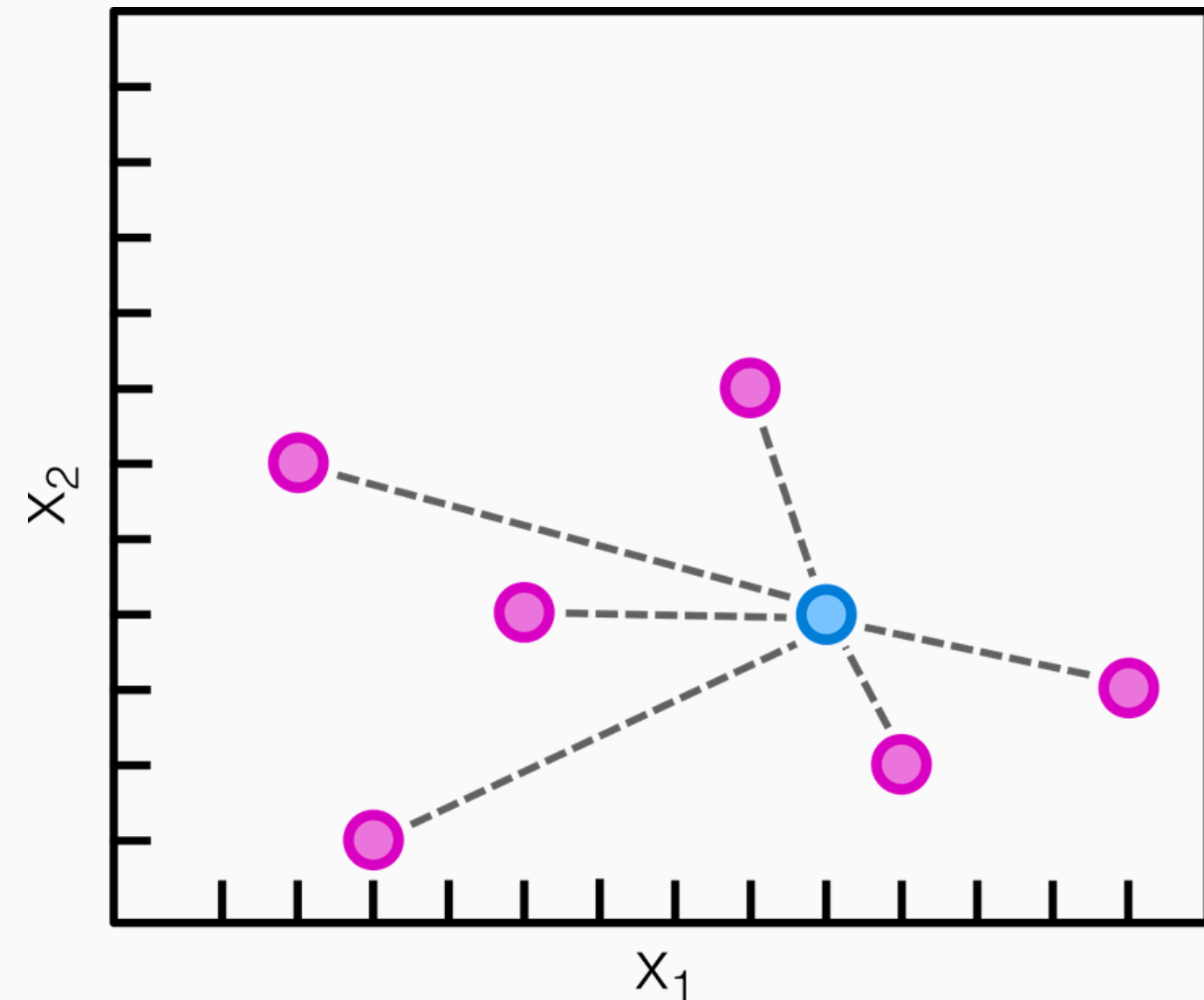
Encontrando a los vecinos: fuerza bruta

Un detalle que estamos pasando por alto en este modelo es la implementación de la búsqueda de los vecinos.

La forma más básica de hacer esto es por **fuerza bruta**:

Dado una nueva observación, calcular la distancia a todos con respecto al set de entrenamiento.

El problema es que no escala bien con dataset grandes ya que tiene una complejidad de $O(DN^2)$, con N observaciones y D atributos.



Encontrando a los vecinos: K-D Tree

Entonces necesitamos usar algoritmos de búsqueda.

Uno algoritmo de búsqueda popular es **K-D Tree**:

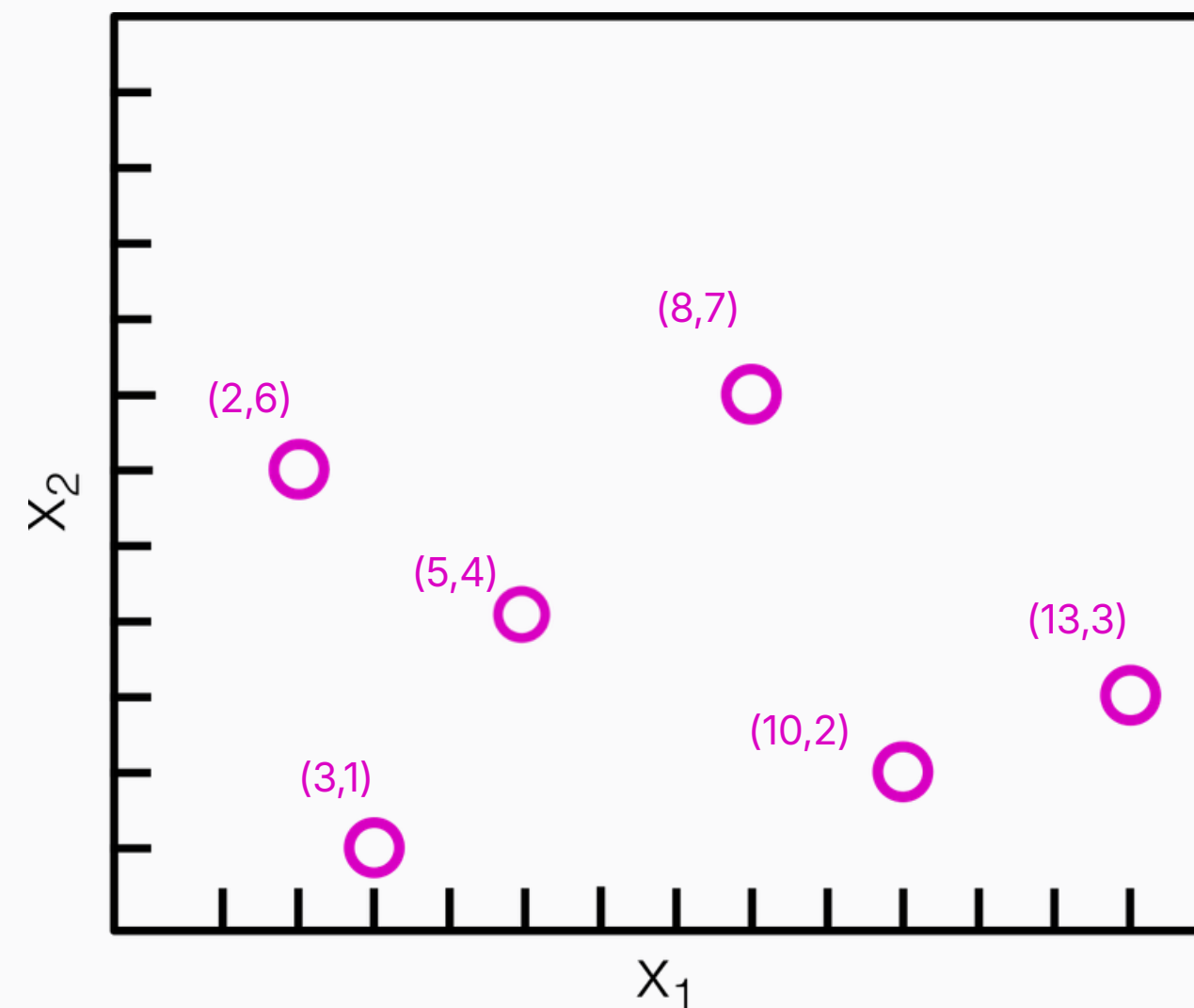
- K-D Tree arma un árbol de búsqueda.
- Se basa en el concepto de que si un punto A está muy distante de B y B está cerca de C, entonces A está lejos de C sin necesidad de explícitamente calcularlo. En este caso se reduce a complejidad máxima de $O[DN \log(N)]$.
- Divide sucesivamente el espacio D dimensional. En cada nodo se selecciona una dimensión y se realiza una partición en dos regiones.

Para entenderlo vamos a ver un ejemplo en 2 dimensiones.

Encontrando a los vecinos: K-D Tree

- Tenemos dos dimensiones **x1** y **x2**, y 6 puntos (observaciones).

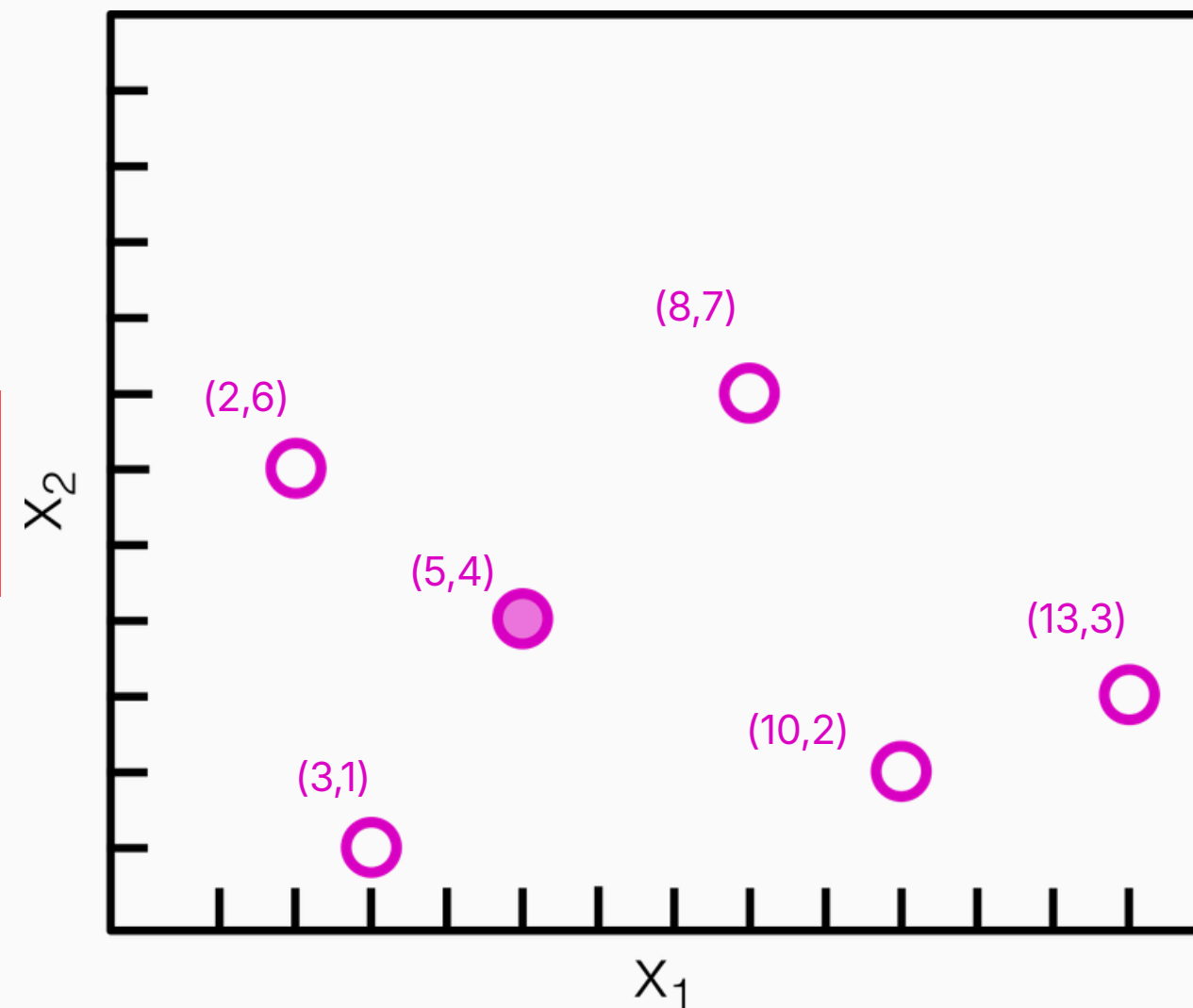
	x1	x2
P1	2	6
P2	3	1
P3	5	4
P4	8	7
P5	10	2
P6	13	3



Encontrando a los vecinos: K-D Tree

- Empezamos tomando un punto para armar el árbol, elijamos el punto P3.

	x1	x2
P1	2	6
P2	3	1
P3	5	4
P4	8	7
P5	10	2
P6	13	3

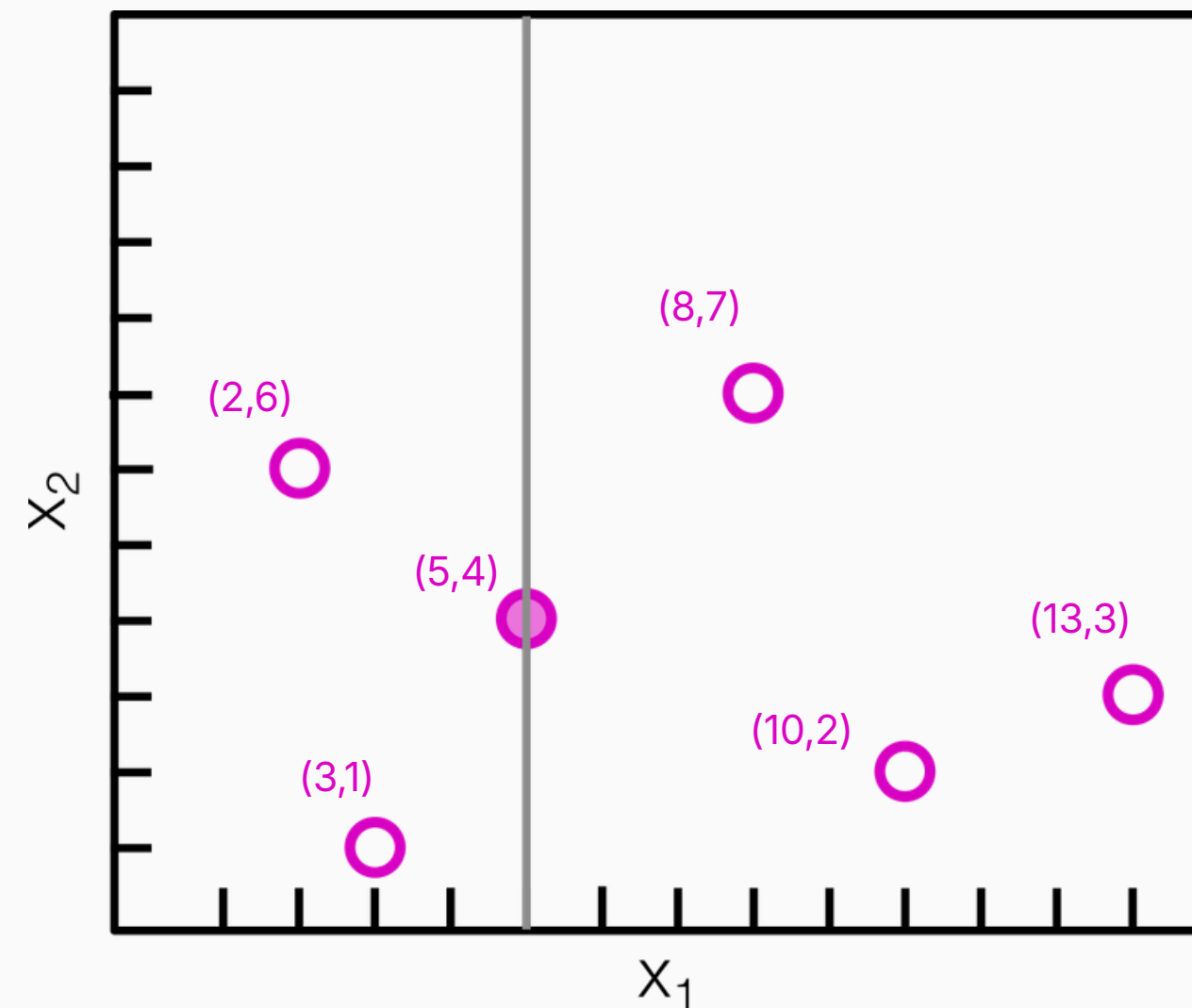


(5,4)

Encontrando a los vecinos: K-D Tree

- Separamos el espacio en dos con respecto al eje x_1 .

	x_1	x_2
P1	2	6
P2	3	1
P3	5	4
P4	8	7
P5	10	2
P6	13	3

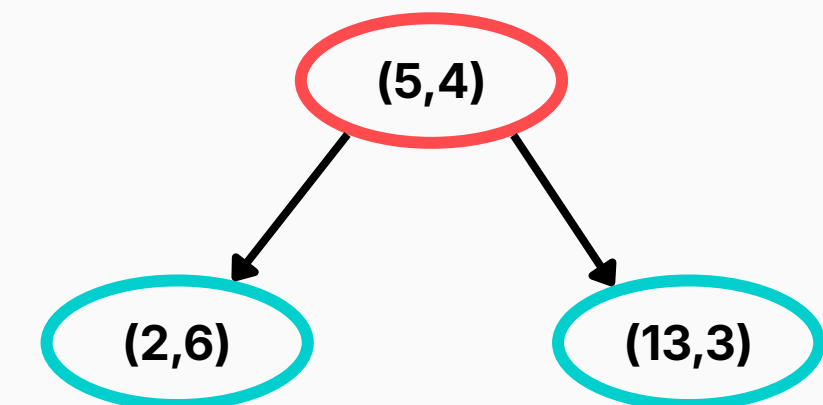
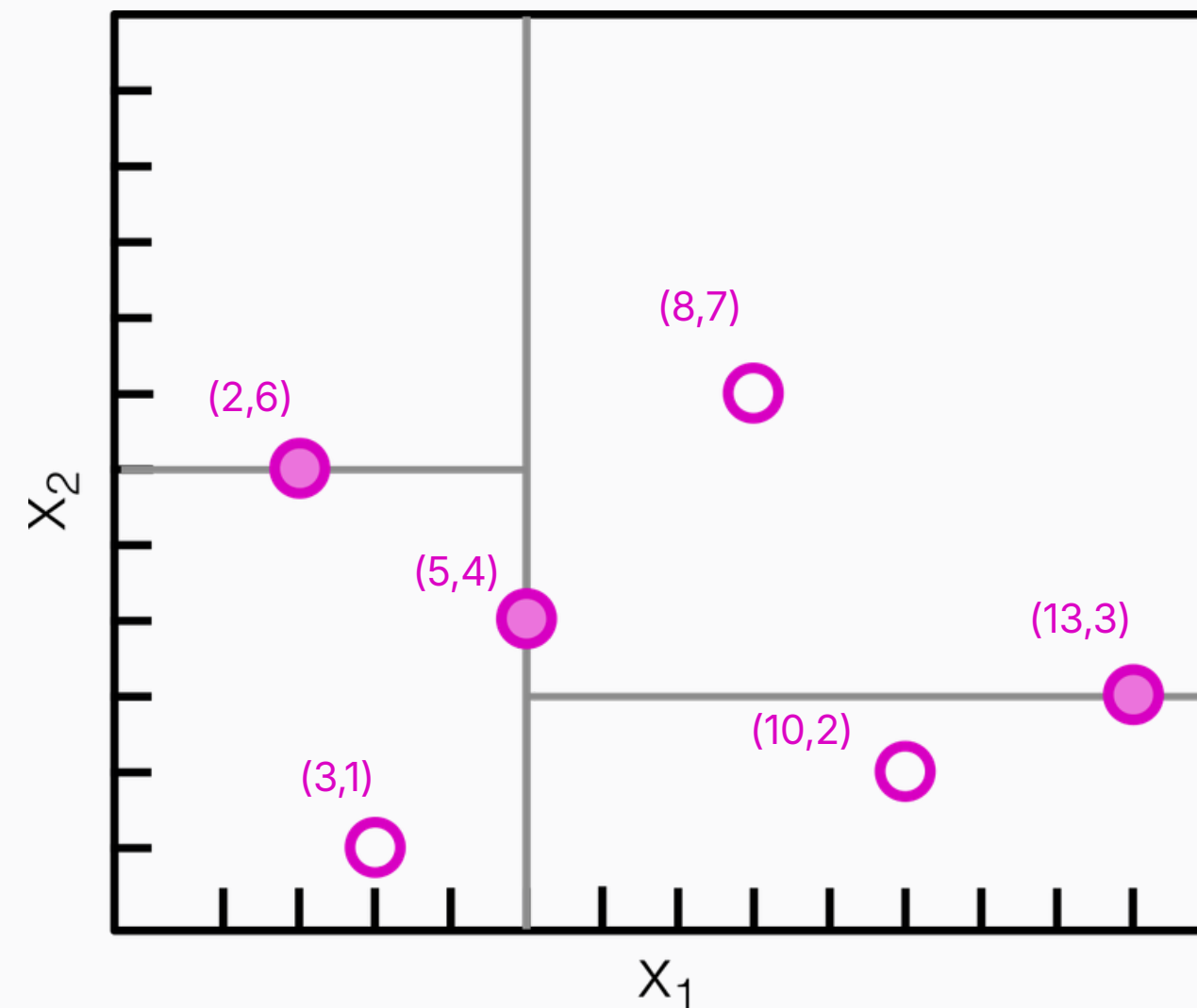


(5,4)

Encontrando a los vecinos: K-D Tree

- Una vez separado el espacio, para cada lado, buscamos los puntos que están arriba o debajo inmediatamente con respecto al eje **x2**, y los usamos para separar con respecto a este eje:

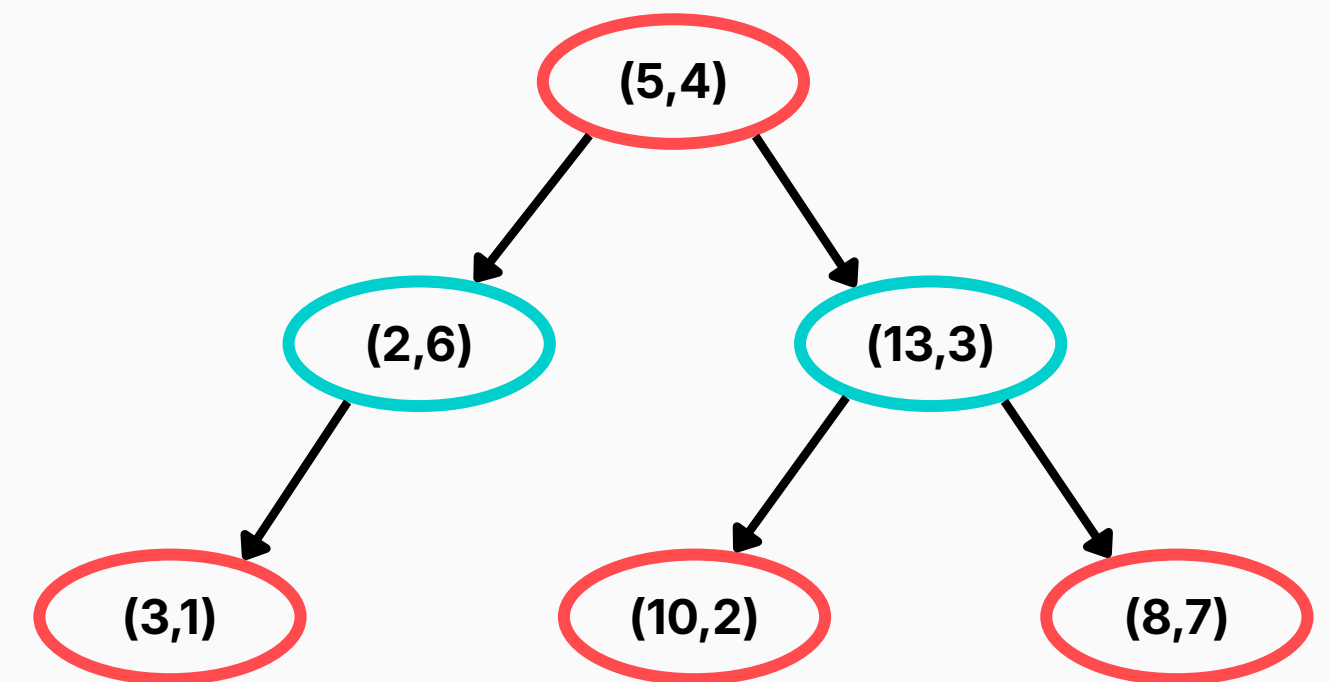
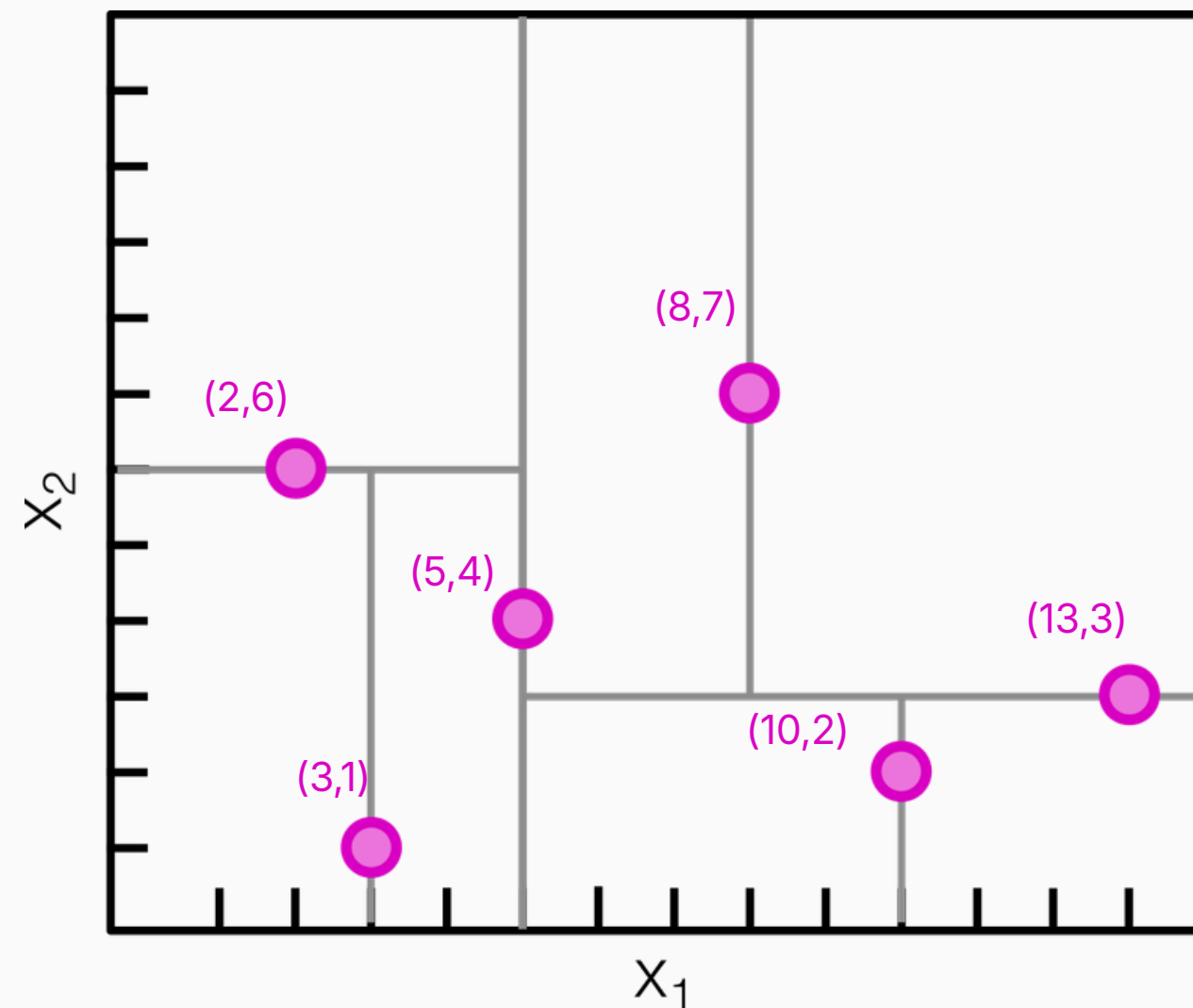
	x1	x2
P1	2	6
P2	3	1
P3	5	4
P4	8	7
P5	10	2
P6	13	3



Encontrando a los vecinos: K-D Tree

- Finalmente, para la siguiente profundidad, organizamos a los siguientes nodos con respecto a x_1 , y así sucesivamente...

	x_1	x_2
P1	2	6
P2	3	1
P3	5	4
P4	8	7
P5	10	2
P6	13	3

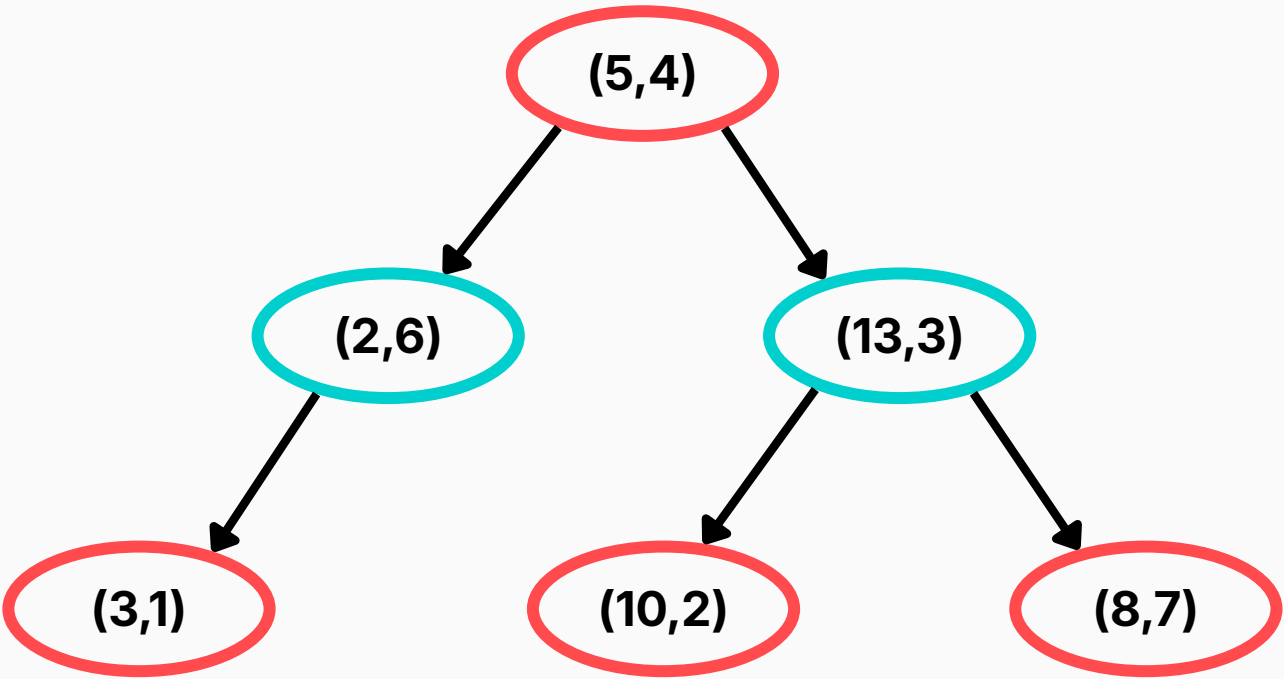
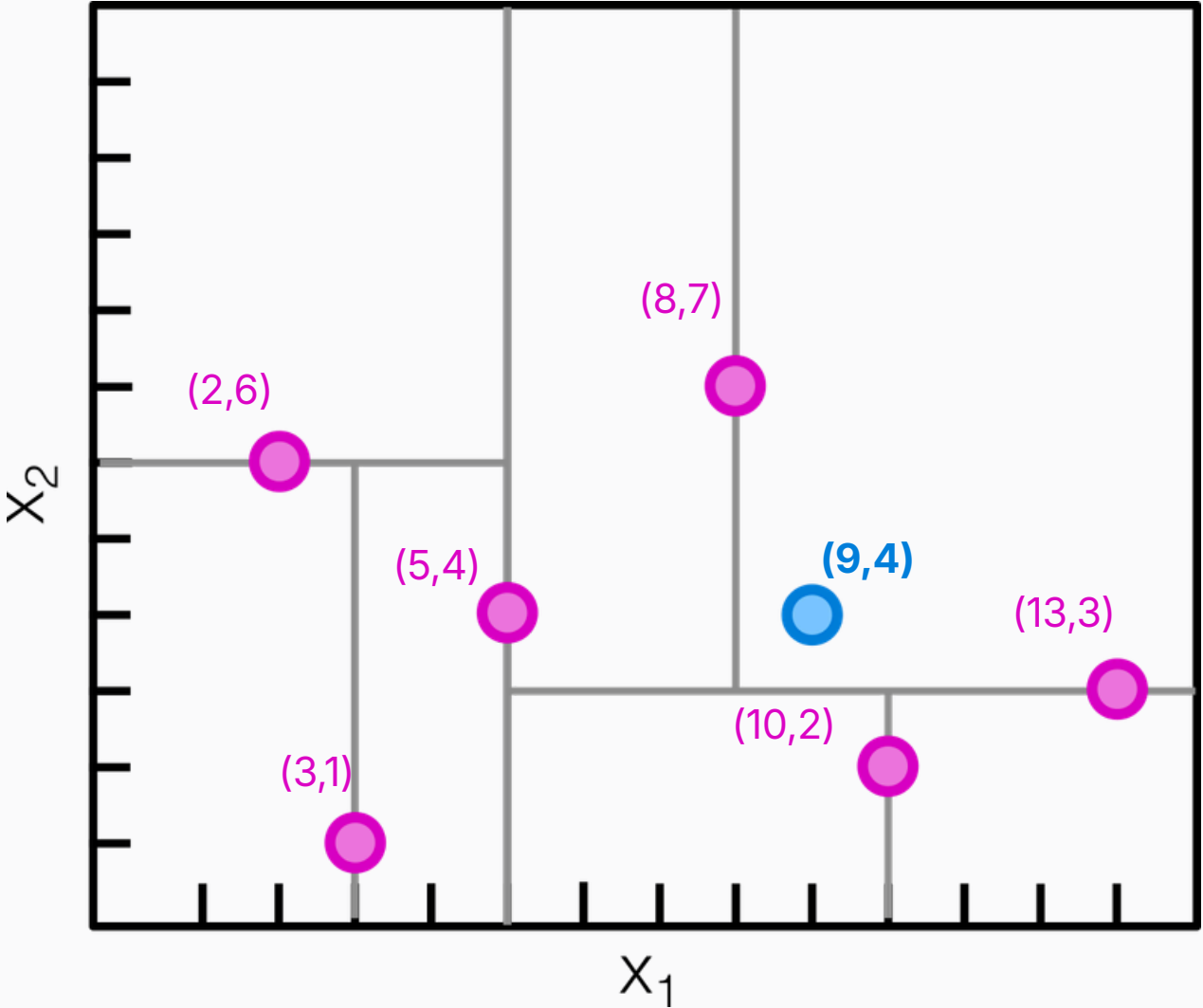


Encontrando a los vecinos: K-D Tree

- Una vez que construimos el árbol, podemos usarlo para calcular el vecino más cercano a un nuevo punto. Por ejemplo, dado el punto (9,4), el KD-Tree permite identificar rápidamente cuál es su vecino más cercano y calcular la distancia a él.

	x1	x2
P1	2	6
P2	3	1
P3	5	4
P4	8	7
P5	10	2
P6	13	3

<i>p</i>	9	4
----------	---	---

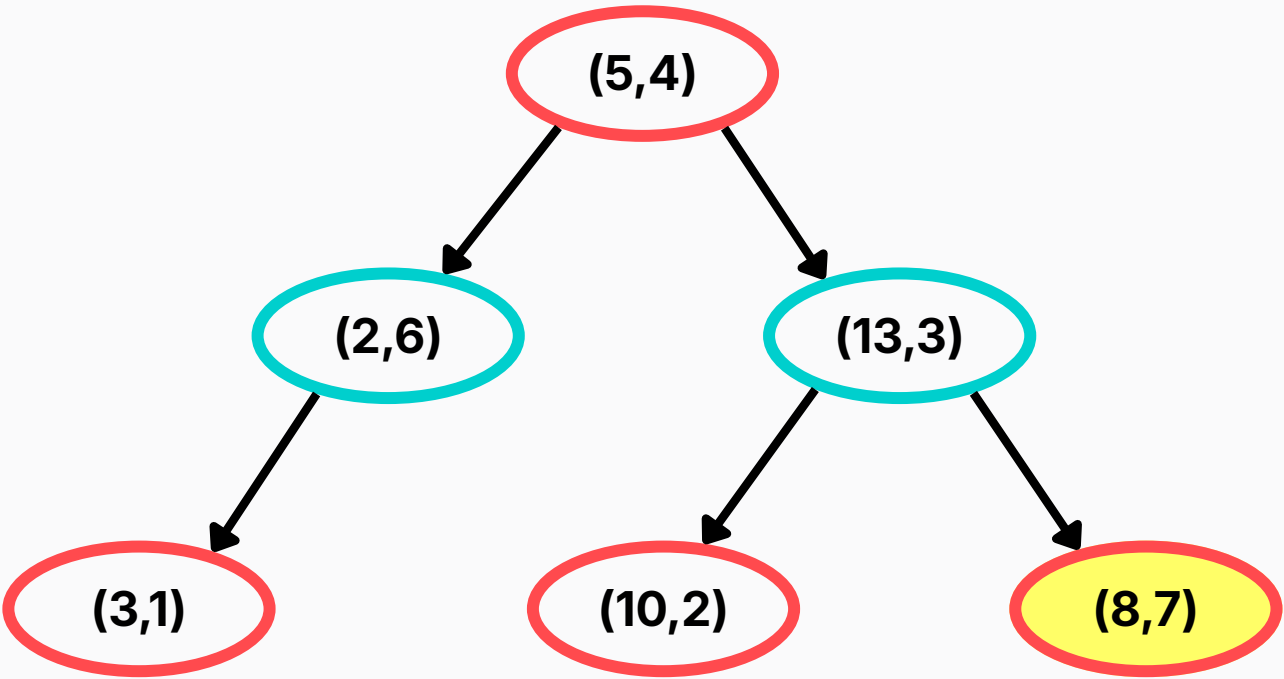
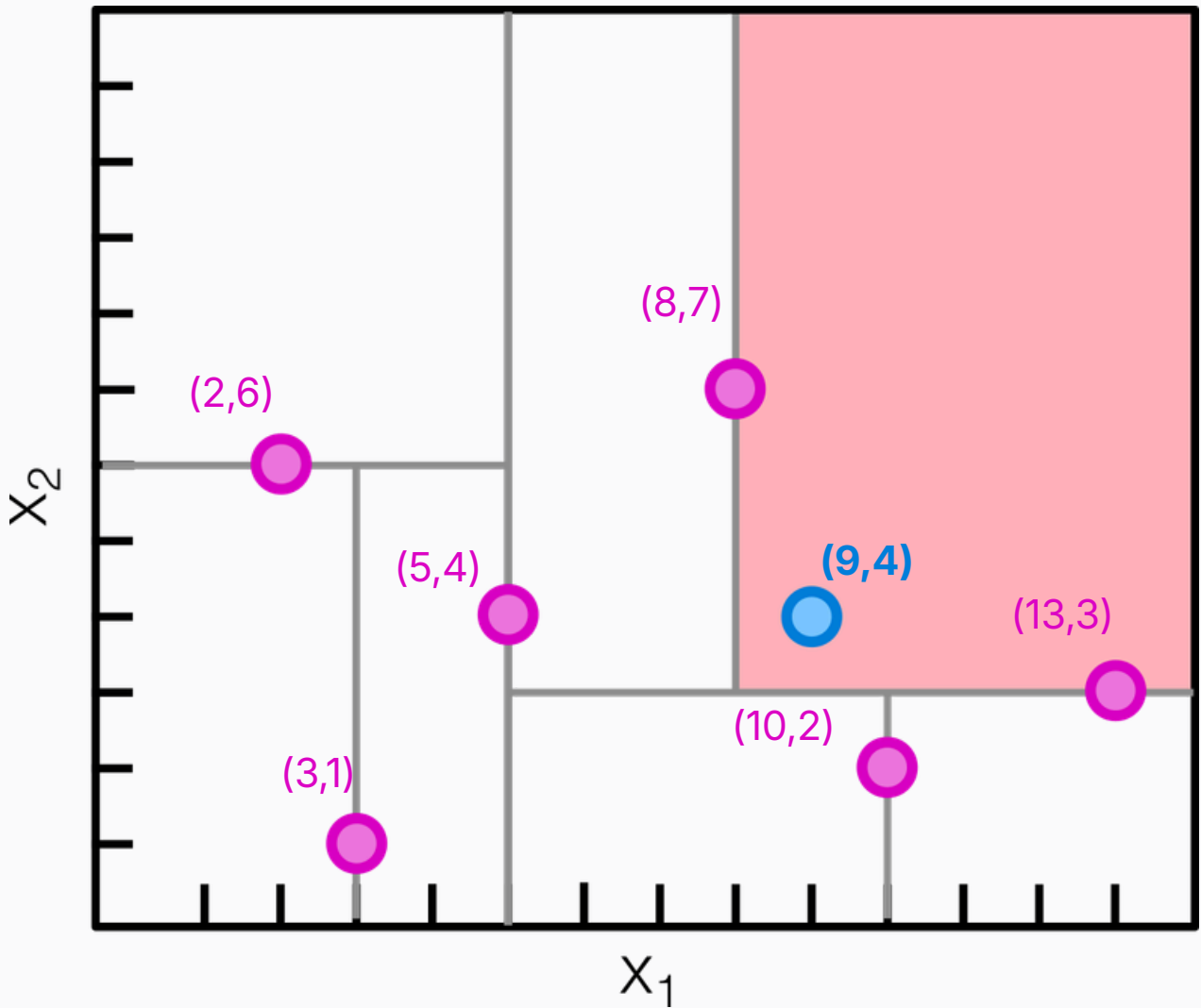


Encontrando a los vecinos: K-D Tree

- Se recorre el árbol siguiendo las particiones hasta llegar a la hoja donde caería el punto que estamos buscando.

	x1	x2
P1	2	6
P2	3	1
P3	5	4
P4	8	7
P5	10	2
P6	13	3

<i>p</i>	9	4
----------	---	---

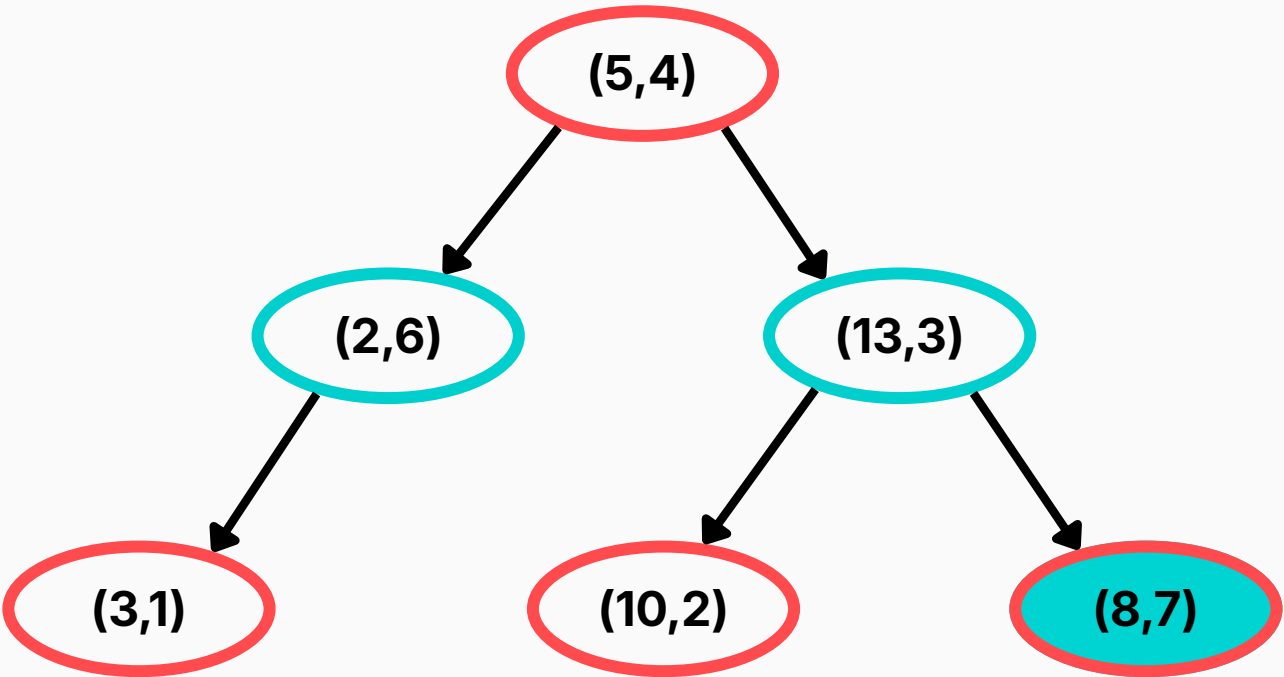
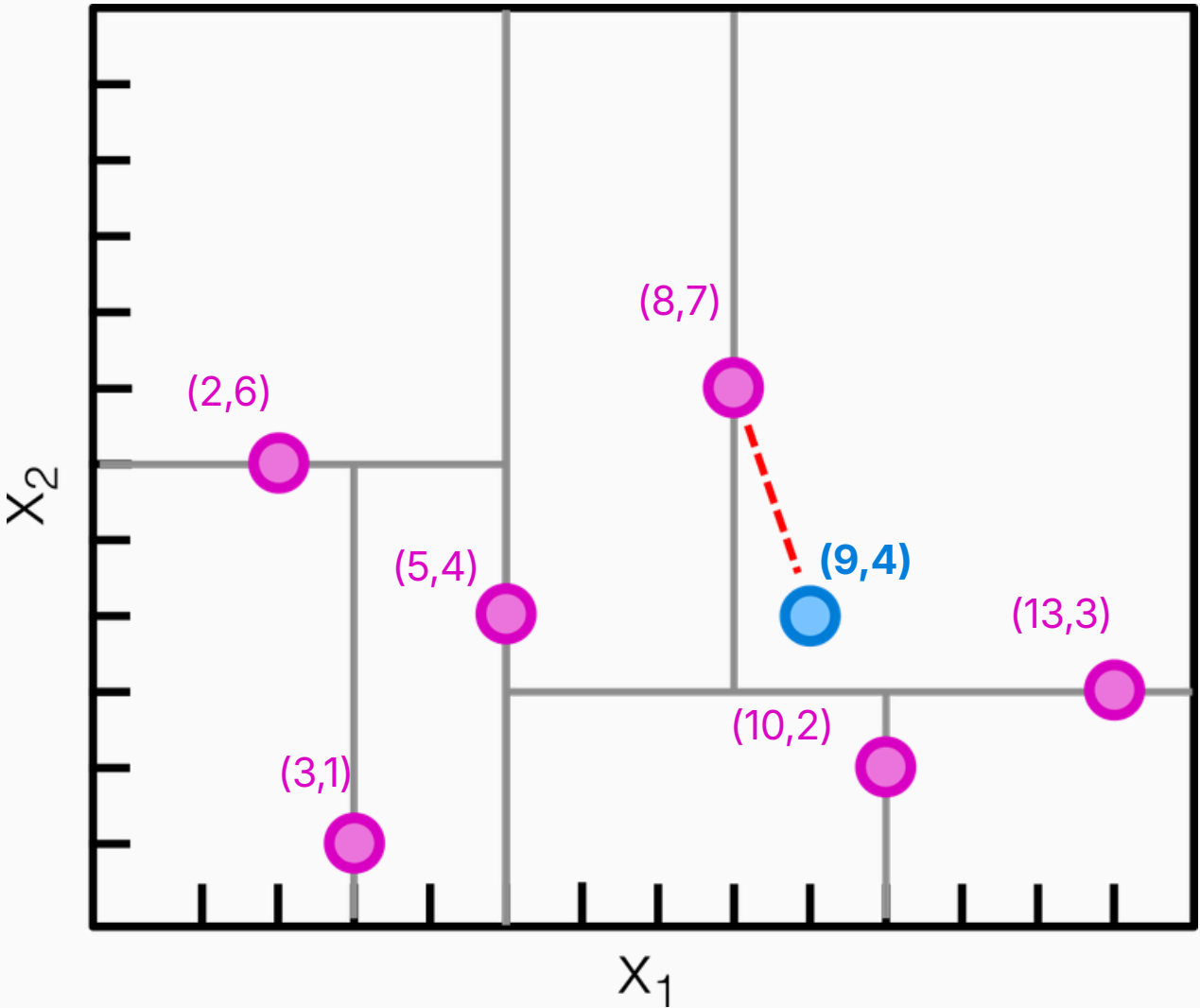


Encontrando a los vecinos: K-D Tree

- Calculamos la distancia del punto a ese nodo. Es el mejor candidato hasta el momento.

	x1	x2
P1	2	6
P2	3	1
P3	5	4
P4	8	7
P5	10	2
P6	13	3

<i>p</i>	9	4
----------	---	---

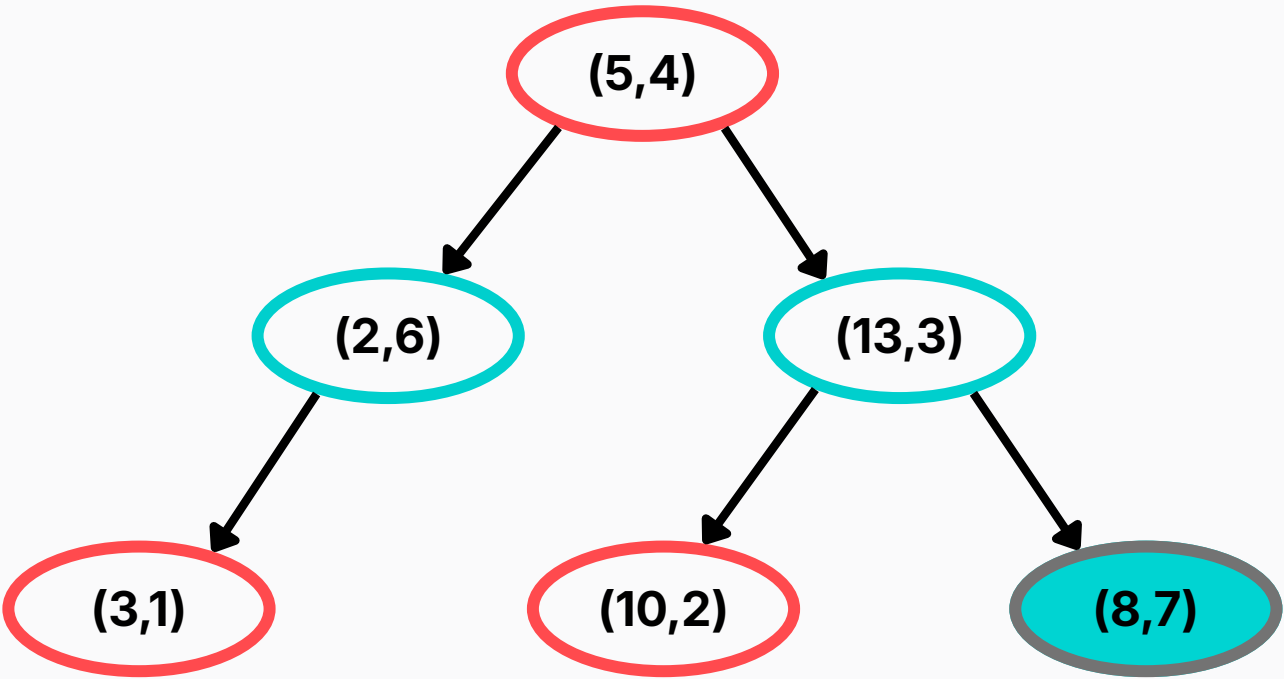
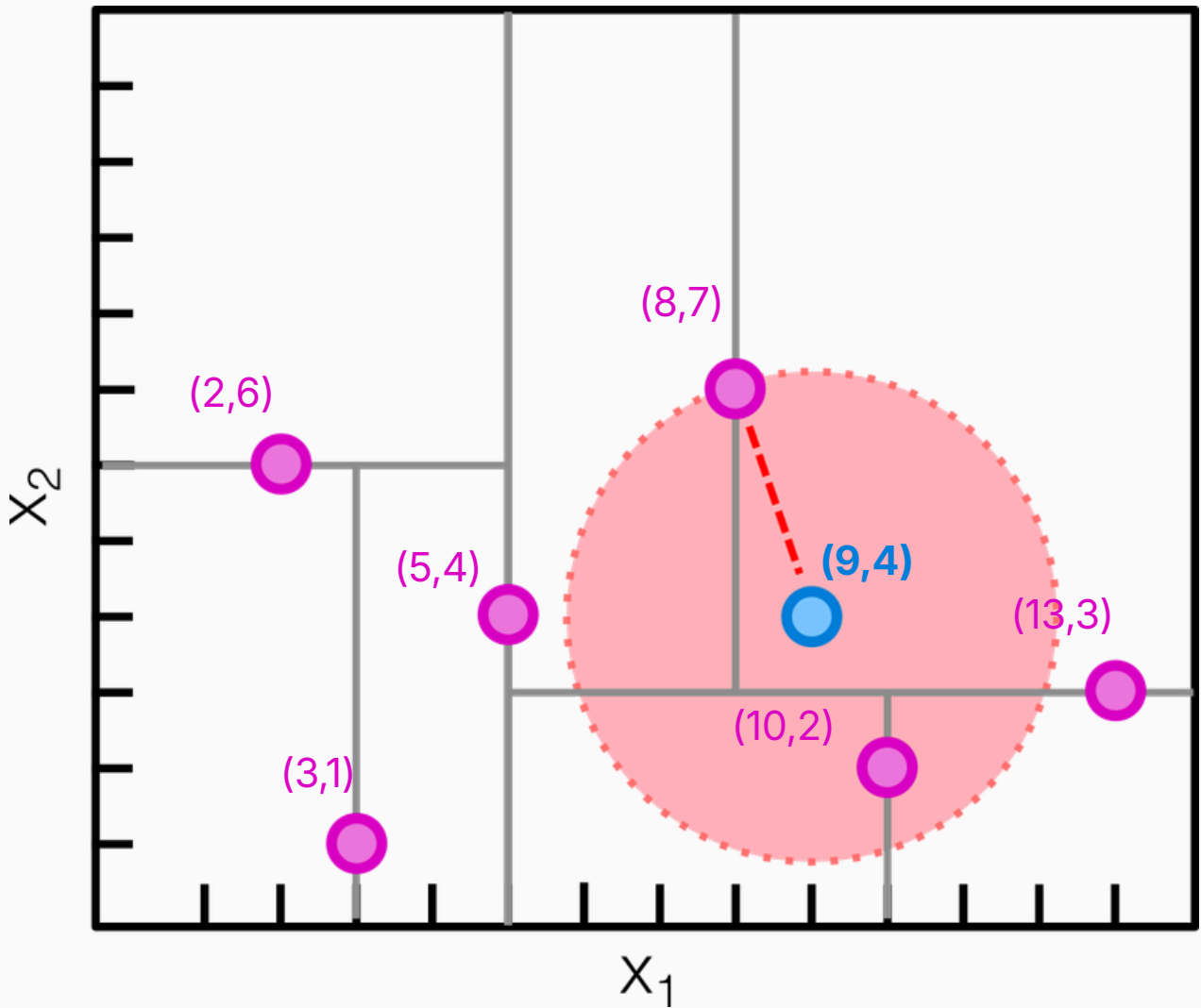


Encontrando a los vecinos: K-D Tree

- Comenzamos a subir por el árbol revisando los nodos más altos. En cada paso se compara la distancia actual al mejor candidato con la distancia al plano de separación utilizado en ese nodo.

	x1	x2
P1	2	6
P2	3	1
P3	5	4
P4	8	7
P5	10	2
P6	13	3

p	9	4
-----	---	---

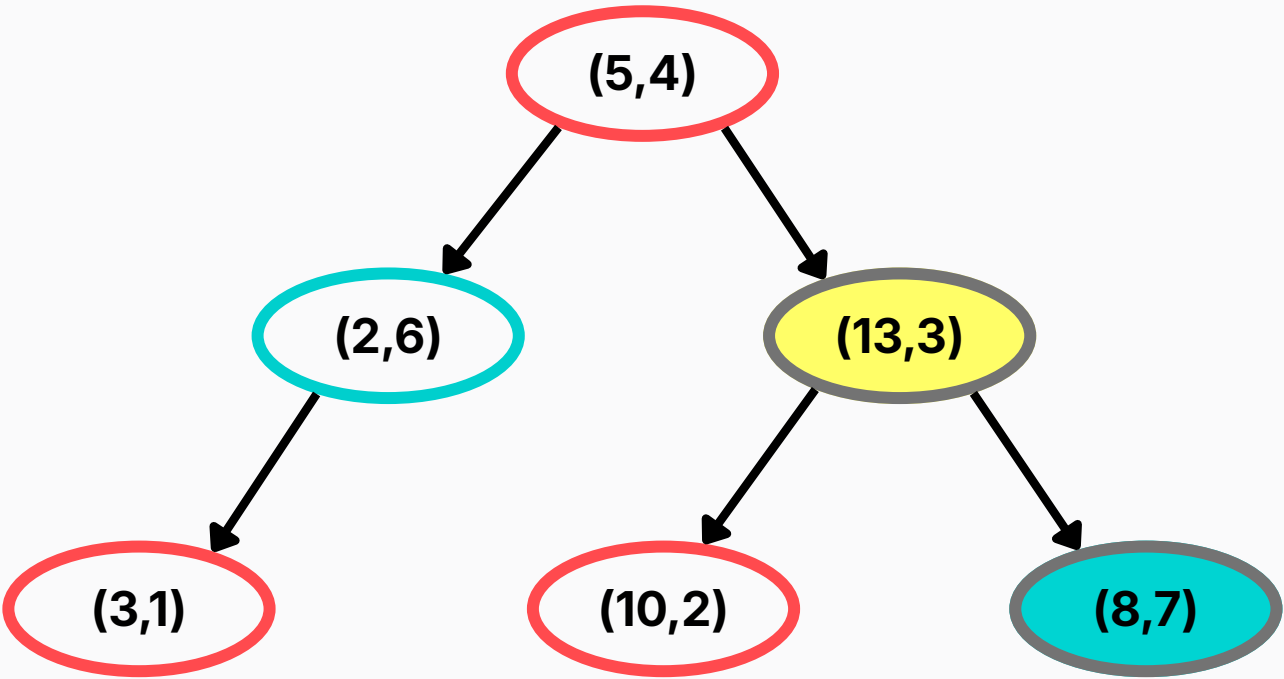
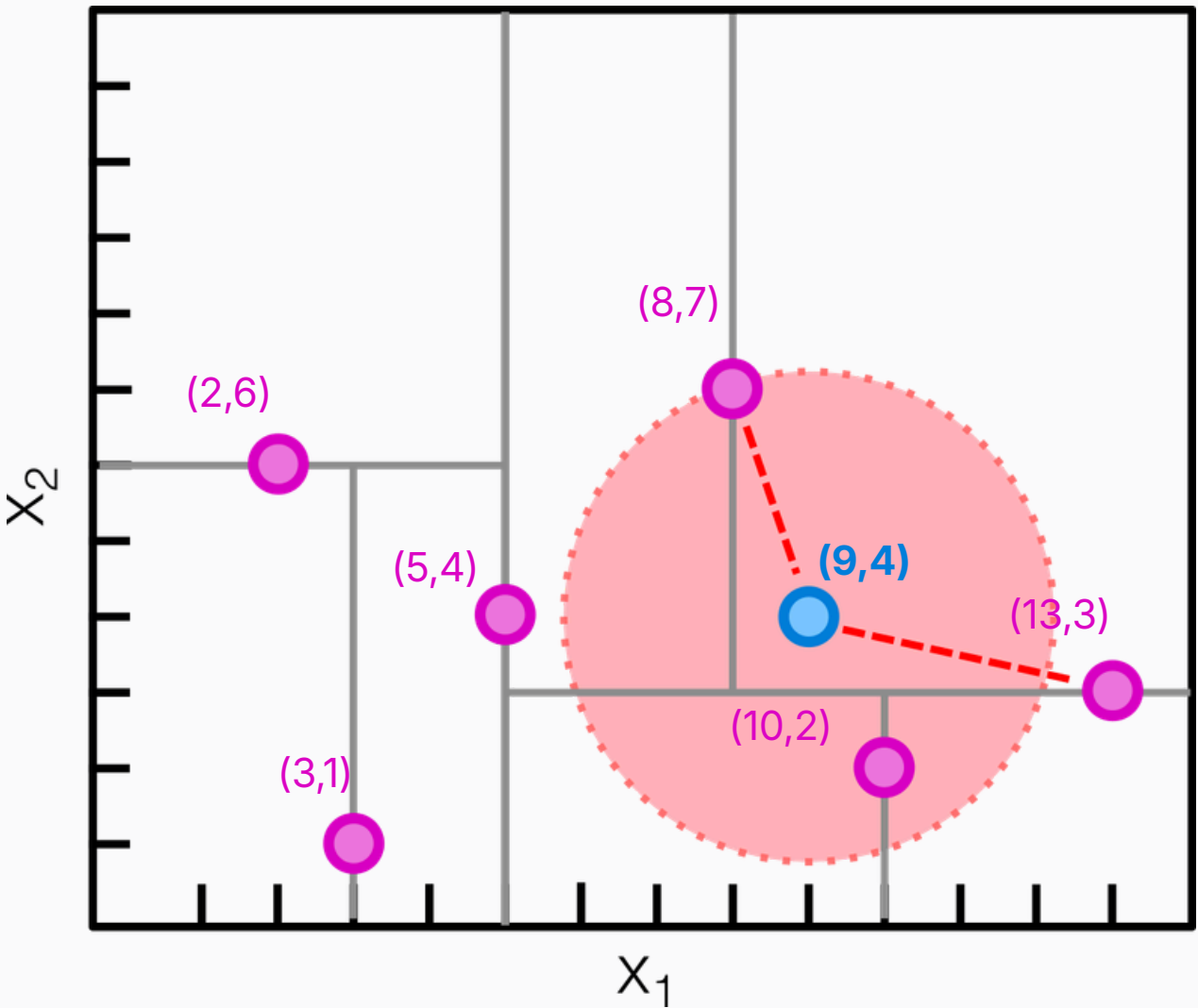


Encontrando a los vecinos: K-D Tree

- Calculamos la distancia del punto al nuevo nodo. Es mayor que la que tenemos hasta ahora. Entonces la descartamos.

	x1	x2
P1	2	6
P2	3	1
P3	5	4
P4	8	7
P5	10	2
P6	13	3

<i>p</i>	9	4
----------	---	---

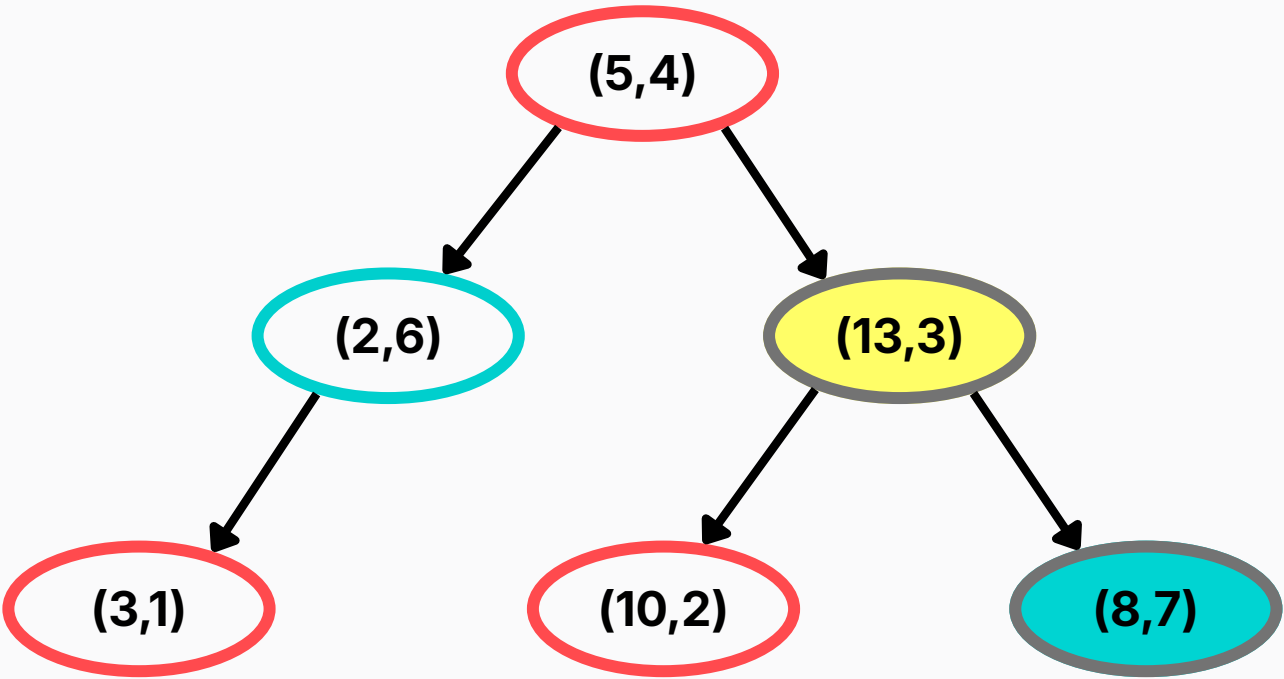
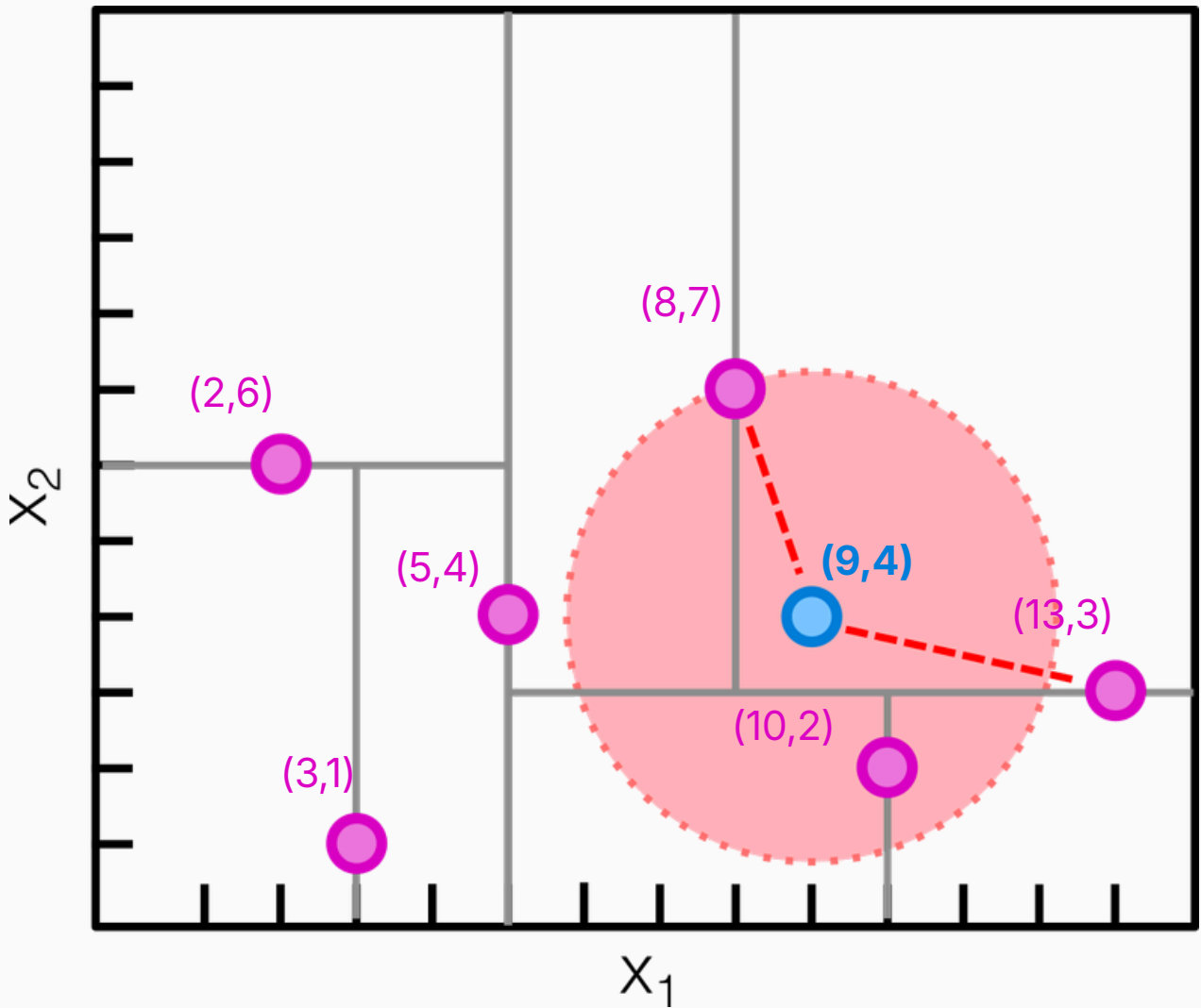


Encontrando a los vecinos: K-D Tree

- ¿Vale la pena explorar la otra rama? Comparamos la distancia actual al mejor candidato con la **distancia al plano de separación** utilizado en ese nodo.

	x1	x2
P1	2	6
P2	3	1
P3	5	4
P4	8	7
P5	10	2
P6	13	3

p	9	4
---	---	---

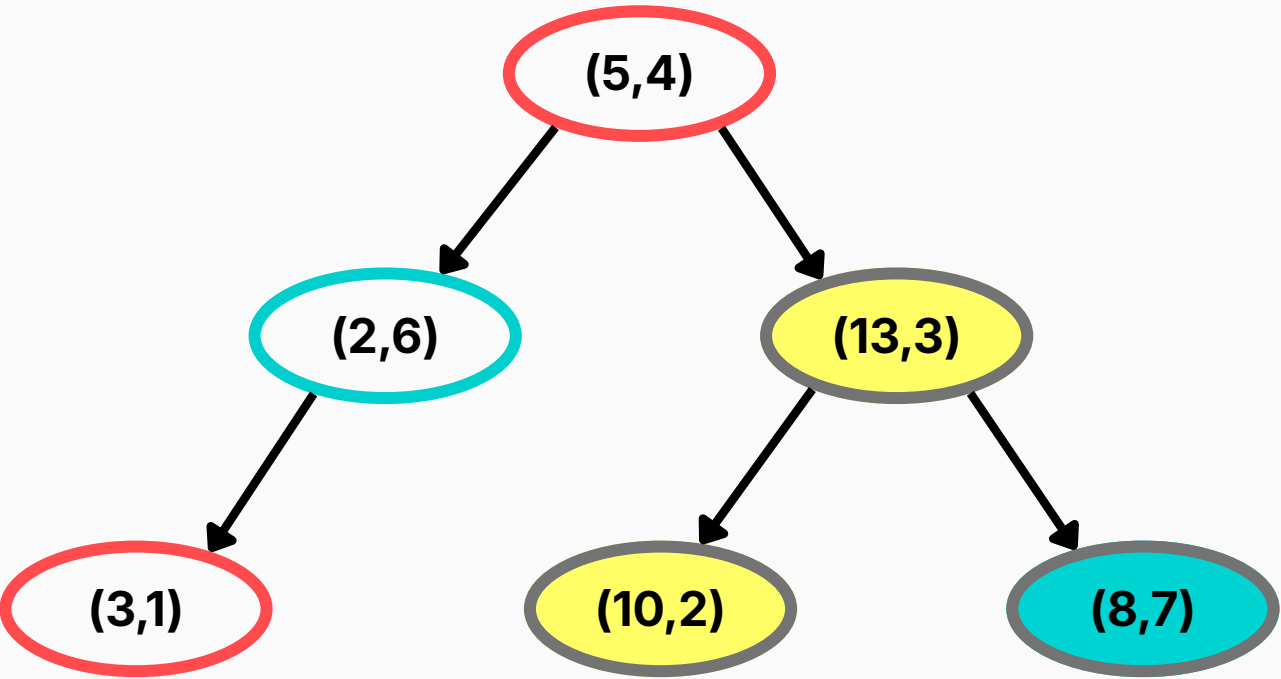
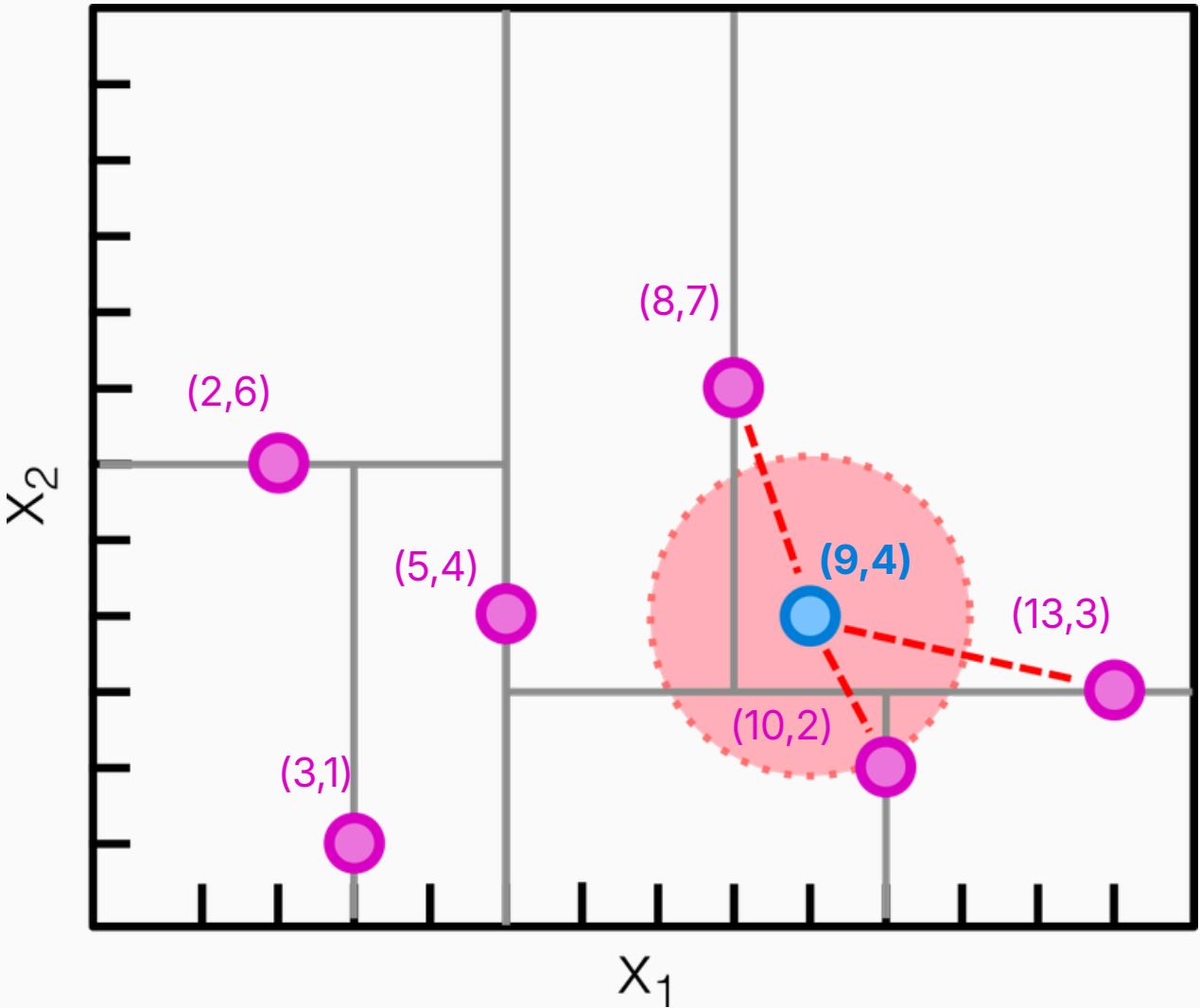


Encontrando a los vecinos: K-D Tree

- Como el plano de separación está más cerca que nuestro mejor candidato, sabemos que esa otra rama puede contener un punto más cercano y la exploramos.

	x1	x2
P1	2	6
P2	3	1
P3	5	4
P4	8	7
P5	10	2
P6	13	3

<i>p</i>	9	4
----------	---	---

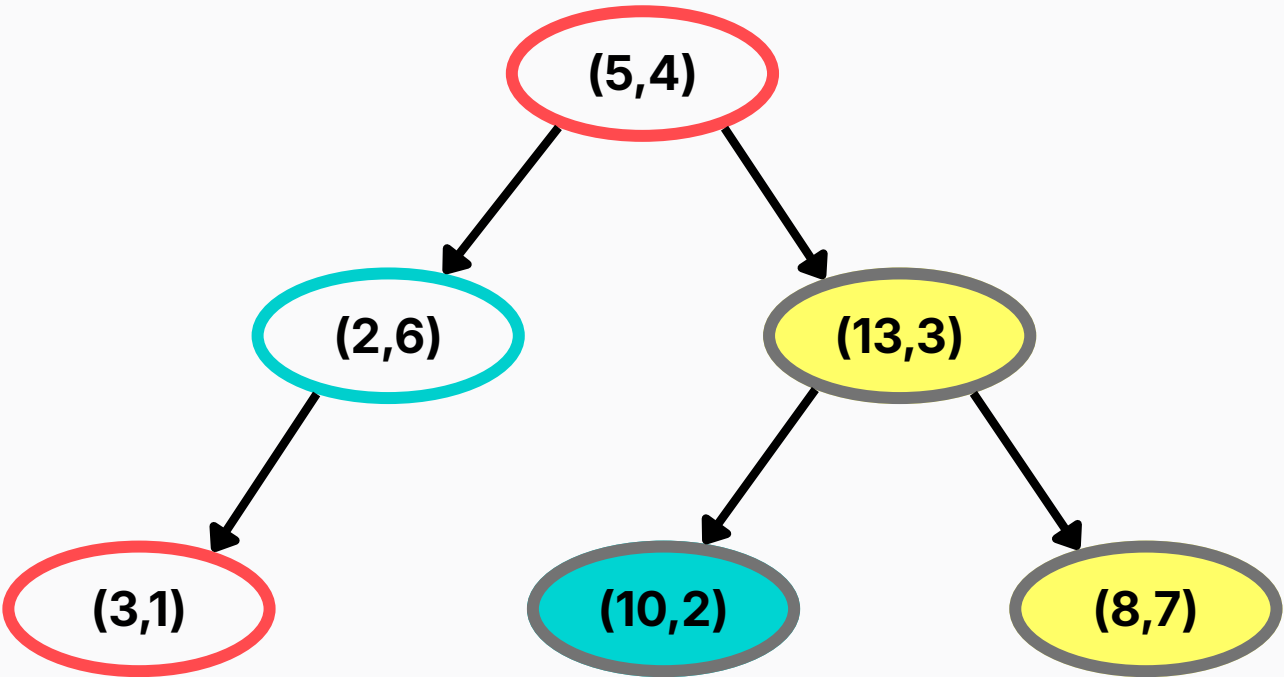
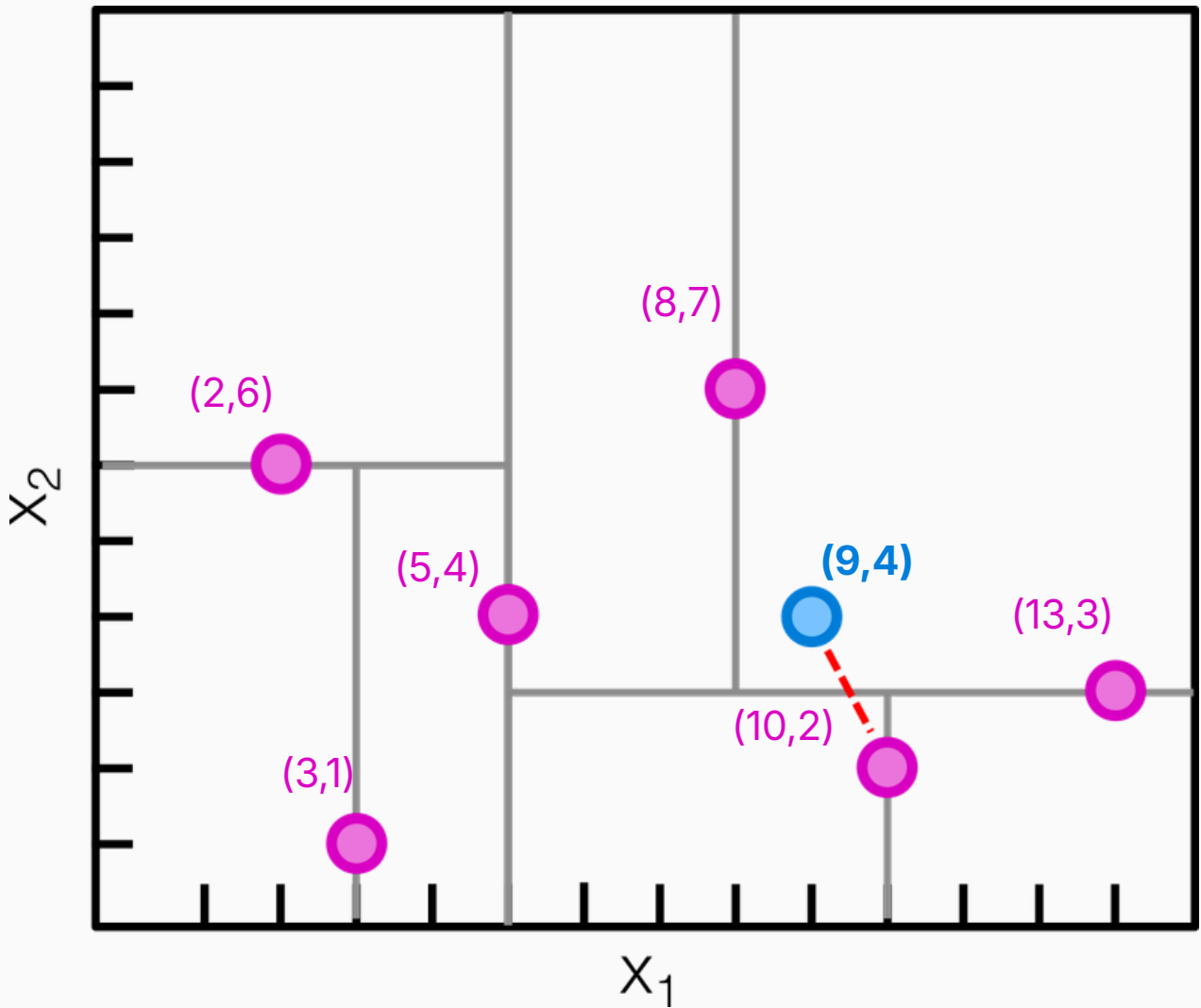


Encontrando a los vecinos: K-D Tree

- La distancia a este nodo es menor que la anterior. Tenemos un nuevo candidato.

	x1	x2
P1	2	6
P2	3	1
P3	5	4
P4	8	7
P5	10	2
P6	13	3

p	9	4
---	---	---

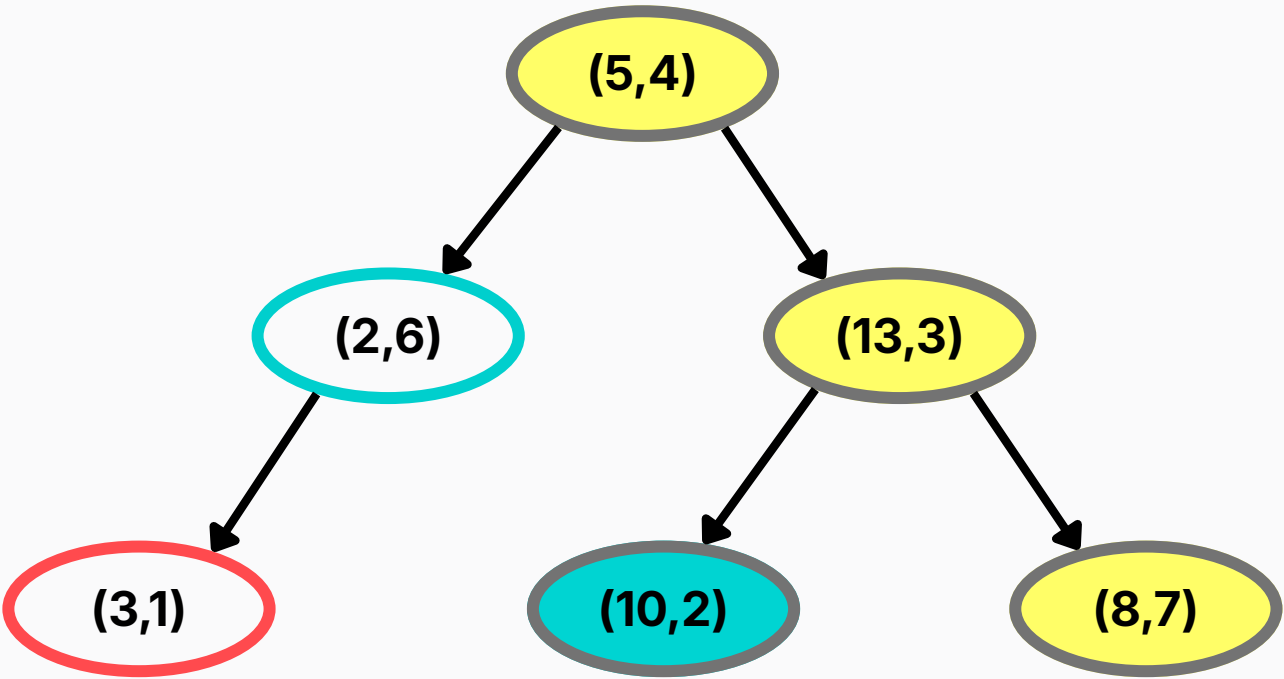
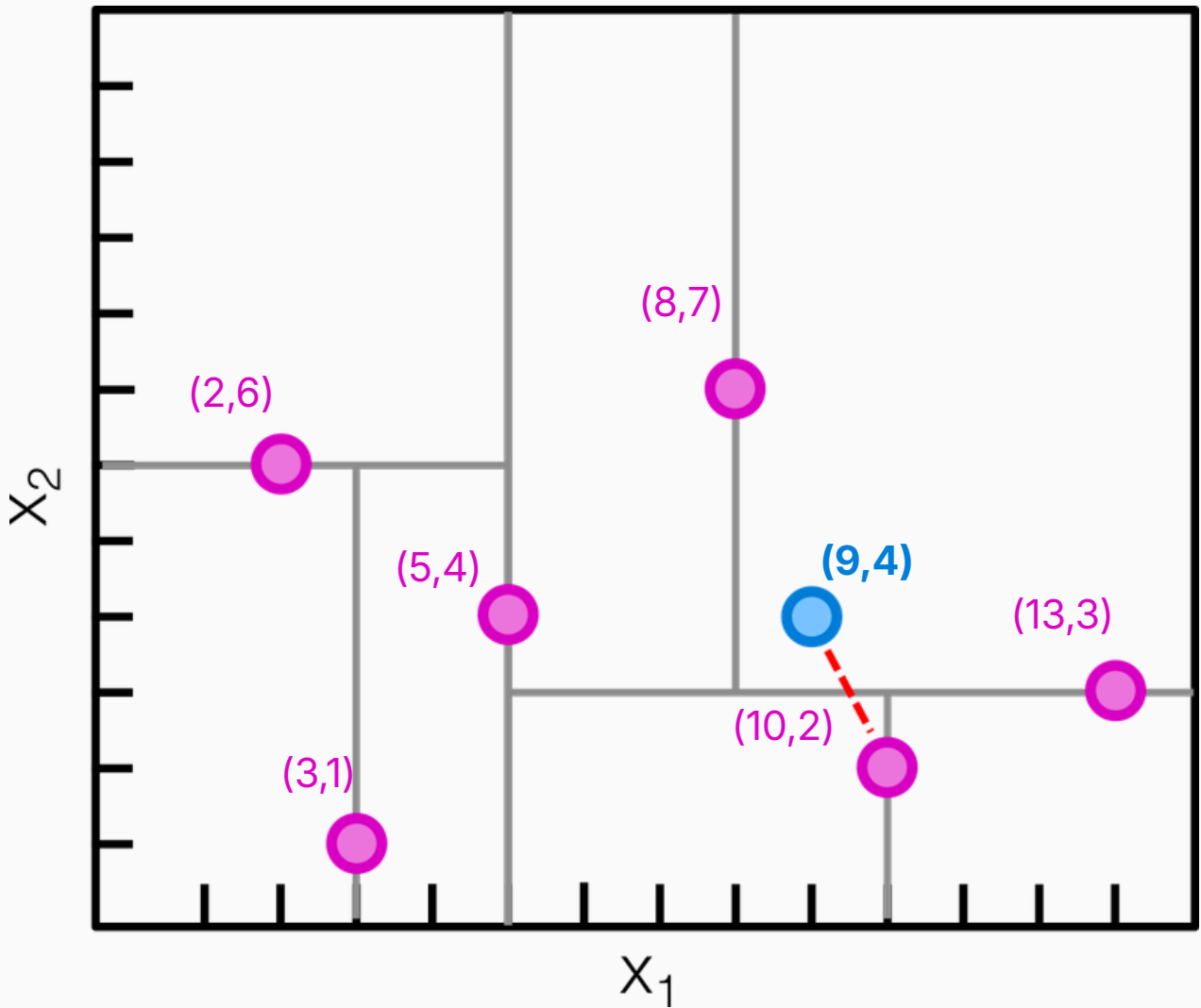


Encontrando a los vecinos: K-D Tree

- Seguimos retrocediendo en el árbol y repetimos los pasos. Calculamos la distancia del punto al nuevo nodo, es mayor que la que tenemos hasta ahora, la descartamos.

	x1	x2
P1	2	6
P2	3	1
P3	5	4
P4	8	7
P5	10	2
P6	13	3

<i>p</i>	9	4
----------	---	---

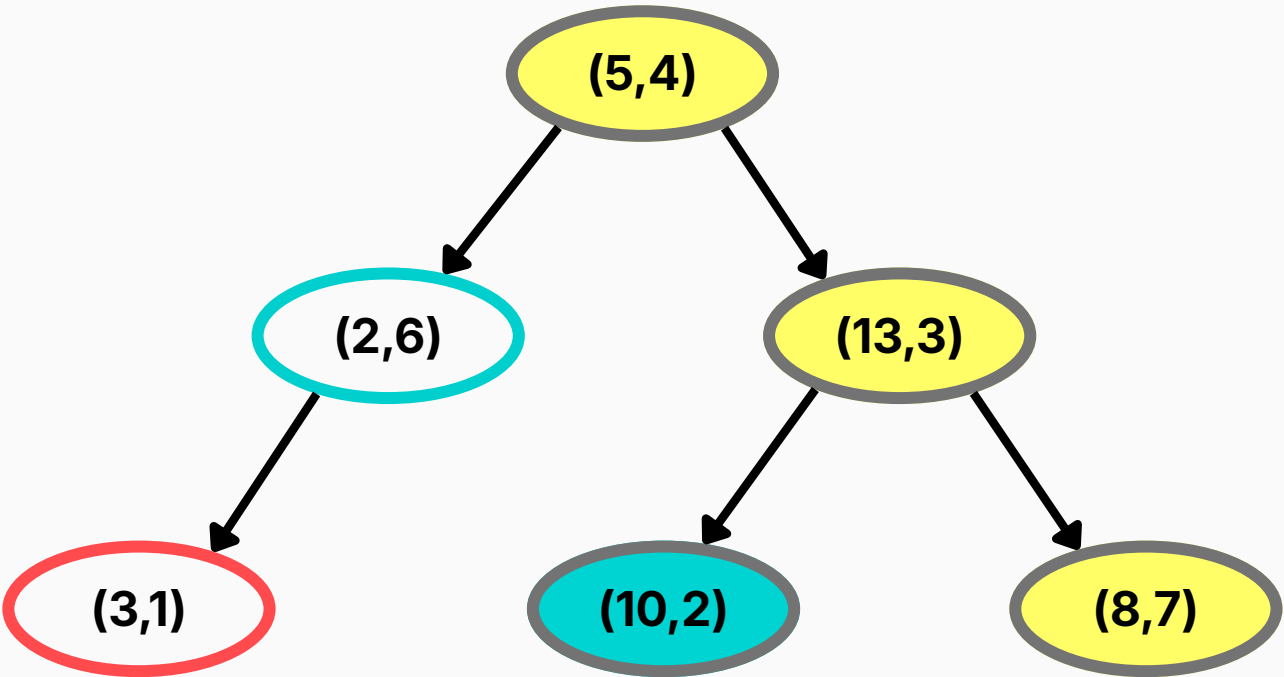
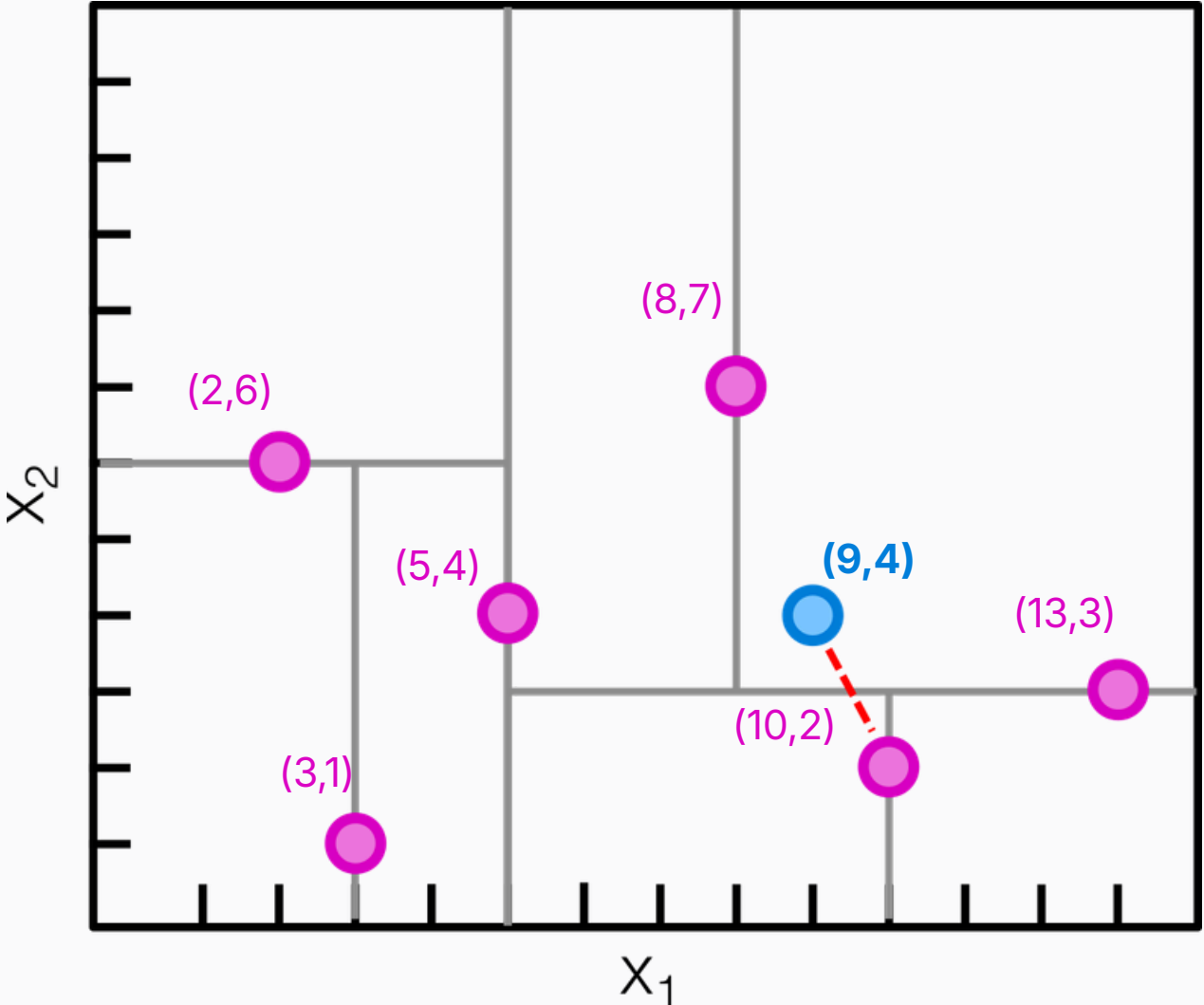


Encontrando a los vecinos: K-D Tree

- Comparamos la distancia actual al mejor candidato con la distancia al plano de separación utilizado en ese nodo. Como el plano de separación está más lejos, sabemos que la otra rama no puede contener un punto más cercano y la descartamos.

	x1	x2
P1	2	6
P2	3	1
P3	5	4
P4	8	7
P5	10	2
P6	13	3

<i>p</i>	9	4
----------	---	---

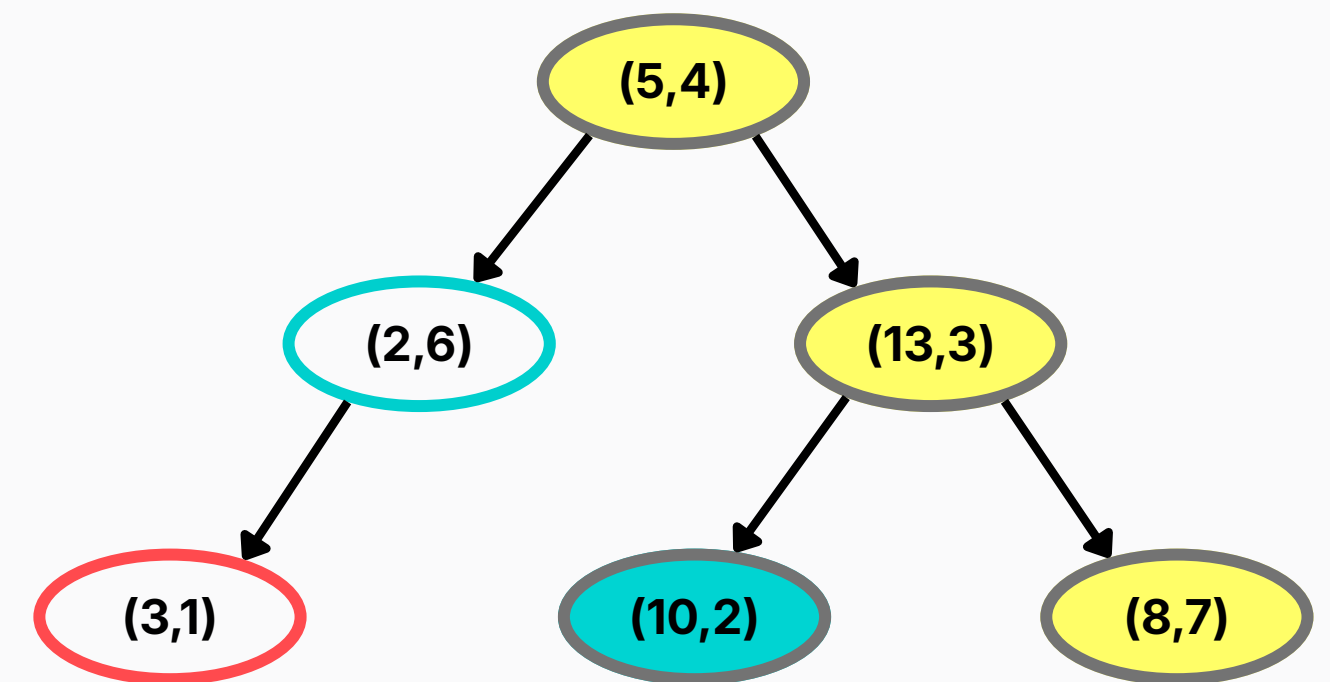
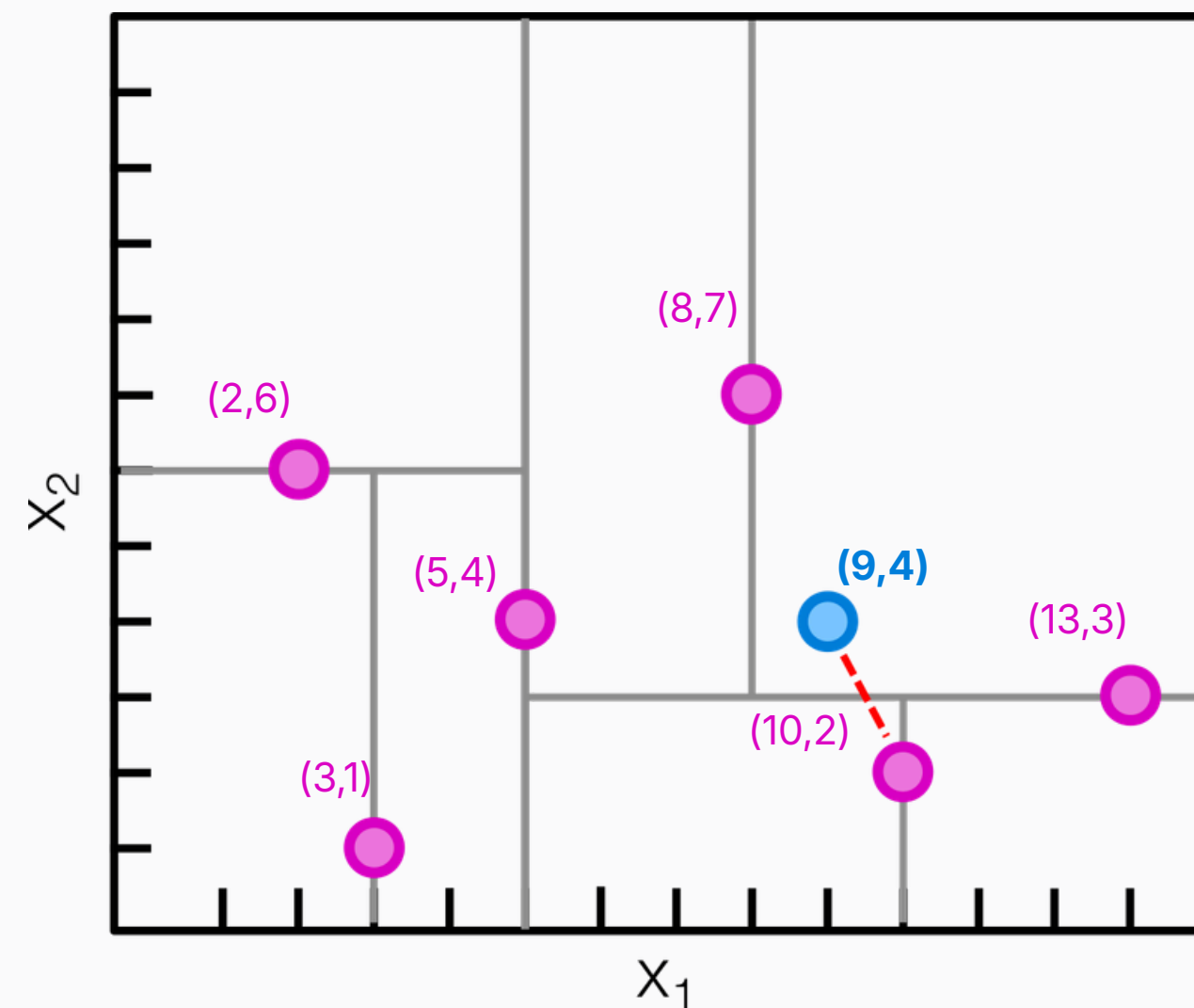


Encontrando a los vecinos: K-D Tree

- Como no hay más nodos, termina nuestra búsqueda y encontramos el más cercano con muy pocos cálculos.

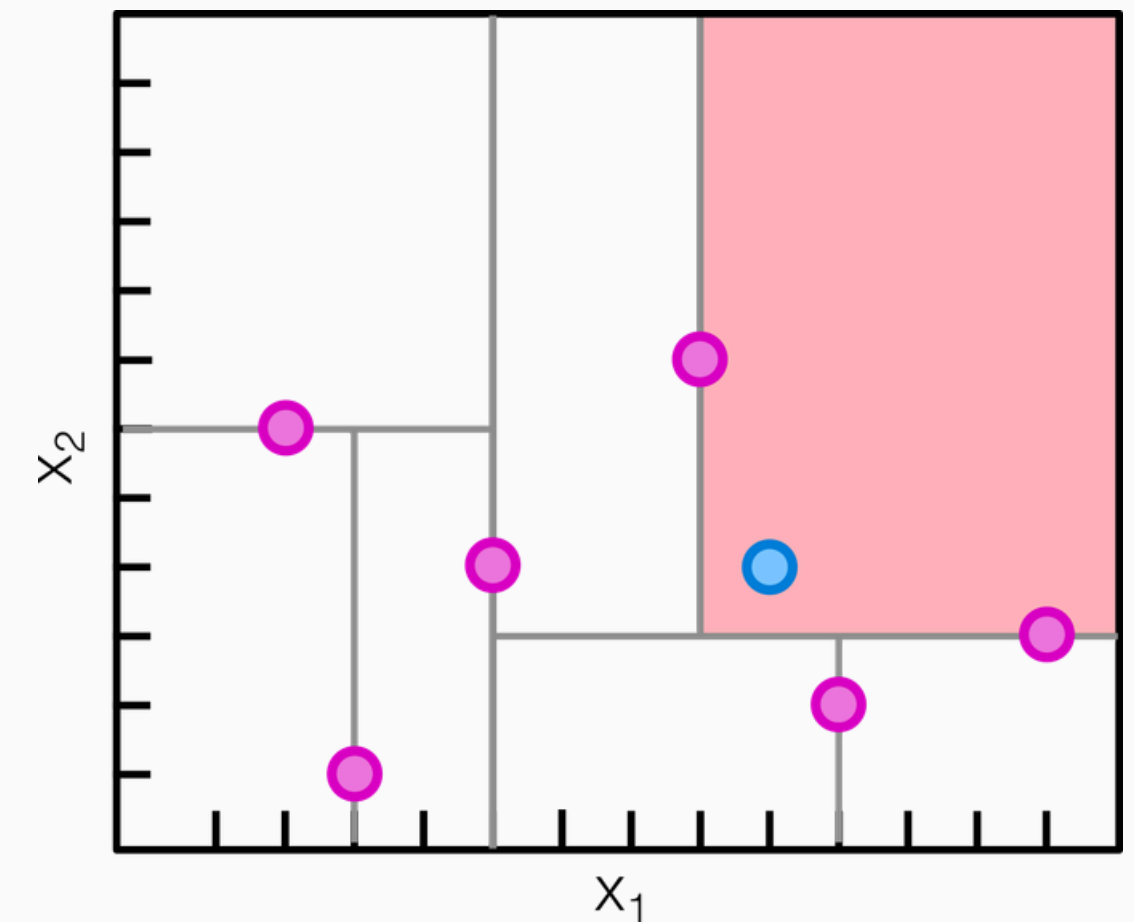
	x1	x2
P1	2	6
P2	3	1
P3	5	4
P4	8	7
P5	10	2
P6	13	3

p	9	4
-----	---	---



Encontrando a los vecinos: K-D Tree

- Este algoritmo, con pequeños cambios, se puede usar para encontrar los K-vecinos más cercanos.
- El entrenamiento es el armado de este árbol.
- En el caso general, en las regiones puede haber más de un punto. En cada celda se puede hacer una búsqueda de fuerza bruta, aunque con muchos menos puntos.



Encontrando a los vecinos: K-D Tree

El algoritmo K-D Tree funciona bien para baja dimensionalidad, pero en grandes dimensiones sufre del efecto de la maldición de dimensión. En grandes dimensiones y tal como se estructura el árbol, se termina en casos tan malos como la búsqueda en fuerza bruta.

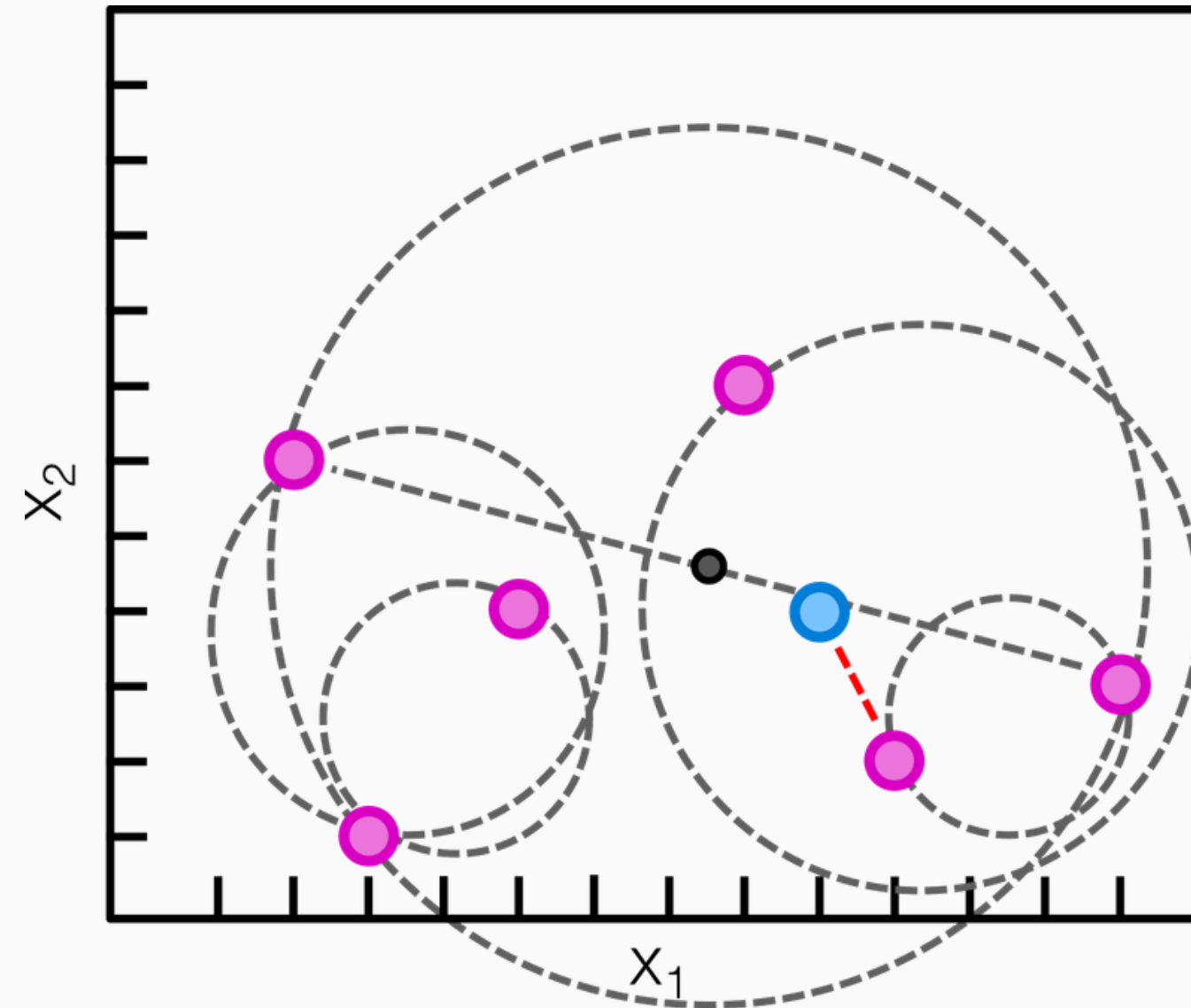
Para intentar resolver esto, se utiliza **Ball Tree**.

La **maldición de dimensionalidad** advierte que a medida que aumenta el número de dimensiones, los datos se vuelven más dispersos y los algoritmos basados en la distancia pueden perder precisión o eficiencia.

Encontrando a los vecinos: Ball Tree

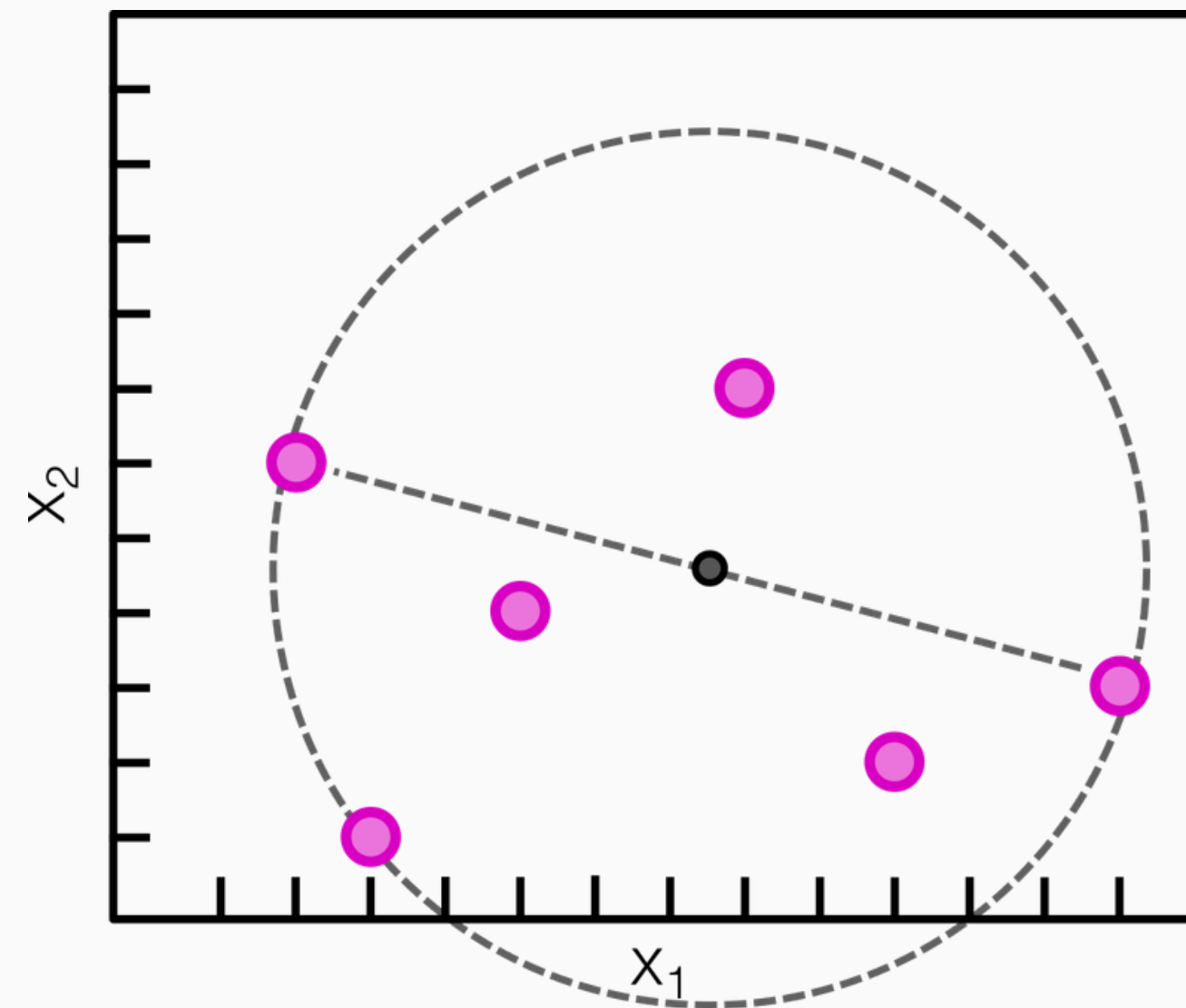
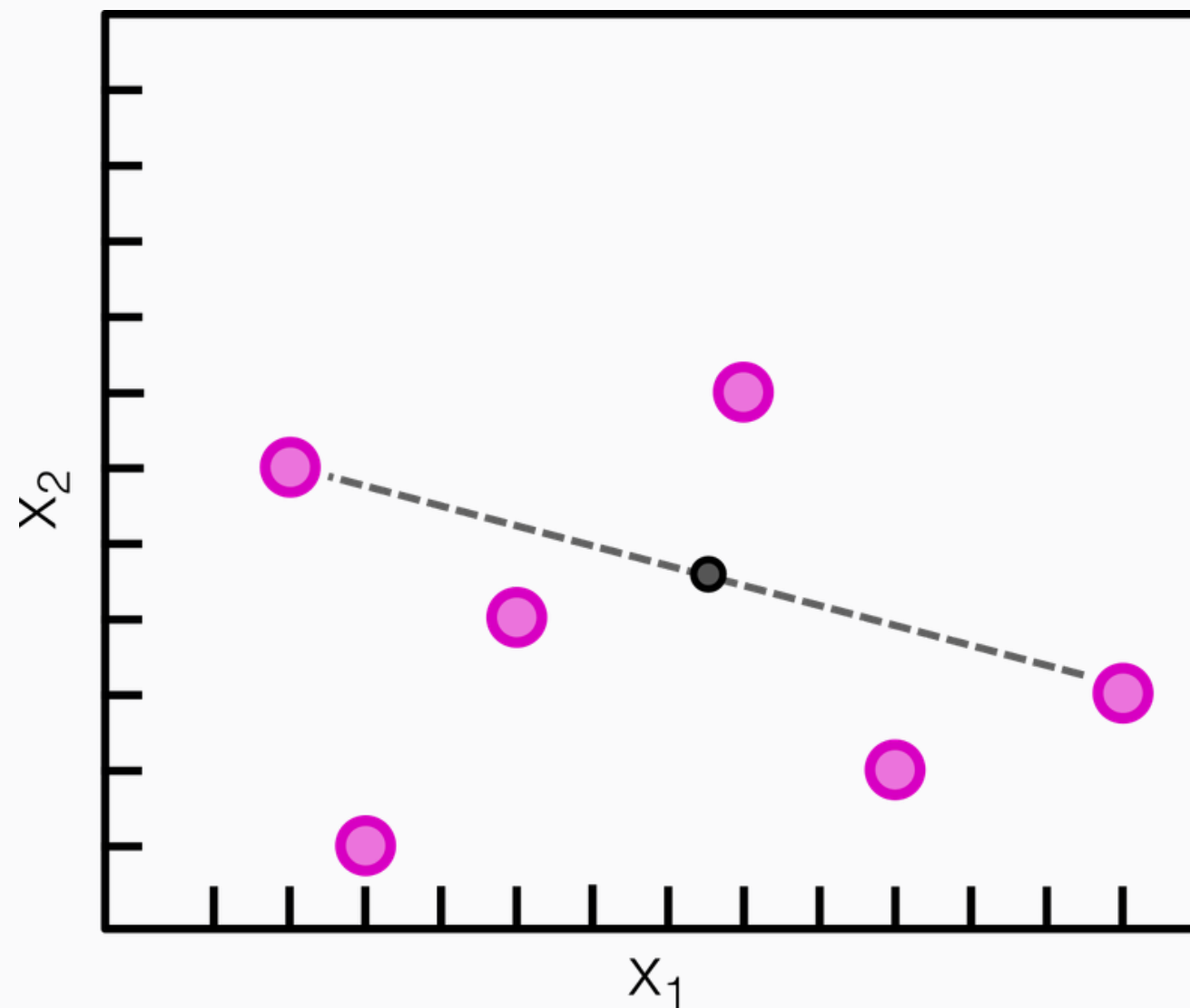
En vez de usar regiones divididas a lo largo de los ejes cartesianos, los **Ball Tree** dividen recursivamente los datos en una serie de hiperesferas anidadas.

Esto hace que la construcción del árbol sea más costosa, pero resulta que puede ser muy eficiente en datos en dimensiones muy altas.



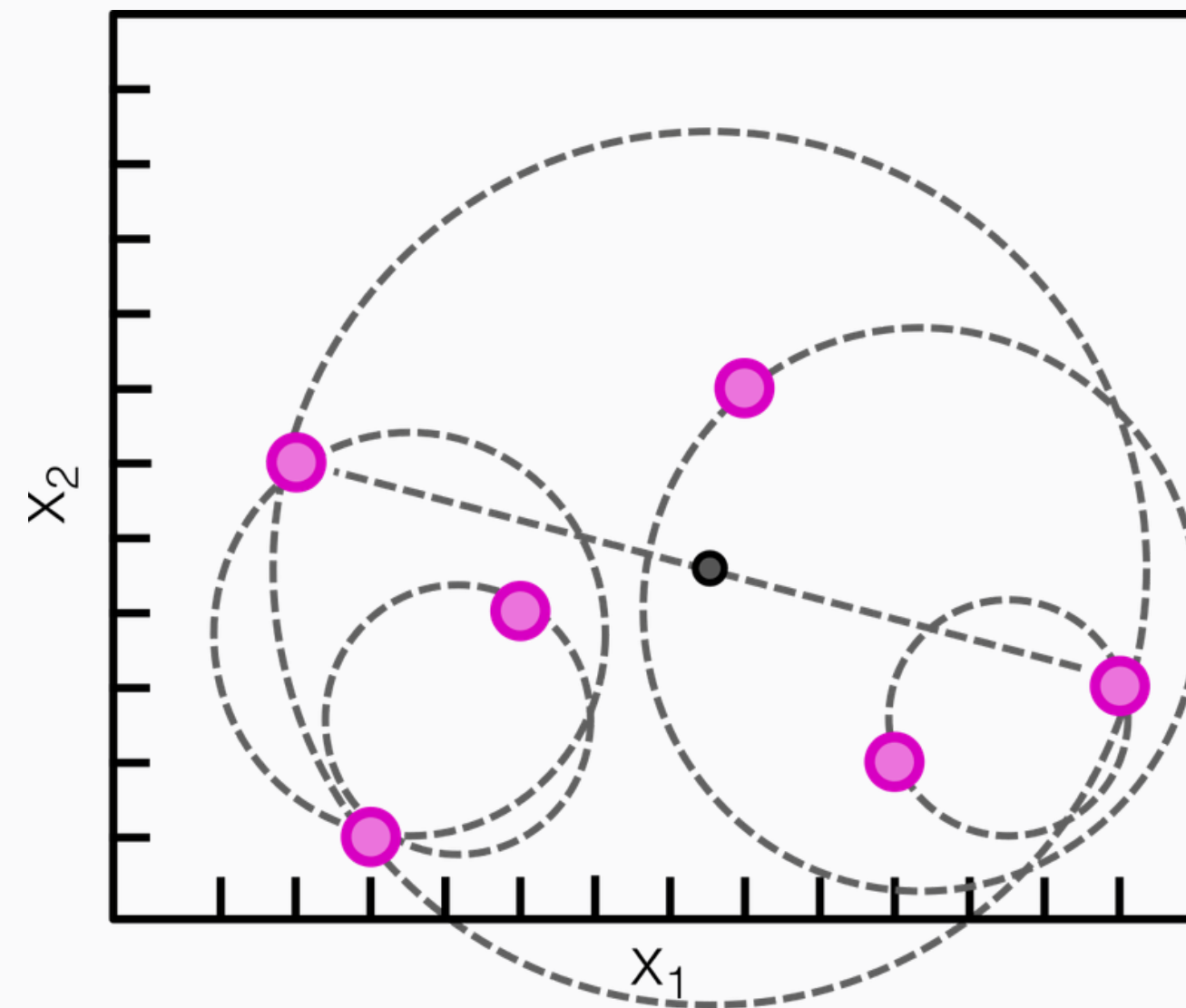
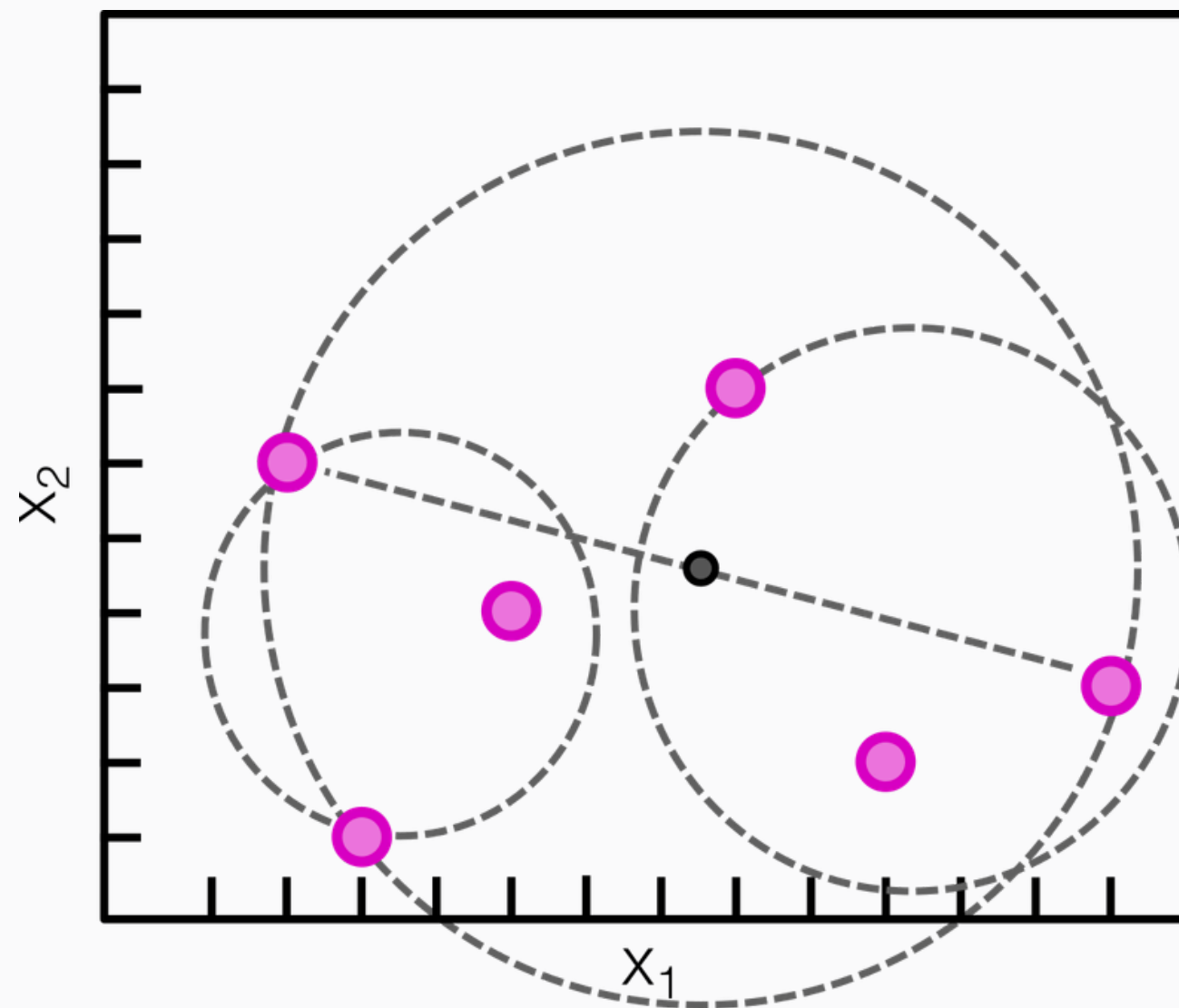
Encontrando a los vecinos: Ball Tree

Inicialmente se buscan los dos puntos más alejados. Luego se construye una circunferencia entre ellos.



Encontrando a los vecinos: Ball Tree

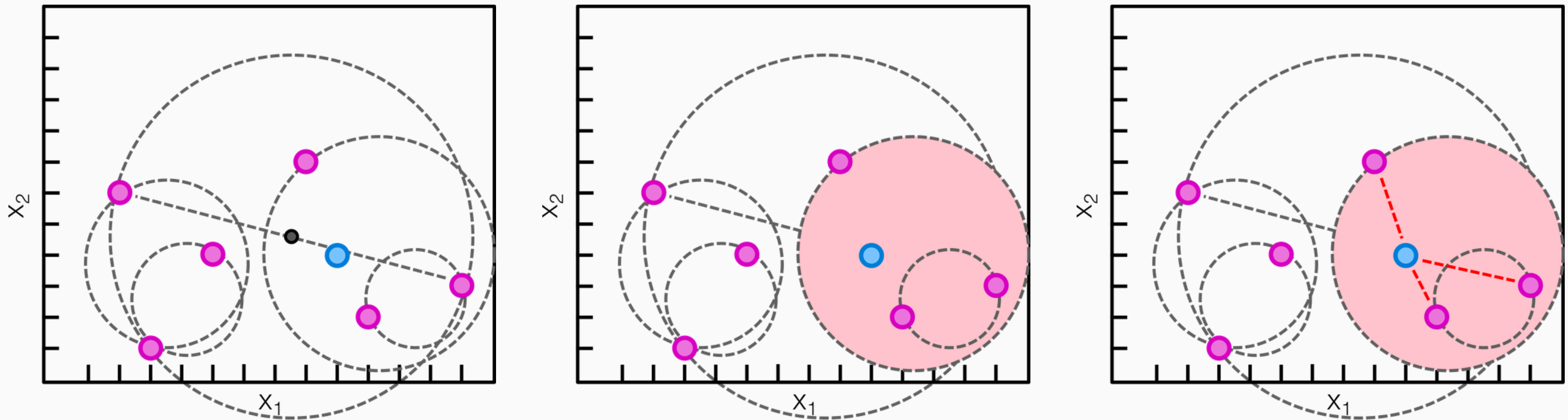
Similar a T-D Tree, se separa en mitades y mitades, pero ahora lo hacemos en esferas en vez de los ejes cartesianos.



Encontrando a los vecinos: Ball Tree

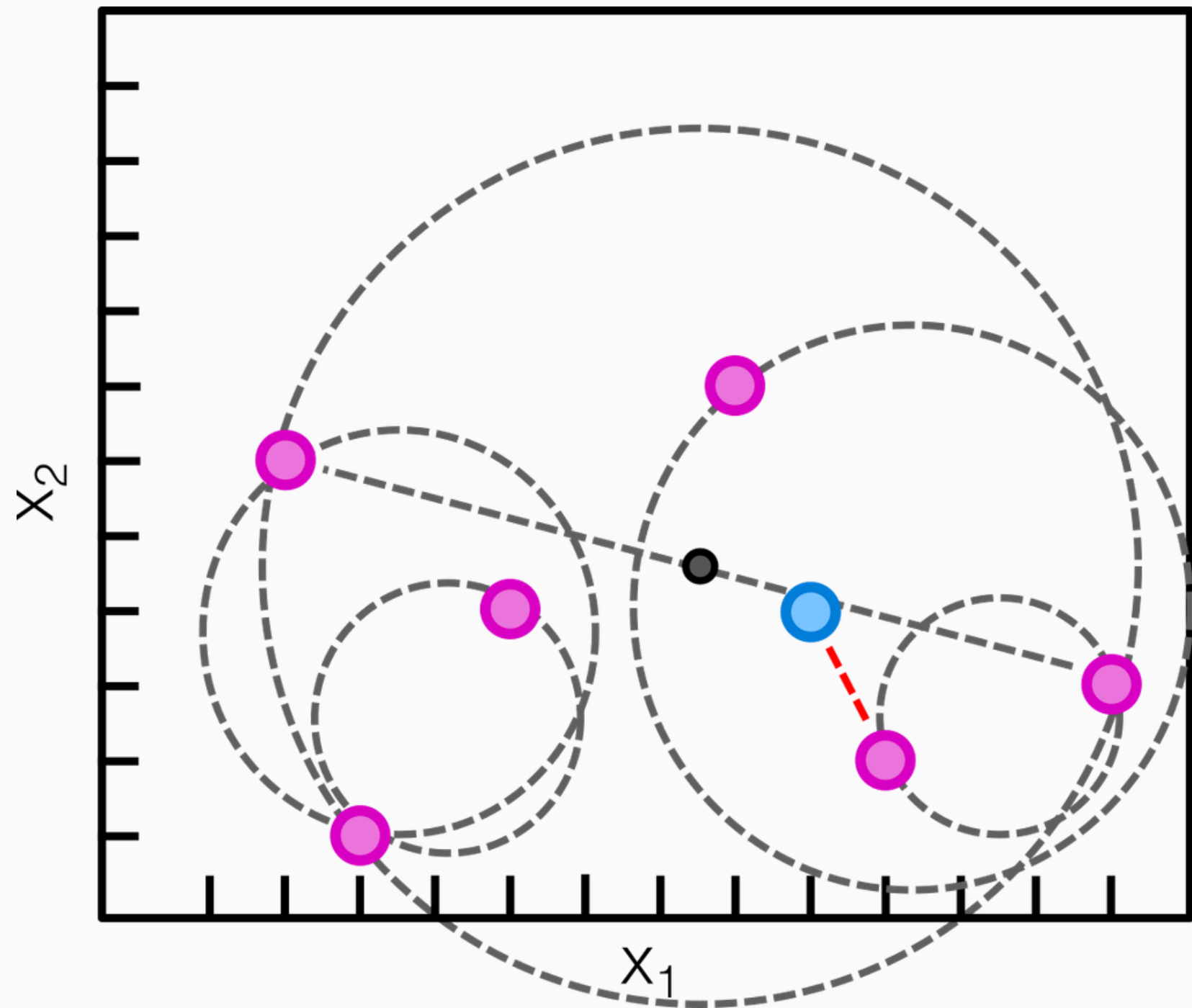
Una vez construido el árbol, cuando tenemos un nuevo dato, vemos cuál es la esfera que lo contiene (la más pequeña).

Y en esta esfera calculamos las distancias de quienes estén en esa esfera.



Encontrando a los vecinos: Ball Tree

Esta estructura es más pesada y lleva más tiempo de entrenamiento para almacenamiento y para navegarse el árbol cuando buscamos un parámetro nuevo.



Encontrando a los vecinos: Ball Tree

Scikit-Learn implementa estos tres tipos de estructuras para k-NN (y otros algoritmos como k-means).

Si se deja que elija automáticamente, lo que hace esta librería es:

- Si son pocos datos, usa fuerza bruta.
- Si son muchos, pero de baja dimensión, usa K-D Tree.
- Alta dimensionalidad, utiliza Ball Tree.



Vamos a practicar un
poco ...

MÉTODOS DE AJUSTE DE LOS HIPERPARÁMETROS

Métodos de ajuste de hiperparámetros

- Recordando de Inteligencia Artificial, los **hiperparámetros** son aquellos parámetros que caracterizan un modelo de aprendizaje automático, cuyos valores no se ajustan de forma automática durante el entrenamiento.
- El ajuste de los hiperparámetros tiene gran **impacto en el desempeño** del modelo.
- Los hiperparámetros tienen que ser optimizados para cada conjunto de datos ya que los hiperparámetros óptimos **difieren para distintos conjuntos de datos**.

Métodos de ajuste de hiperparámetros

- El proceso de selección de hiperparámetros se conoce como **ajuste de hiperparámetros** u **optimización de hiperparámetros**.
- Los métodos de ajuste tienen como propósito seleccionar los hiperparámetros que **maximizan/minimizan una métrica** de rendimiento.
- La métrica que se intenta maximizar/minimizar depende de:
 - La arquitectura del modelo
 - Los datos
 - Los propios hiperparámetros

Métodos de ajuste de hiperparámetros

En Inteligencia Artificial se vio que validación cruzada nos sirve para ayudarnos a buscar los hiperparámetros que mejor se nos ajustan a nuestros modelos, permitiendo mantener la generalidad.

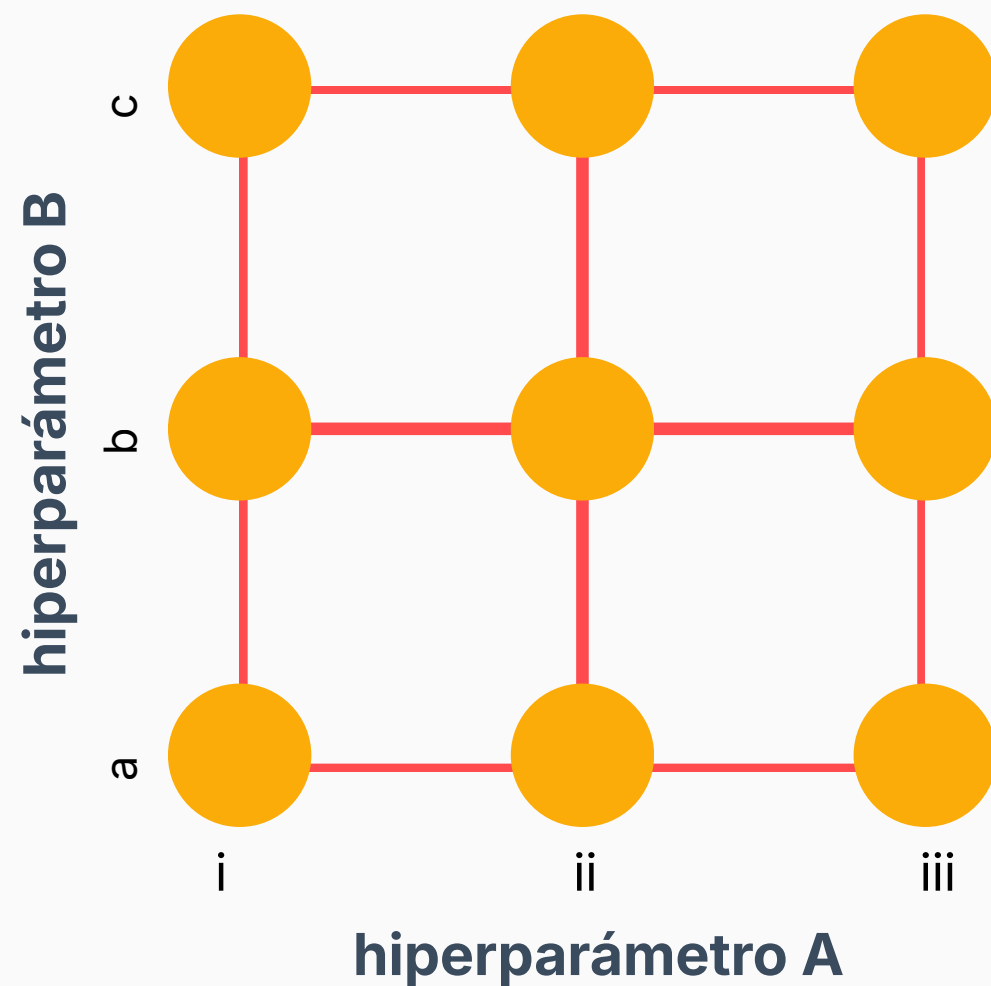
Pero por sí solo, no alcanza. Necesitamos de alguna forma movernos por el espacio de búsqueda. Una búsqueda consiste en:

- Un modelo (de regresión o clasificación)
- Un espacio de parámetros
- **Un método de búsqueda o de muestreo de candidatos**
- Un esquema de validación cruzada
- Una función de puntaje

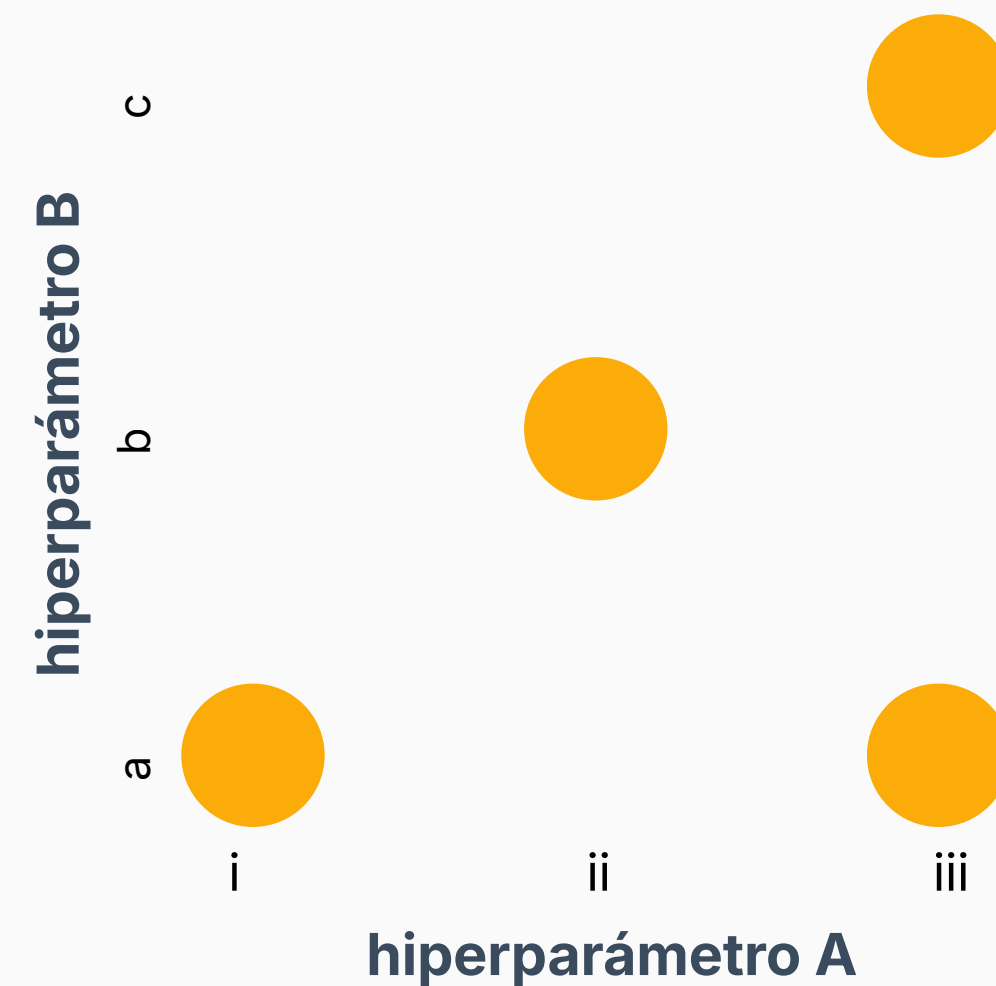
Métodos de ajuste de hiperparámetros

Dos métodos de búsqueda típicos:

Búsqueda de grilla



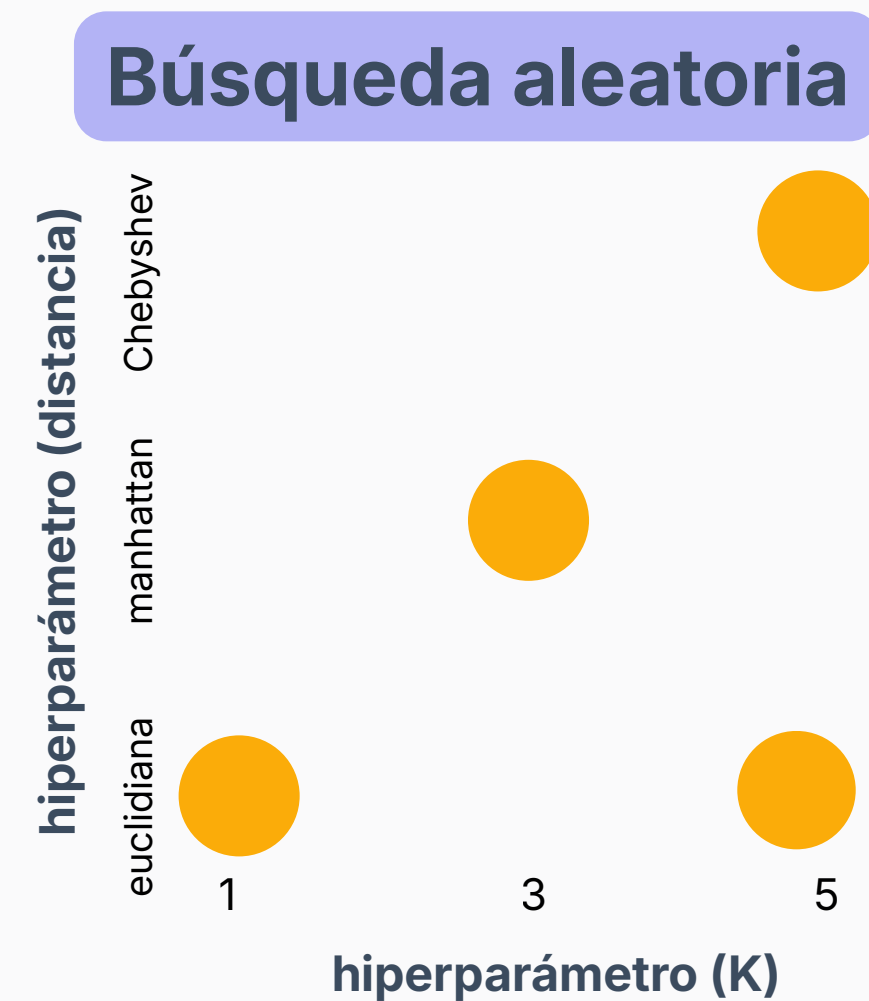
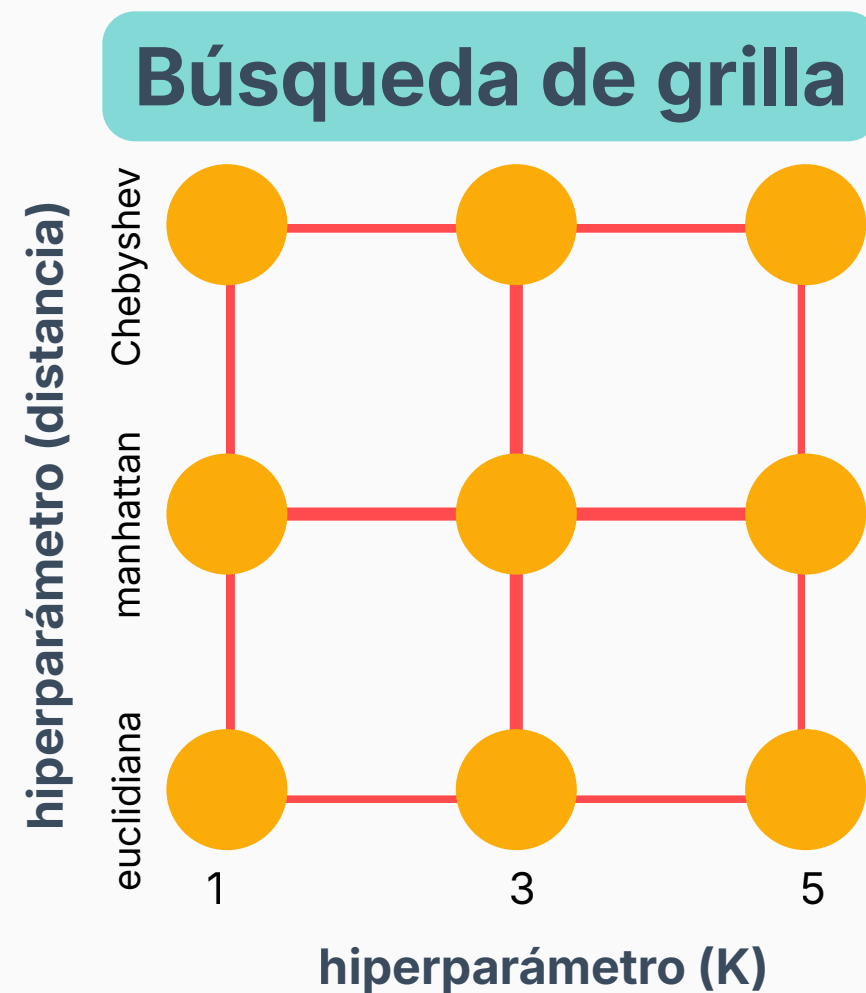
Búsqueda aleatoria



Métodos de ajuste de hiperparámetros

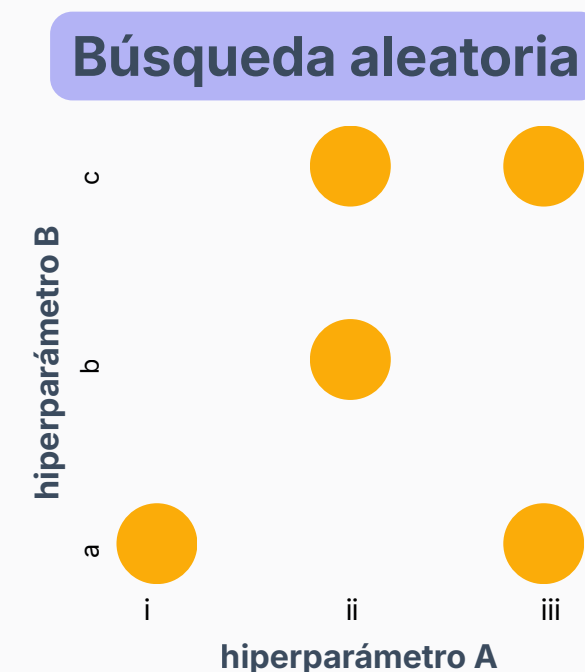
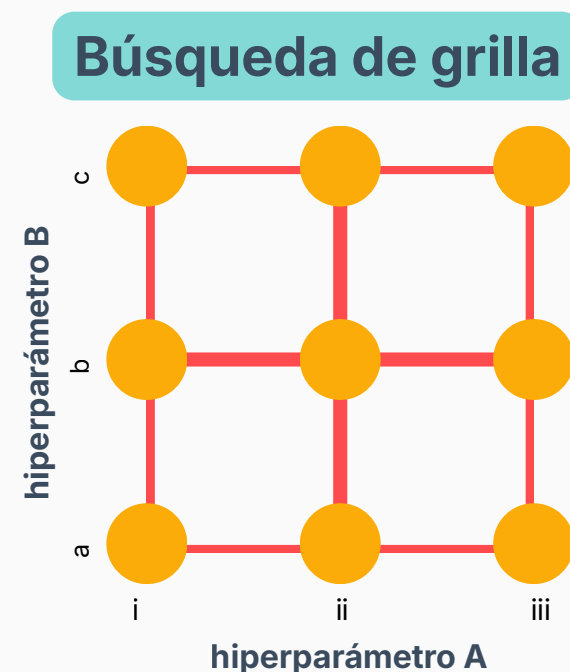
Ejemplo:

KNN con 2 hiperparámetros: método de distancia y número de vecinos (K).



Métodos de ajuste de hiperparámetros

Una mejora que se puede aplicar a ambos métodos, a expensa de una mayor duración, es incorporar la reducción a la **mitad sucesiva** (*successive halving*).



La idea es tener una especie de torneo. En este torneo se arranca buscando los hiperparámetros, pero usando un subset de entrenamiento chico.

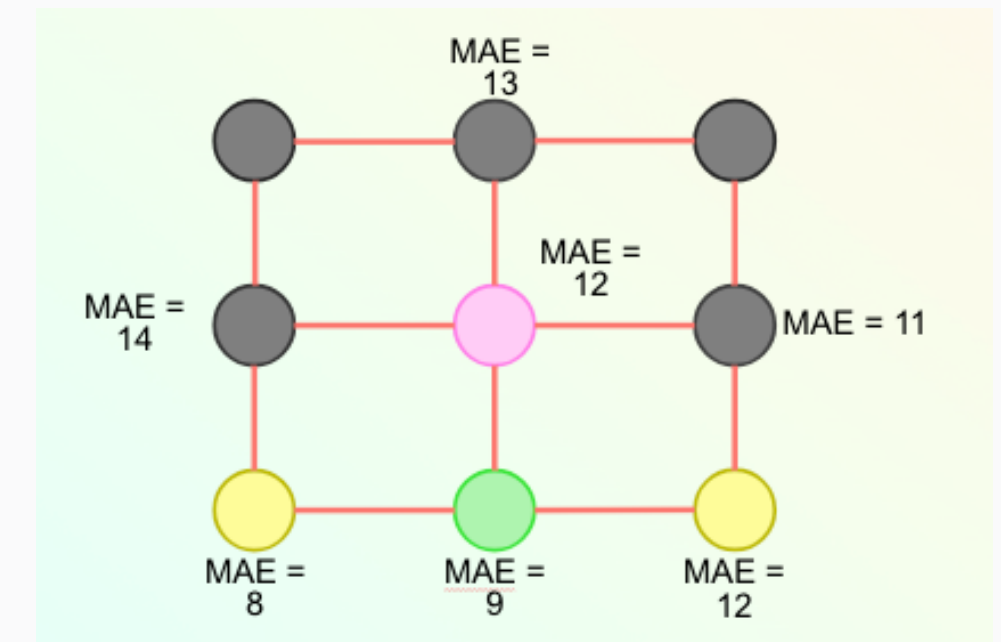
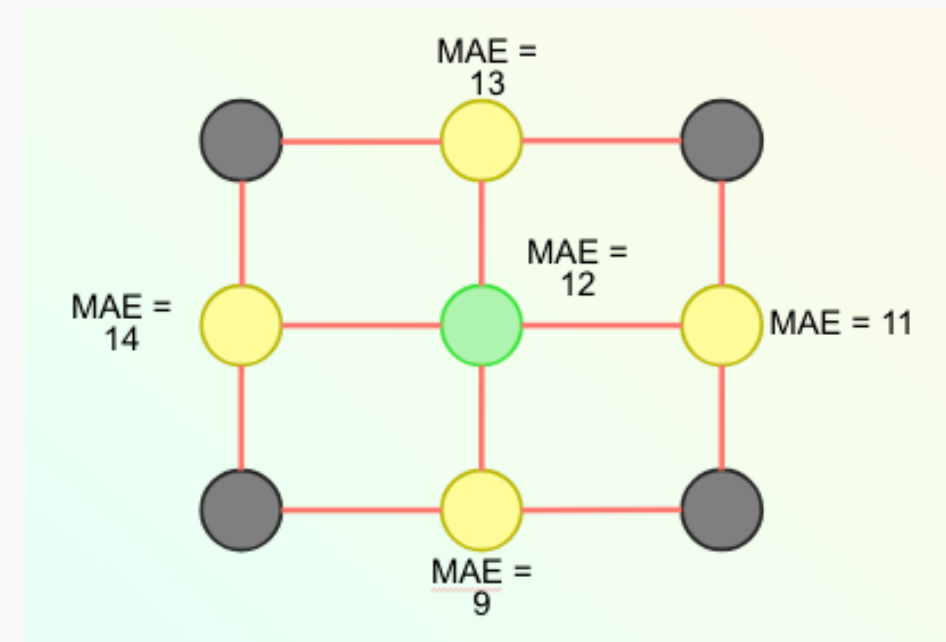
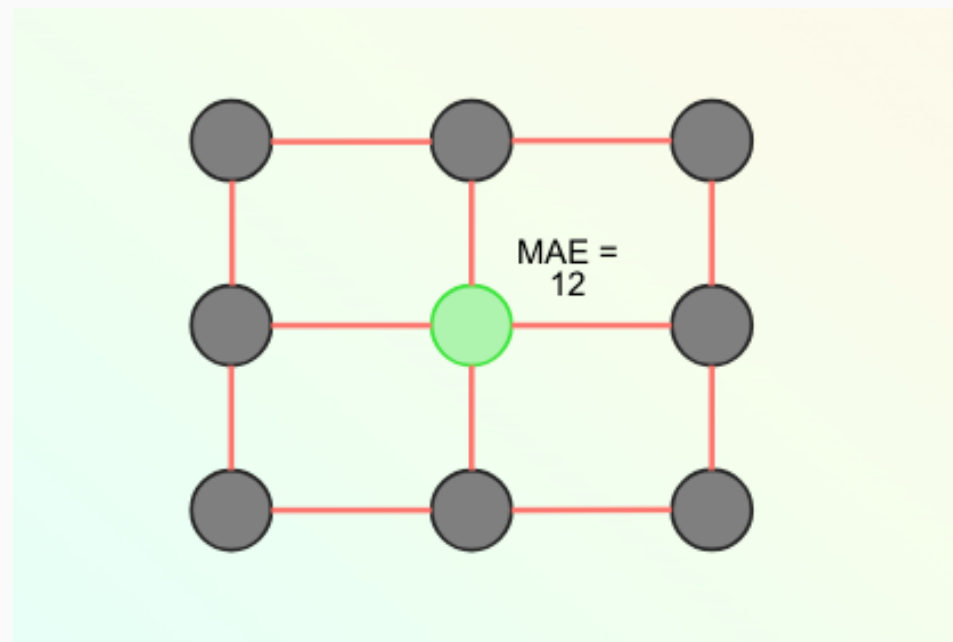
Una vez que termina la búsqueda, se elige la mitad de las combinaciones con mejores resultados. Con esa mitad, se aumenta el set de entrenamiento. Este proceso se repite sucesivamente hasta que queda una única combinación de hiperparámetros, que se considera la mejor candidata.



Vamos a practicar un
poco ...

***MÉTODOS MÁS
AVANZADOS***

Métodos avanzados: gradiente desc./asc.



Con el método del **gradiente descendente / ascendente** podemos explorar el espacio sin tener que ir por todos los parámetros exhaustivamente, sino con algo de inteligencia.

El problema de este método es que se depende de dónde se empiece, es fácil de caer en mínimos locales dado la complejidad del espacio de hiperparámetros (se puede usar gradiente estocástico para remediar), y que es difícil de paralelizar (aunque es posible múltiples comienzos).

Métodos avanzados: búsqueda bayesiana

Probar todas las combinaciones posibles puede ser muy costoso, especialmente cuando hay muchos hiperparámetros o cuando entrenar el modelo lleva mucho tiempo.

La **búsqueda bayesiana** intenta resolver este problema de manera inteligente.

Aprende de las pruebas anteriores para decidir qué combinación evaluar a continuación.

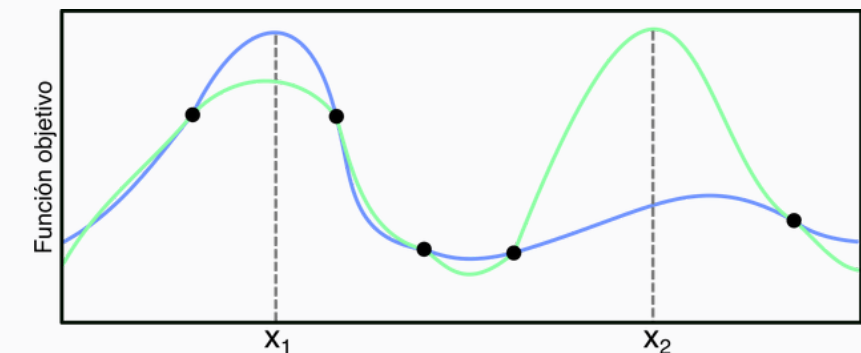
La **búsqueda bayesiana** es una estrategia secuencial para optimización de funciones de caja negra, es decir, funciones cuya forma desconocemos.

Métodos avanzados: búsqueda bayesiana

¿Cuál es el propósito de la búsqueda bayesiana en la optimización de hiperparámetros?

La búsqueda bayesiana es un **problema de optimización de una función objetivo**. La función objetivo representa el desempeño del modelo de acuerdo a una **métrica** (accuracy, F1-score, etc.) en función de sus hiperparámetros.

función objetivo(hiperparámetros) → desempeño



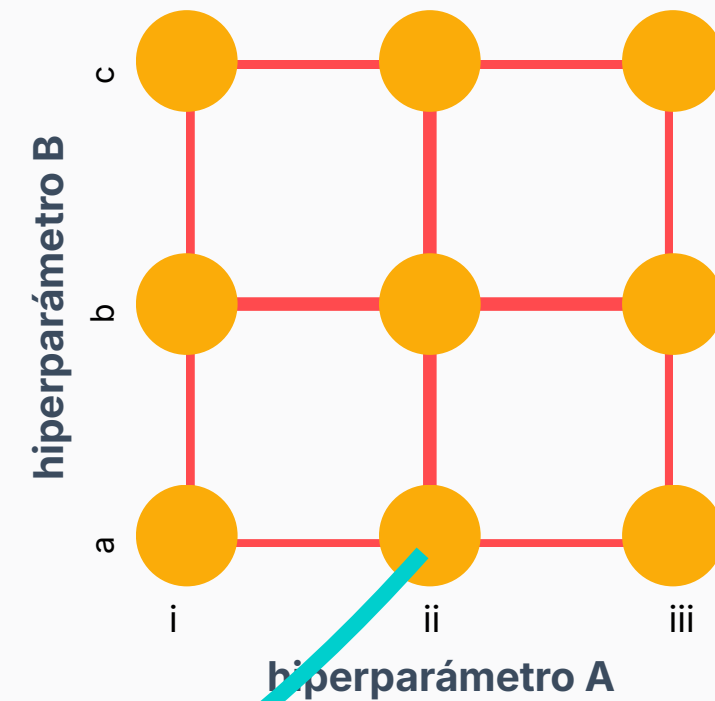
Lo que queremos es encontrar la combinación de hiperparámetros que maximiza (o minimiza) la función (es decir, la métrica).

Métodos avanzados: búsqueda bayesiana

La búsqueda bayesiana es un proceso iterativo:

1. Prueba una combinación de hiperparámetros.
2. Observa el desempeño obtenido.
3. Aprende de ese resultado.
4. Decide cuál es la siguiente combinación a probar para seguir mejorando.

Ejemplo: (ii, a) → accuracy 0.85



Métodos avanzados: búsqueda bayesiana

El algoritmo intenta equilibrar dos estrategias:

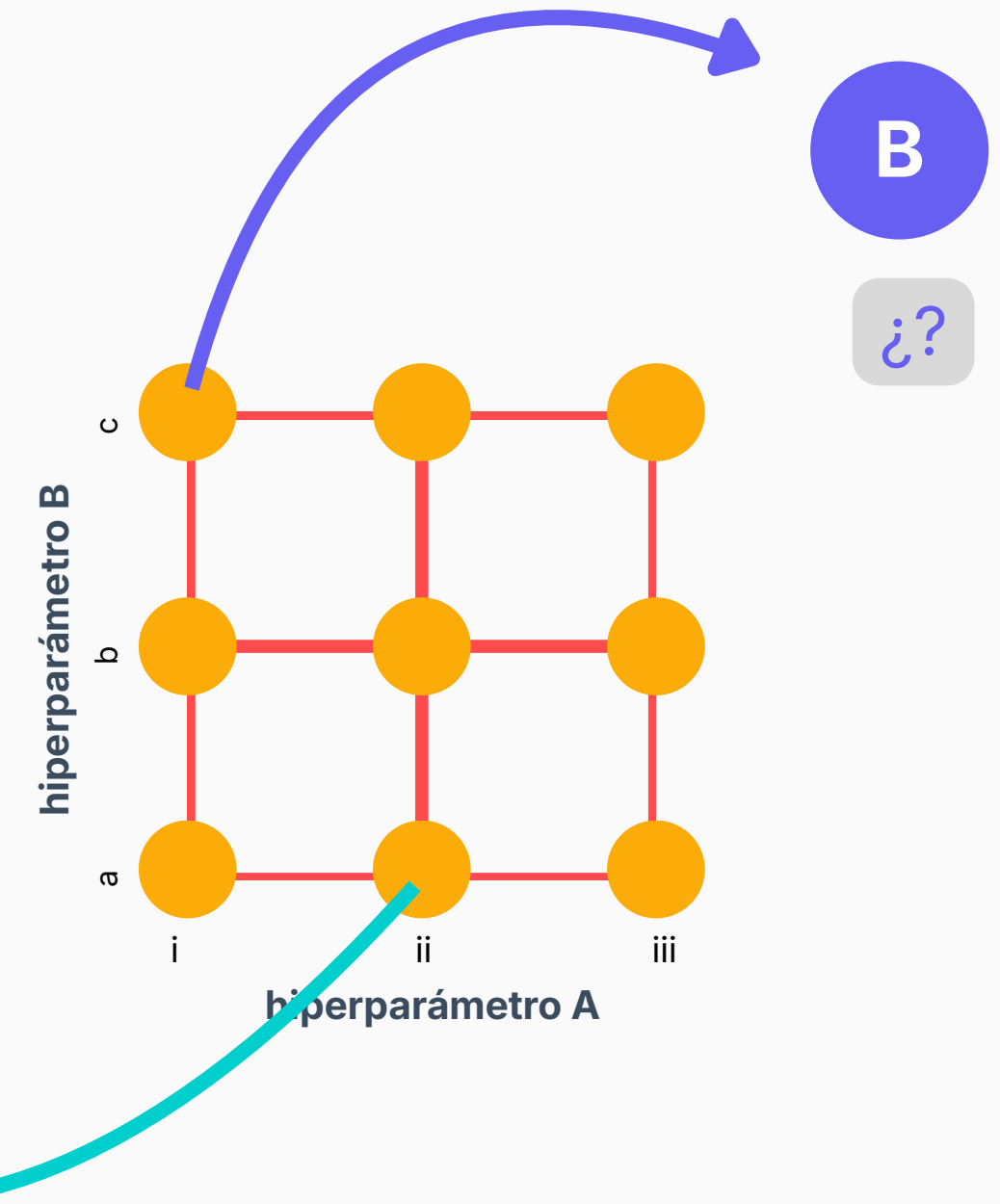
- **Explotación**: probar configuraciones similares a las que ya dieron buenos resultados.
- **Exploración**: probar configuraciones en zonas donde todavía existe mucha incertidumbre.

A

(ii, a) → accuracy 0.85

Ejemplo:

- la combinación A parece prometedora → explotar porque la función tengamos buenos resultados.
- sobre la combinación B tenemos poca información → explorar porque la incertidumbre aquí es muy grande.



Métodos avanzados: búsqueda bayesiana

La optimización bayesiana consta de dos componentes:
un **modelo probabilístico** y una **función de adquisición**.

Modelo probabilístico

- Es una distribución a priori que estima cómo se comporta la función objetivo.
- El modelo intenta aproximar la función objetivo (cuya forma real desconocemos) a partir de las evaluaciones realizadas hasta el momento.
- Se actualiza después de cada evaluación.

Ejemplos:

- Modelo gaussiano.
- Estimador Tree-Parzen (TPE).

Métodos avanzados: búsqueda bayesiana

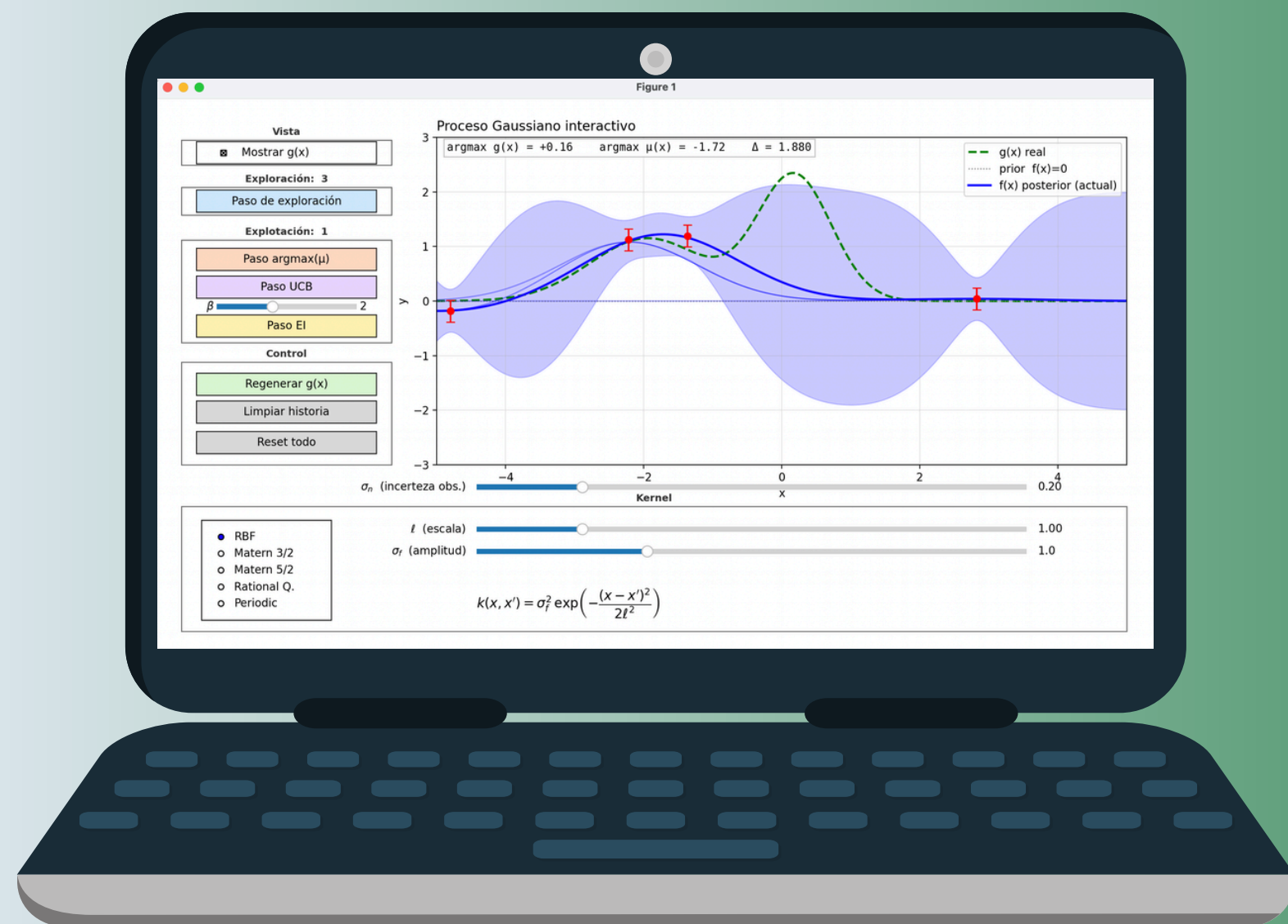
La optimización bayesiana consta de dos componentes:
un **modelo probabilístico** y una **función de adquisición**.

Función de adquisición

- En base a la información que tiene hasta el momento (modelo probabilístico) decide cuál será la próxima combinación a evaluar.
- Buscar equilibrar:
 - explotación (sugerir combinaciones en base al rendimiento).
 - exploración (sugerir combinaciones en base a la incertidumbre).

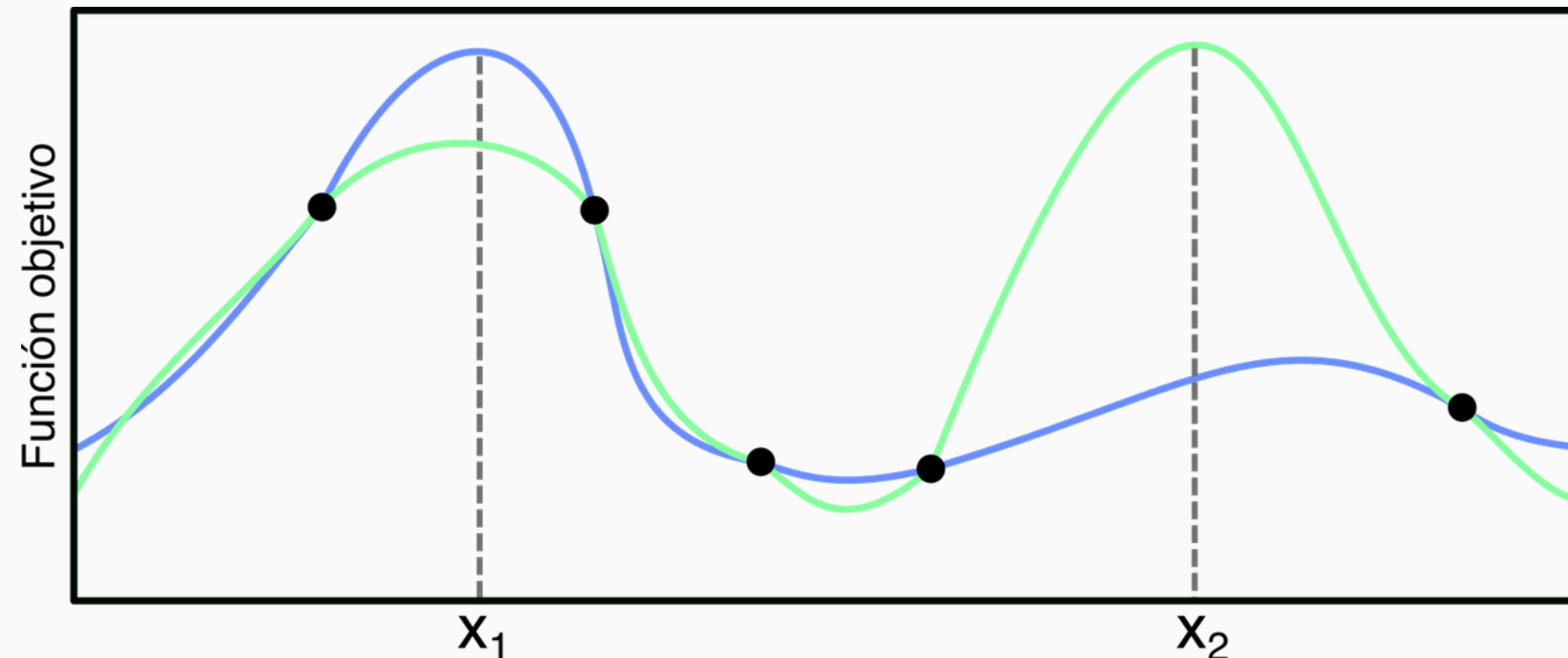
Ejemplos:

- *Expected-Improvement* (EI)



Vamos a practicar un
poco ...

Métodos avanzados: búsqueda bayesiana



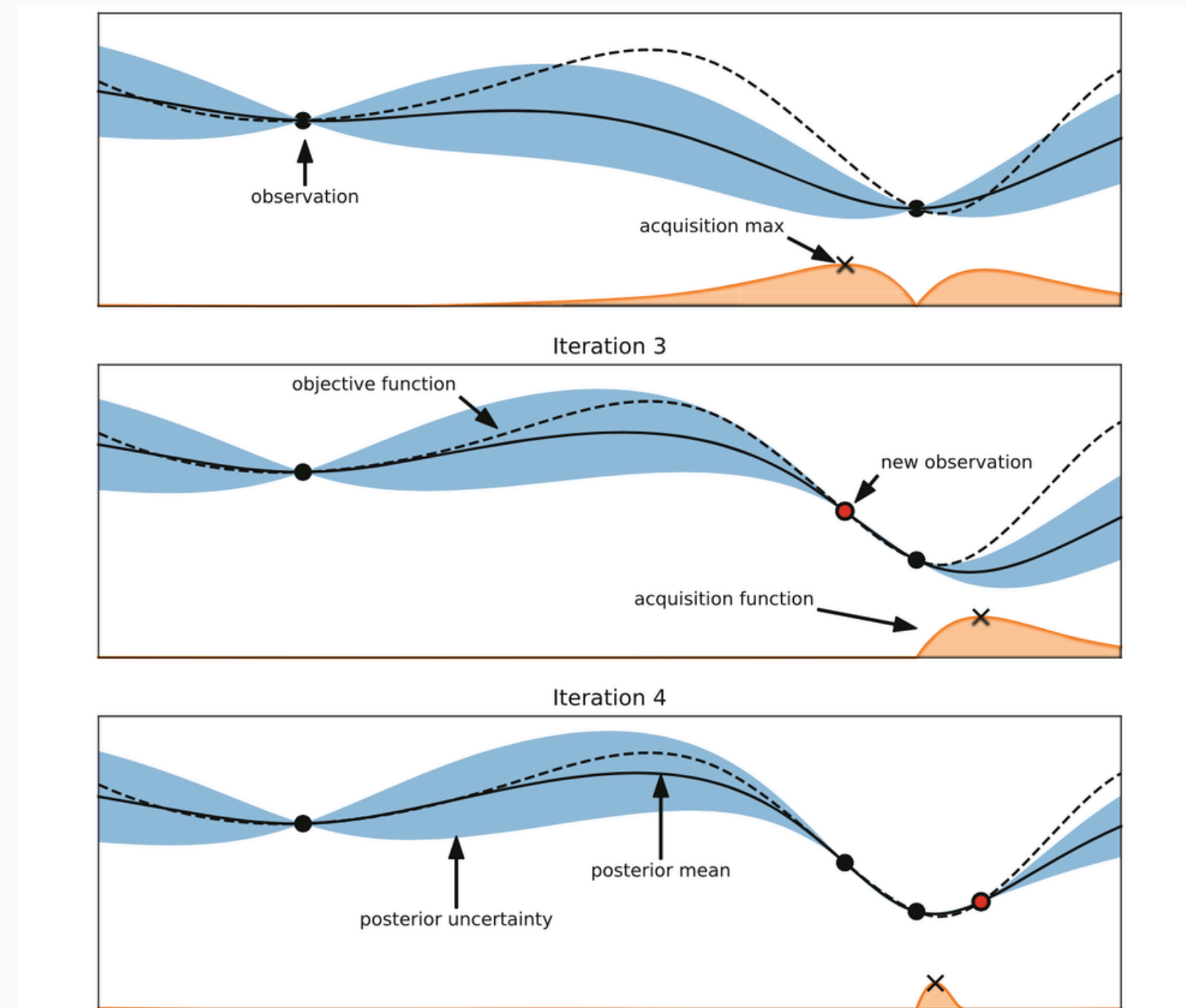
En cada iteración:

- el **modelo probabilístico** (dist. a priori) es ajustado con las observaciones que se tienen hasta ese momento
- la **función de adquisición** podría sugerir:
 - valores cerca de x_1 , porque la función probablemente devolverá un valor alto (explotación).
 - valores cerca de x_2 , porque la incertidumbre aquí es muy grande (exploración).

Métodos avanzados: búsqueda bayesiana

Ejemplo: Varias iteraciones de la optimización bayesiana

La función de adquisición es alta donde el modelo predice un objetivo alto (explotación) y donde la incertidumbre de la predicción es alta (exploración).





Vamos a practicar un
poco ...

FRAMEWORK DE BÚSQUEDA

Framework de búsqueda



O P T U N A

Optuna es un framework para la optimización de hiperparámetros que nos ofrece técnicas más avanzadas de búsqueda.

Para mejorar la eficiencia emplea dos mecanismos:

- **Selección** de combinaciones de hiperparámetros que pueden dar buenos resultados.
- **Podado** (*pruning*) de configuraciones de hiperparámetros poco prometedoras.

Framework de búsqueda



OPTUNA

Optuna funciona de manera iterativa:

1. Prueba una combinación de hiperparámetros.
2. Observa el desempeño obtenido.
3. Aprende de ese resultado.
4. Propone nuevas combinaciones prometedoras (**selección**).
5. Descarta tempranamente las configuraciones que parecen tener bajo desempeño (**podado**).

El proceso se repite hasta encontrar una buena configuración.

Framework de búsqueda



O P T U N A

Selección de hiperparámetros

Para poder seleccionar, Optuna utiliza **algoritmos de optimización** que aprenden de los resultados obtenidos durante la búsqueda.

Entre los más utilizados se encuentran:

- **TPE** (*Tree-structured Parzen Estimator*). Es el algoritmo por defecto.
- CMA-ES (*Covariance Matrix Adaptation Evolution Strategy*).
- Proceso gaussiano
- Grid search
- etc.

https://optuna.readthedocs.io/en/stable/tutorial/10_key_features/003_efficient_optimization_algorithms.html#efficient-optimization-algorithms

Framework de búsqueda

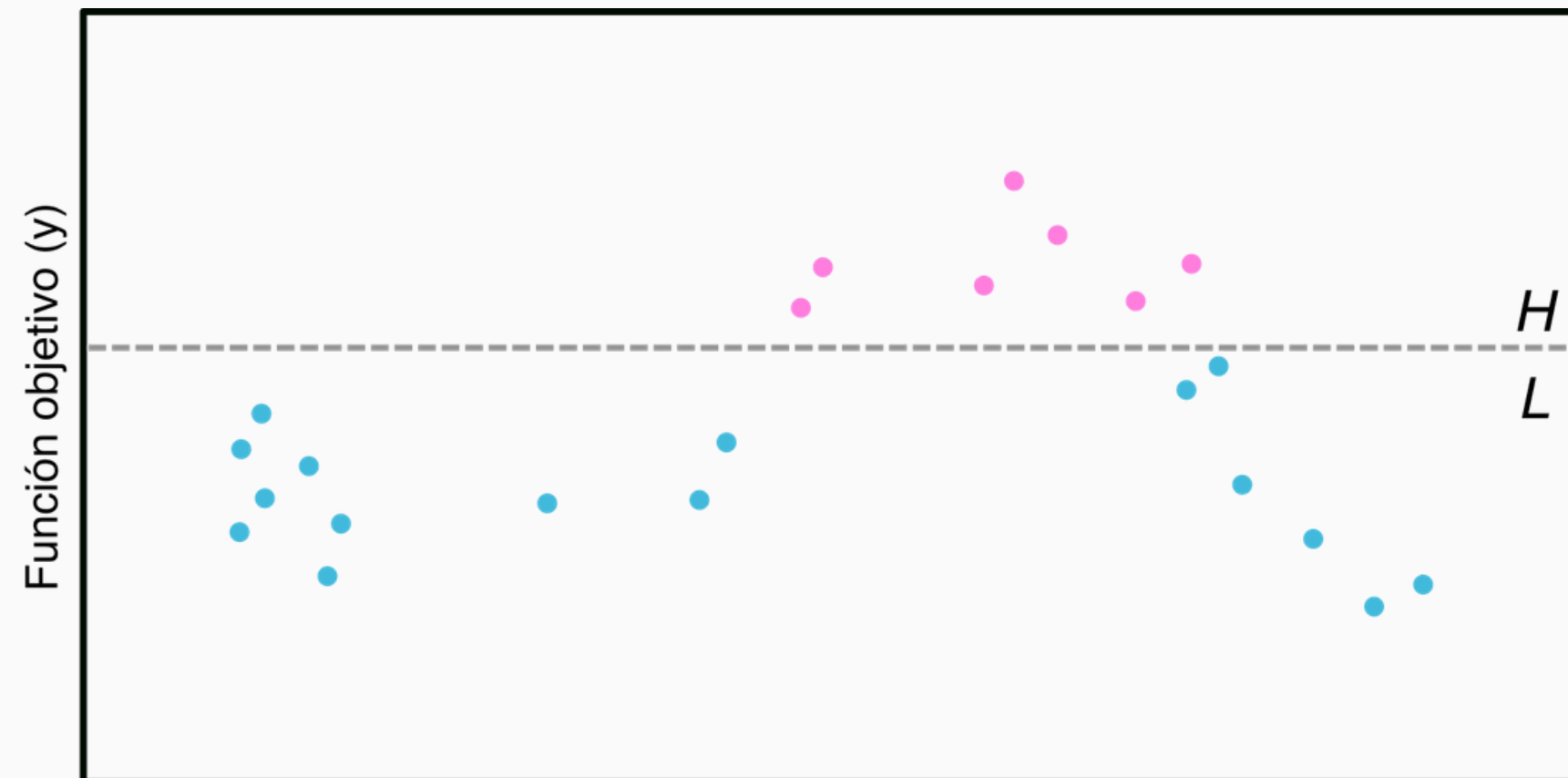


O P T U N A

Selección: estimador TPE (*Tree-structured Parzen*)

TPE separa las configuraciones de hiperparámetros ya evaluadas en dos grupos, según si caen por debajo o por encima de algún cuantil fijo:

- ***H***: Configuraciones que obtuvieron buenos resultados (valores altos de la función objetivo).
- ***L***: Configuraciones que obtuvieron malos resultados (valores chicos de la función objetivo).



Framework de búsqueda



O P T U N A

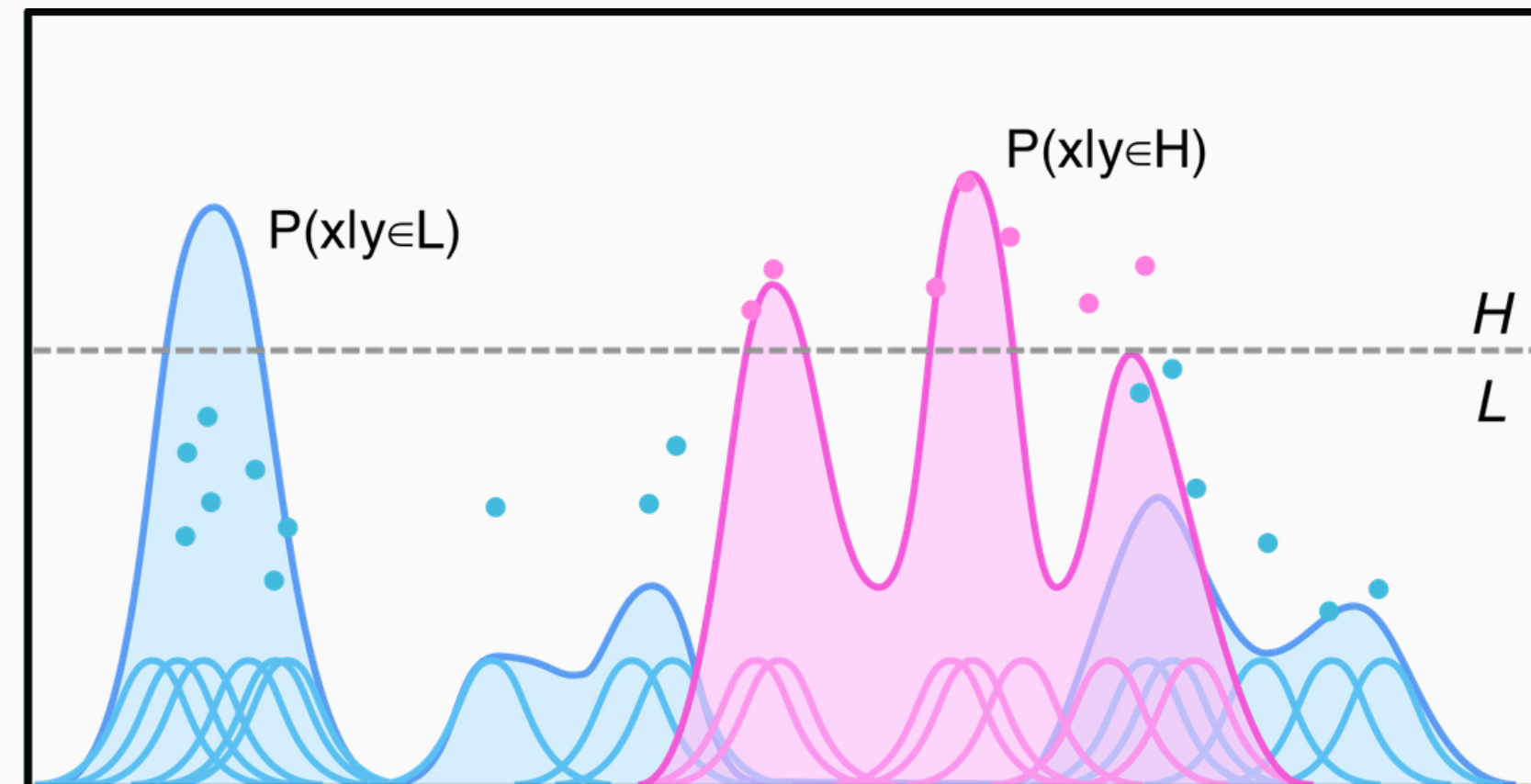
Selección: estimador TPE (*Tree-structured Parzen*)

Se estima la densidad de probabilidad de L y H asumiendo una distribución normal en cada dato observado y sumando todas las distribuciones.

Se construyen dos modelos probabilísticos:

$$P(x | y \in H)$$

$$P(x | y \in L)$$



Framework de búsqueda



O P T U N A

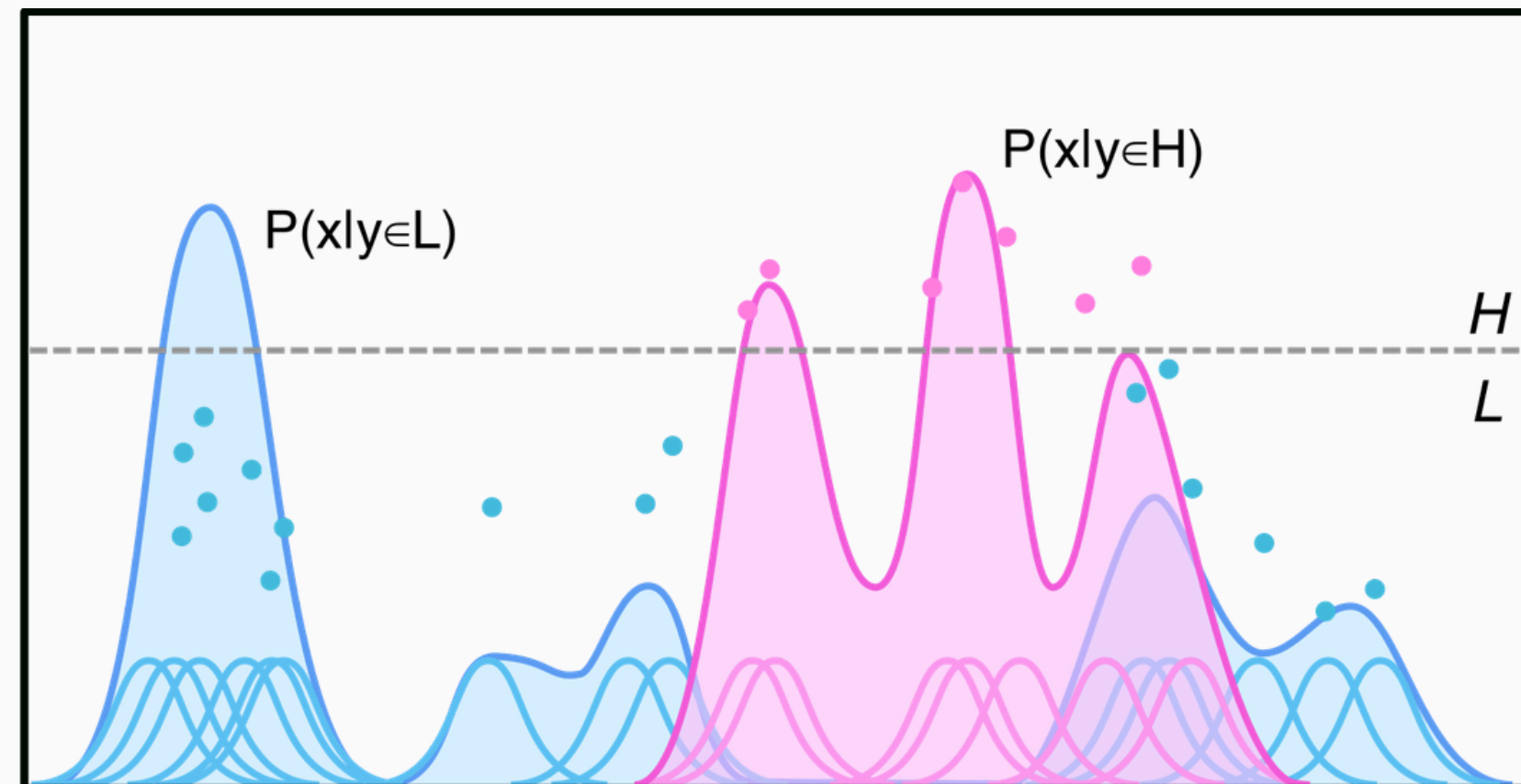
Selección: estimador TPE (*Tree-structured Parzen*)

Intuitivamente lo que buscamos son configuraciones de hiperparámetros que sean:

- muy probables bajo la distribución buena (que se parezcan a las que funcionaron bien).
- no muy probables bajo la distribución mala (que sean distintas de las que funcionaron mal).

La **función de adquisición** elige N muestras que maximicen:

$$\frac{P(x | y \in H)}{P(x | y \in L)}$$

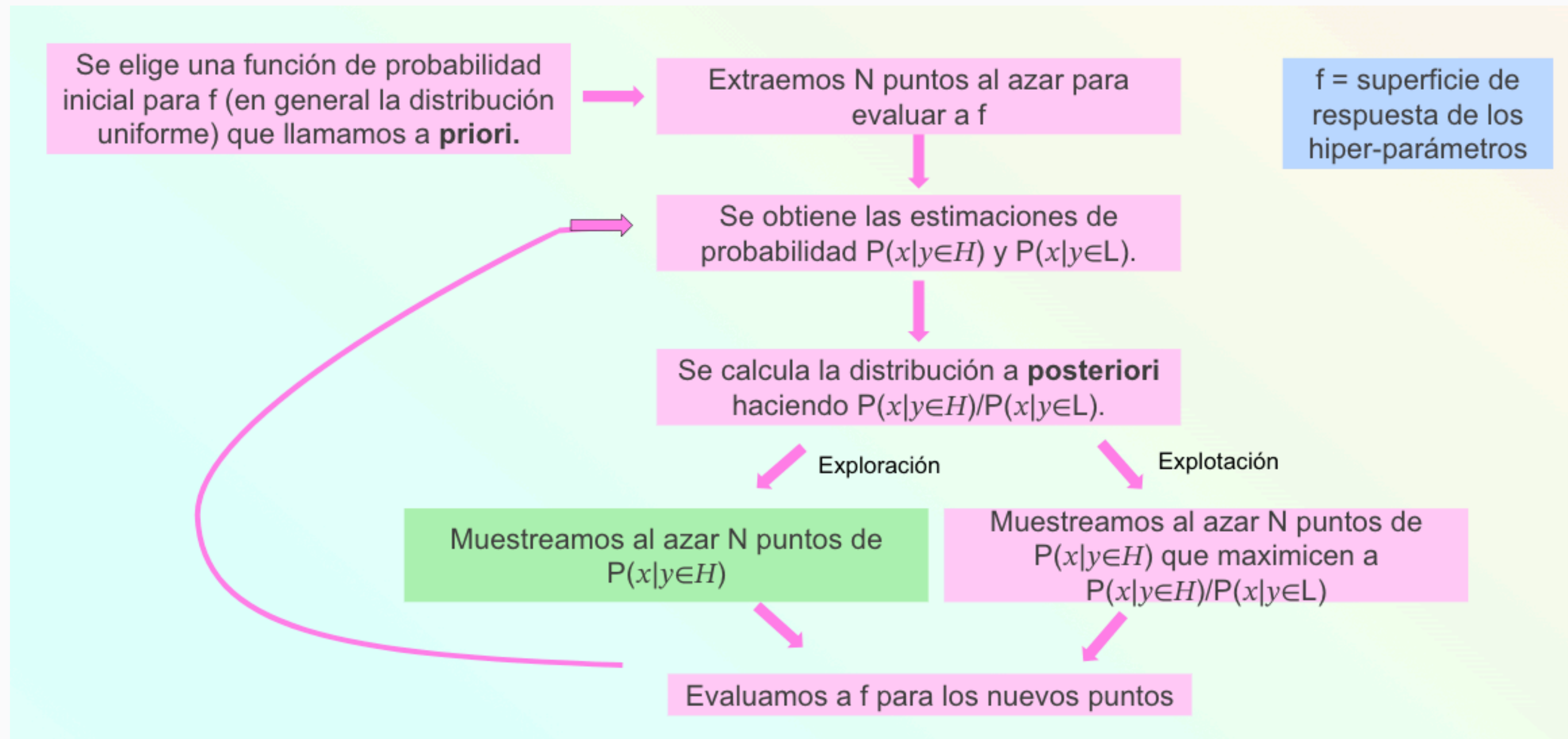


<https://proceedings.neurips.cc/paper/2011/hash/86e8f7ab32cfd12577bc2619bc635690-Abstract.html>



Framework de búsqueda

Selección: estimador TPE (*Tree-structured Parzen*)



Framework de búsqueda



OPTUNA

Podado

El podado permite reducir el espacio de búsqueda, determinando anticipadamente qué combinación de hiperparámetros no va a dar buenos resultados.

Optuna ofrece diferentes algoritmos de podado:

- reducción a la mitad sucesiva (*Successive Halving*).
- podado por mediana
- podado por umbral
- etc.

https://optuna.readthedocs.io/en/stable/tutorial/10_key_features/003_efficient_optimization_algorithms.html#pruning-algorithms



Vamos a practicar un
poco ...

En la próxima clase veremos ...

Máquinas de vectores de soporte. Maximal Margin Classifier. Clasificador de Vector de soportes. Máquinas de vectores de soporte en regresión.